Deusto
Facultad de Ingeniería
Ingeniaritza Fakultatea

Grado en Ingeniería Informática

Informatikako Ingeniaritzako gradua

Proyecto fin de grado

Gradu amaierako proiektua

Design and implementation of a safe driving system using reinforcement learning techniques

Isaac Esteban Herbozo Valladares

Directora: Rebeca Cortazar Goicoechea

Bilbao, julio de 2026

Deusto
Facultad de Ingeniería
Ingeniaritza Fakultatea

Grado en Ingeniería Informática

Informatikako Ingeniaritzako gradua

Proyecto fin de grado

Gradu amaierako proiektua

Design and implementation of a safe driving system using reinforcement learning techniques

Isaac Esteban Herbozo Valladares

Directora: Rebeca Cortazar Goicoechea

Bilbao, julio de 2026

Abstract

This work presents CALA-SafeRL, a modular framework for Safe Reinforcement Learning applied to autonomous driving in the CARLA simulator. The central challenge addressed is how to train a Proximal Policy Optimization (PPO) agent that learns to drive under realistic traffic conditions without catastrophic failures during the learning process itself. The framework is structured as a chain of Gymnasium wrappers, giving the agent access to two interchangeable safety shields, a dense reward shaper and the simulator, all of which can be combined with minimal coupling. The first shield performs a lightweight threshold-based check on semantic LIDAR readings together with CARLA's Waypoint API. The second implements an adaptive-horizon strategy, predicting the vehicle's future trajectory with a kinematic bicycle model calibrated for a Tesla Model 3 and selecting the least invasive corrective action from a prioritised candidate set. The agent's observation space has 739 dimensions, combining three semantic LIDAR channels with lane, lane-marking, vehicle and route features derived from the Waypoint API. Training stability is improved through running-mean normalisation of observations, return scaling, Generalised Advantage Estimation, clipped surrogate updates with a target-KL early stop, and a performance-driven curriculum that progressively increases traffic density. The framework is accompanied by a live metrics pipeline built on SQLite with a Streamlit dashboard. Experiments are conducted on the Town04 highway map and systematically compare the three shielding configurations against baseline PPO. The resulting system is open, reproducible and can serve as a foundation for further research into safety-aware policy learning.

Contents

List of Figures

List of Tables

1. INTRODUCTION

1.1. CONTEXT AND MOTIVATION

Road transport is one of the dominant causes of avoidable injury and death worldwide. According to the World Health Organization, approximately 1.19 million people die every year as a direct consequence of road traffic crashes, with human error being the leading contributing factor [1]. This figure has motivated a sustained effort, both in industry and in academia, to automate vehicle control up to the point where the driver can be completely displaced from the control loop. The SAE J3016 standard formalises this progression in six levels, from no automation (Level 0) to full self-driving with no operational design domain restrictions (Level 5) [2]. While most commercially deployed systems remain at Level 2, research platforms increasingly target Levels 4 and 5, where the vehicle must cope with the full variety of traffic situations, weather conditions and road topologies encountered in real-world operation.

Among the techniques that have received growing attention for high-autonomy driving, Reinforcement Learning (RL) [3] occupies a distinctive position. Classical model-predictive and rule-based controllers require a hand-crafted description of every situation and an explicit control law. In contrast, an RL agent learns a policy end-to-end by interacting with an environment and maximising a scalar reward signal. Recent surveys of deep RL for autonomous driving report promising results on lane keeping, overtaking, intersection handling and dense urban scenarios [4, 5].

The main obstacle to deploying these systems, however, is not their peak performance but their behaviour *during learning*. Vanilla RL relies on stochastic exploration, which is incompatible with the safety requirements of a vehicle operating on public roads where even a single exploratory collision is unacceptable. The research field generally referred to as *Safe Reinforcement Learning* (Safe RL) addresses this tension through multiple strategies [6]: encoding constraints in the reward function, constraining the policy optimisation problem itself or intercepting unsafe actions with a runtime verifier known as a *safety shield* [7]. This project belongs to the latter family: a PPO agent [8] is trained with two alternative shields that monitor its proposed actions, reason about their near-future consequences, and substitute them when the resulting trajectory is judged unsafe. The shield layer runs entirely during training, so safety violations are not merely discouraged by the reward function but actively prevented at the actuator.

1.2. PROBLEM STATEMENT

The central problem addressed in this thesis can be stated as follows. A vehicle, represented as an RL agent, drives inside the CARLA simulator [9]. The agent observes a partial representation of the environment through virtual sensors and a limited number of kinematic variables, and must produce low-level control commands (steering, throttle and brake). The goal is to learn a policy that

1. *Maximises* a task reward that encodes progress, speed compliance and lane keeping,
2. *Minimises* unsafe events such as collisions, off-road excursions and stalls, and
3. *Tolerates* the shift in traffic density that occurs as the scenario is progressively complicated during training, and
4. *Generalises* beyond the shield, so that the learned policy remains safe even when the shield is removed at deployment.

The research question that guides the work is then: *can a lightweight, interchangeable safety-shield layer, combined with a dense reward shaper and a performance-driven curriculum, train a PPO agent that is competitive with unconstrained PPO in terms of reward while keeping crash and off-road rates close to zero during learning?* A second question, which emerges from the first, is whether the shield (the very mechanism that enforces safety during training) induces a *dependence* that the agent cannot shed, and if so, how that dependence can be measured and reduced. This question is taken up in Chapter 9.

1.3. OBJECTIVES

General objective

To design, implement and empirically analyse a modular Safe RL framework for autonomous driving on the CARLA simulator, combining a PPO agent with interchangeable safety shields, a dense reward shaper and a curriculum manager.

Specific objectives

SO1. Build a Gymnasium-compatible wrapper of the CARLA Python API that exposes a 739-dimensional observation space (three semantic LIDAR channels plus lane-marking, lane, vehicle and route features) and a continuous two-dimensional action space, running in synchronous mode.

SO2. Implement a PPO agent with generalised advantage estimation, clipped surrogate updates, value clipping, a target-KL early stop, cosine learning-rate annealing and online normalisation of the vector observation block.

SO3. Develop a basic safety shield that combines LIDAR thresholds with information from CARLA's Waypoint API and applies a proportional correction to steering and brake whenever the agent's action is deemed unsafe.

SO4. Develop an adaptive-horizon safety shield that classifies the current situation into three risk levels, predicts the vehicle's trajectory with a kinematic bicycle model, and selects the least invasive corrective action from a priority-ordered candidate set.

SO5. Design a dense reward shaper that combines the sparse CARLA rewards with components for velocity tracking, lane centring, heading alignment, smoothness, progress, idle avoidance and lane-change tolerance.

SO6. Build a performance-driven curriculum that progressively introduces non-playable vehicles (NPCs) into the scenario, with automatic rollback to earlier stages on performance collapse.

SO7. Provide a live metrics pipeline based on SQLite (WAL mode) and a Streamlit dashboard so that each training run can be inspected while it is still in progress.

SO8. Measure and reduce the agent's *dependence* on the shield by instrumenting a shield-off evaluation probe and studying two strategies: a Large-Language-Model meta-controller that loosens the shield over training, and a behavioural-cloning scheme that turns the shield into a teacher, to train a policy that remains safe when the shield is removed at deployment.

SO9. Investigate whether the residual collision gap left after weaning can be closed with a dense reward *learned* from a Large Language Model's safety preferences (a Motif-style preference reward delivered as policy-invariant potential-based shaping), trained over a manufactured exposure curriculum, and report the outcome (positive or negative) under controlled, de-confounded conditions.

1.4. SCOPE AND LIMITATIONS

The scope of this project is deliberately bounded to keep the contribution verifiable and reproducible within the duration of a Bachelor's thesis.

- The simulator version is *CARLA 0.9.16*. Compatibility with earlier or future versions is not guaranteed.
- The main benchmark map is *Town04*, a highway-oriented layout that is well suited for lane-keeping and overtaking experiments. Urban maps (Town01–Town03) and the maximum difficulty map (Town05) are supported by the framework but not systematically evaluated.
- Two shielding strategies are developed and compared: *basic* (threshold + Waypoint API correction) and *adaptive-horizon* (trajectory prediction with a kinematic bicycle model). Alternatives based on reachability analysis, model-predictive safety filters or formal verification are left for future work.
- The vehicle model used by the adaptive shield is the kinematic bicycle model [10]. A dynamic model with tyre slip is not considered because, at the speeds and lateral accelerations encountered in the experiments, the kinematic approximation is a reasonable trade-off between fidelity and computational cost.
- Perception is based on semantic LIDAR plus CARLA's Waypoint API. Camera-based perception, sensor fusion and learned occupancy grids are not part of the observation space.
- The framework trains a *single ego vehicle*. Multi-agent safe RL is an explicit direction for future extensions.

1.5. METHODOLOGY OVERVIEW

The project follows an iterative engineering methodology structured around four phases (design, implementation, training and analysis) that are revisited in successive loops as new empirical evidence becomes available.

1. **Design.** The software architecture is expressed as a layered wrapper chain on top of Gymnasium. Each layer (environment, shield, reward shaper) has a single responsibility and a well-defined interface, so that components can be swapped or reconfigured without touching the others.
2. **Implementation.** The Python code base is organised into self-contained modules (`src/CARLA/Env`,

`src/PPO`, `src/Adaptative_Shield`, `src/reward_shaper.py`, `src/curriculumManager.py`, `src/Metrics`). Automated style checks with Ruff and targeted unit tests on the reward shaper prevent regressions.

3. **Training.** Each experimental configuration is launched via `main_train.py` with explicit command-line flags. The agent writes metrics to a per-run SQLite database and checkpoints every 200 episodes.

4. **Analysis.** The `dataVisualizer.py` Streamlit dashboard is used to inspect reward curves, safety rates, curriculum transitions and shield activations during and after training. Observed pathologies are translated into new design iterations.

1.6. DOCUMENT STRUCTURE

The remainder of this document is organised as follows.

Chapter 2 frames the work as a managed engineering project: the work breakdown structure, the schedule and Gantt chart, the milestones, the human-resources plan and the risk register.

Chapter 3 estimates the cost of the project as a professional undertaking, with labour distributed across the roles assumed by the author, amortised equipment, energy and overhead.

Chapter 4 reviews the theoretical background. It introduces Markov Decision Processes, derives the PPO objective from the policy gradient, describes the Generalised Advantage Estimator, surveys the main families of Safe RL, formalises the kinematic bicycle model and summarises the features of the CARLA simulator that are relevant to this project.

Chapter 5 captures the functional and non-functional requirements that the framework must satisfy, together with the technical constraints imposed by the simulator, the hardware and the Bachelor-thesis timeline.

Chapter 6 describes the system design: the wrapper chain, the structure of the observation and action spaces, the Actor–Critic network, the two safety shields, the reward shaper and the curriculum manager.

Chapter 7 details the implementation: technology stack, project layout, key algorithms in pseudocode, training and evaluation pipelines, and reproducibility considerations.

Chapter 8 presents the experimental protocol and a structured template to report the empirical results obtained on Town04.

Chapter 9 studies the agent's dependence on the shield: it introduces the shield-off evaluation probe, reports a negative result for a Large-Language-Model meta-controller that loosens the shield, and a positive result for a behavioural-cloning scheme that turns the shield into a teacher of unaided safe driving.

Chapter 10 investigates whether the residual collision gap left after weaning can be closed with a dense reward *learned* from a Large Language Model's safety preferences, delivered as policy-invariant potential-based shaping over a manufactured exposure curriculum, and reports a de-confounded negative result.

Chapter 12 assesses the ethical dimension of the work: the choice of a deterministic safety layer over a purely learned policy, and its alignment with engineering codes of conduct, social impact and the University's values.

Chapter 11 summarises the contributions, evaluates the fulfilment of the objectives enumerated above, and proposes future work.

Appendices A, B, C, D and E provide the complete list of default hyperparameters, a full breakdown of the observation space, a reproducibility guide, mathematical formulae, and a breakdown on the usage of AI in the research, respectively.

2. PROJECT PLANNING AND MANAGEMENT

The previous chapter framed CALA-SafeRL as a research problem. This chapter frames it as a *managed engineering project*. It maps the work onto the five canonical project-management stages (Section 2.1) and decomposes it into a work breakdown structure with an effort estimate (Section 2.2). It then records the milestones (Section 2.3), the task precedences and the resulting schedule and Gantt chart (Section 2.4), the human-resources plan (Section 2.5), and, finally, the risk register and the deviations that actually occurred (Section 2.6). The plan presented here is the *baseline* against which execution was monitored. Because the engineering methodology is iterative (Section 1.5), the baseline was re-planned at the end of each design $\rightarrow$ implementation $\rightarrow$ training $\rightarrow$ analysis loop rather than followed rigidly.

2.1. PROJECT-MANAGEMENT APPROACH

The project is organised around the five standard project-management stages (definition, planning, execution, monitoring and control, and closure), adapted to a research-and-development context in which the requirements are partly discovered during execution. Table 2.1 maps each stage to the concrete activities and artefacts of CALA-SafeRL. A point worth highlighting is the *monitoring and control* stage: in a conventional software project it is realised through status reports, whereas here it is realised through engineering instrumentation: the live metrics dashboard (Section 6.8), the shield-off probe (Section 9.2) and the KL and curriculum gates, which provide continuous, quantitative feedback on whether the work is converging.

Table 2.1.: The five project-management stages mapped to concrete activities and artefacts of CALA-SafeRL.

Stage	**Activities in this project**	**Artefacts**
Definition	Problem framing, research question, scope and objectives	Sections 1.1–1.4
Planning	Work breakdown, schedule, resource and risk plans	This chapter
Execution	Iterative design, implementation, training and analysis loops	Chapters 6–9
Monitoring & Control	Live metrics dashboard, shield-off probe, KL and curriculum gates, rolling re-planning	Sections 6.8, 9.2
Closure	Final evaluation, memoir, defense, reproducibility package	Chapter 11, Appendix C

2.2. WORK BREAKDOWN STRUCTURE AND EFFORT ESTIMATION

The project is decomposed into eleven work packages (WP0–WP10). WP0 (management) and WP10 (writing) run continuously alongside the technical packages, and the remaining packages map directly onto the specific objectives SO1–SO8 of Section 1.3. Table 2.2 lists each package, its main tasks, the objective(s) it serves and the estimated effort. The total of $550\,\mathrm{h}$ is the author's post hoc reconstruction of the effort actually invested. It is consistent with, and somewhat above, the nominal workload of a 12-ECTS Bachelor's thesis (≈ 300–$360\,\mathrm{h}$), the difference reflecting the research-heavy nature of the shield-dependence study (WP9) and the large number of training runs required (WP8).

Table 2.2.: Work breakdown structure and effort estimation. The linked specific objectives (SO) are those of Section 1.3.

ID	Work package	Main tasks	Obj.	Hours
WP0	Project management	Planning, scheduling, supervision meetings, risk tracking, version control	–	35
WP1	Research & state of the art	RL and Safe-RL literature, shielding, CARLA, bicycle model	–	50
WP2	Environment & wrappers	`CarlaEnv`, synchronous mode, semantic LIDAR, Waypoint features, 739-d observation	SO1	60
WP3	PPO agent	Actor–Critic network, GAE, clipped surrogate, KL early stop, normalisation	SO2	55
WP4	Safety shields	Basic shield, adaptive-horizon shield, bicycle model, shield credit assignment	SO3, SO4	70
WP5	Reward shaper	Dense reward components, speed/lane gates, unit tests	SO5	30
WP6	Curriculum manager	Five-stage NPC ramp, rollback rule, in-place update	SO6	22
WP7	Metrics & dashboard	SQLite WAL logger, Streamlit/Plotly dashboard	SO7	25
WP8	Experimentation	Training runs, shield ablations, evaluation protocol, analysis	–	55
WP9	Shield-dependence study	Shield-off probe, LLM meta-controller, behavioural-cloning teacher, failure analysis	SO8	68
WP10	Thesis writing & defense	Memoir, figures, reproducibility guide, defense preparation	–	80
			Total	**550**

2.3. MILESTONES

Six milestones mark the closure of a major block of work and gate the start of the next. Table 2.3 lists them with their target dates and deliverables. Milestones M3–M5 correspond to the three empirical achievements of the project: a working shielded agent, the main shield ablation, and the resolution of the shield-dependence problem.

Table 2.3.: Project milestones.

ID	Target	Milestone / deliverable
M1	Sep 2025	Project definition and baseline plan approved
M2	Nov 2025	State of the art consolidated; core bibliography fixed
M3	Jan 2026	Full wrapper chain and PPO agent operational; first shielded training run
M4	Mar 2026	Main experimentation complete (shield ablation and curriculum)
M5	May 2026	Shield-dependence study complete (LLM negative result, BC positive result)
M6	Jun 2026	Thesis submitted and defended

2.4. TASK SCHEDULING AND GANTT CHART

The technical work packages form a mostly linear dependency chain: the environment (WP2) must exist before the agent (WP3), both are needed before the shields (WP4), and all of the components must be in place before systematic experimentation (WP8) and the dependence study (WP9). Table 2.4 records the precedences, and the resulting *critical path* is WP1 $\rightarrow$ WP2 $\rightarrow$ WP3 $\rightarrow$ WP4 $\rightarrow$ WP8 $\rightarrow$ WP9 $\rightarrow$ WP10. The reward shaper, curriculum manager and metrics pipeline (WP5–WP7) are off the critical path and were developed in parallel with the shields.

Table 2.4.: Task precedence. WP0 and WP10 run continuously alongside the technical packages.

WP	Predecessors	Relationship
WP0	–	Continuous
WP1	–	Start
WP2	WP1	Finish-to-start
WP3	WP2	Finish-to-start
WP4	WP2, WP3	Finish-to-start
WP5	WP2	Finish-to-start (parallel)
WP6	WP2	Finish-to-start (parallel)
WP7	WP2	Finish-to-start (parallel)
WP8	WP3, WP4, WP5, WP6, WP7	Finish-to-start
WP9	WP8	Finish-to-start
WP10	WP1–WP9	Continuous

Figure 2.1 shows the baseline schedule as a Gantt chart over the ten months of the project. The diamonds mark the milestones of Table 2.3. The chart makes the front-loading of research (WP1) and the back-loading of writing (WP10) explicit, and shows the two-month overlap between the close of experimentation (WP8) and the start of the dependence study (WP9) that occurred in spring 2026.

Figure 2.1.: Baseline Gantt chart of the project (September 2025 to June 2026). Bars are the work packages of Table 2.2 and diamonds are the milestones of Table 2.3. Management (WP0) and writing (WP10) span the whole project, and the technical critical path runs WP2, WP3, WP4, WP8 and WP9.

2.5. HUMAN-RESOURCES PLAN

Although the project was carried out by a single author, the work spanned several professional *roles*, each with a distinct skill profile and, in the budget of Chapter 3, a distinct cost rate. Table 2.5 distributes the $550\,\mathrm{h}$ of Table 2.2 across these roles. The RL/ML Engineer role dominates, as expected for a reinforcement-learning thesis, but the Software Engineer role is also substantial because the CARLA wrapper, the metrics pipeline and the dashboard are non-trivial systems-engineering tasks.

Table 2.5.: Roles assumed by the author and their effort. The hourly rates are used in the budget (Chapter 3).

Role	Main responsibilities	Hours	Rate (€/h)
Project Manager	Planning, scheduling, risk management, supervision liaison	45	40
RL/ML Engineer	RL research, PPO, shield logic, reward design, results analysis	256	35
Software Engineer	CARLA wrapper, metrics pipeline, dashboard, tooling	134	30
QA/Test Engineer	Unit tests, deterministic evaluation, training-run validation	45	25
Technical Writer	Memoir, figures, documentation	70	22
Total effort		**550**	

The project also relied on a *support team* that is not part of the author's effort but is costed in Chapter 3: the project Director, who supervised the work and reviewed the memoir; the Bachelor-thesis Coordinator, responsible for the administrative process; and the three-member defense tribunal.

2.6. RISK MANAGEMENT AND DEVIATIONS

Table 2.6 is the project risk register. Because the plan is presented retrospectively, the final column records whether each risk actually materialised. Four of the six risks did, and the way they were handled is documented in the lessons learned of Section 11.3: the curriculum-change deadlock was resolved by updating the NPC count in place rather than rebuilding the environment, the `log_std` saturation was resolved by the tanh-squashed parameterisation, the shortage of compute for multi-seed runs was *accepted* and reported honestly (a single seed with seed-aligned cross-configuration evaluation), and the late-emerging LLM reward-exposure line was de-scoped to future work to protect the core objectives. The 5% contingency reserve in the budget (Chapter 3) is the financial counterpart of this risk exposure.

Table 2.6.: Risk register. The final column records whether each risk materialised during execution.

Risk	**Lik.**	**Imp.**	**Mitigation**	**Status**
CARLA server instability / crashes	High	Med	Synchronous headless mode; checkpoint every 200 episodes	Managed
Deadlock when rebuilding the env on curriculum change	Med	High	In-place `num_npc` update, no reconnection	Resolved
Training instability (`log_std` saturation)	Med	High	tanh-squashed `log_std` parameterisation	Resolved
Insufficient compute for multi-seed runs	High	Med	Seed-aligned cross-configuration evaluation; honest reporting	Accepted
Scope creep (LLM reward-exposure line)	Med	Med	De-scoped to future work; core objectives protected	De-scoped
Single-workstation hardware failure	Low	High	Frequent checkpointing; Git version control	Not hit

With the schedule, resources and risks established, Chapter 3 turns this plan into a cost estimate.

3. PROJECT BUDGET

This chapter estimates the cost of CALA-SafeRL *as if it had been carried out as a professional engineering contract*, rather than as an unpaid academic exercise. The estimate builds directly on the planning of Chapter 2: the labour cost follows the role-and-effort distribution of the human-resources plan (Table 2.5), and the contingency reserve follows the risk register (Table 2.6). Section 3.1 fixes the cost model and its assumptions, the subsequent sections cost labour, equipment, software, energy and travel, and Section 3.7 aggregates them into the total budget.

3.1. COST MODEL AND ASSUMPTIONS

The estimate rests on the following assumptions, chosen to be defensible for a project executed in Spain during the 2025 – 2026 academic year:

- **Labour** is charged per role at the rates of Table 2.5 (40 €/h Project Manager, 35 €/h RL/ML Engineer, 30 €/h Software Engineer, 25 €/h QA/Test Engineer, 22 €/h Technical Writer), reflecting Spanish market rates for a junior-to-mid professional in each speciality.
- **Equipment** is *pre-owned* and charged by linear amortisation, cost $\times$ (months used/useful life) $\times$ dedication, with a 60-month useful life for the workstation, an 11-month project span and an 80% dedication factor.
- **Electricity** is priced at 0.18 €/kWh, a representative Spanish domestic tariff for the period.
- **Indirect costs** (workspace, internet, administration) are taken as 15% of direct costs. A 5% **contingency** reserve covers the risks of Table 2.6.
- **Value-added tax** (IVA) is applied at the Spanish general rate of 21% on the base budget.

3.2. LABOUR COSTS

Labour is by far the dominant cost, as is typical for a research-and-development project whose main input is engineering time. Table 3.1 costs the author's own effort by role, and Table 3.2 costs the support team (Director, Coordinator and tribunal), whose time is a real cost of the project even though it is not borne by the author.

Table 3.1.: Internal labour cost (project team). Hours taken from the human-resources plan (Table 2.5).

Role	Hours	Rate (€/h)	Cost (€)
Project Manager	45	40	1 800.00
RL/ML Engineer	256	35	8 960.00
Software Engineer	134	30	4 020.00
QA/Test Engineer	45	25	1 125.00
Technical Writer	70	22	1 540.00
Subtotal	**550**		**17 445.00**

Table 3.2.: Support-team labour cost.

Member	Hours	Rate (€/h)	Cost (€)
Project Director (supervisor)	30	60	1 800.00
TFG Coordinator	5	50	250.00
Defense tribunal (3 members)	12	50	600.00
Subtotal	**47**		**2 650.00**

The combined labour cost is therefore $17\,445.00 + 2\,650.00 = 20\,095.00$ €.

3.3. EQUIPMENT AND MATERIALS

The project ran on a single pre-owned desktop workstation (the machine described in Section 5.3 and used for the experiments of Chapter 8). Table 3.3 charges only the fraction of its useful life consumed by the project. No consumables of note were required, and, as discussed in Section 3.4, the entire scientific software stack is open-source, so its licensing cost is zero.

Table 3.3.: Amortised equipment cost. Amortisation $=$ cost $\times$ (months used$/$useful life) $\times$ dedication.

Item	Cost (€)	Life (mo)	Used (mo)	Ded.	Amort. (€)
Workstation (i9-9900K, RTX 2080 Ti, 32 GB)	2 000.00	60	11	80%	293.33
Monitor and peripherals	350.00	72	11	80%	42.78
Subtotal					**336.11**

3.4. SOFTWARE AND AI SERVICES

A deliberate design decision was to build the framework entirely on open-source software, which keeps the licensing cost at zero and the work fully reproducible (Table 3.4). The only paid software is the AI coding-and-writing assistant whose use is disclosed in Appendix E. The local Gemma model (served through Ollama) and the free tier of Gemini incur no per-call cost, with their compute accounted for under energy (Section 3.5).

Table 3.4.: Software and AI-service costs. The scientific stack is fully open-source.

Item	**Licence / basis**	**Cost (€)**
CARLA, Gymnasium, PyTorch, NumPy, Matplotlib, Streamlit, Plotly, SQLite, Ruff, pytest	MIT / BSD-3 / Apache-2.0 / PSF / public domain	0.00
AI coding/writing assistant (≈ 20 €/mo $\times$ 8 mo)	Commercial subscription (Appendix E)	160.00
Local LLM (Ollama + Gemma), Gemini free tier	Local / free tier	0.00
Subtotal		**160.00**

3.5. ENERGY CONSUMPTION

Reinforcement-learning training is energy-intensive: each configuration requires hundreds of episodes of CARLA simulation with the policy on the GPU. Table 3.5 estimates the electricity consumed, separating heavy GPU training from lighter development and analysis.

Table 3.5.: Energy consumption. Electricity price: 0.18 €/kWh.

Use	**Hours**	**Power (kW)**	**Energy (kWh)**
GPU training (CARLA + policy)	350	0.45	157.5
Development and analysis	250	0.20	50.0
Total			**207.5**

At 0.18 €/kWh, the $207.5\,\mathrm{kWh}$ consumed amount to 37.35 €. While small in absolute terms, the figure is

reported for completeness and because it scales linearly with the number of training runs, which is the project's main computational cost driver.

3.6. TRAVEL AND ACCOMMODATION

The project was developed locally and supervised through a combination of on-campus and online meetings, so no travel or accommodation was required. This line is therefore 0.00 €.

3.7. TOTAL BUDGET

Table 3.6 aggregates the previous sections. Indirect costs are applied at 15% of direct costs, the contingency reserve at 5% of the direct-plus-indirect subtotal, and IVA at 21% of the base budget. The resulting total cost of the project is $\mathbf{30\,139.73}$ €.

Table 3.6.: Total project budget.

Concept	**Cost (€)**
Labour, project team (Table 3.1)	17 445.00
Labour, support team (Table 3.2)	2 650.00
Equipment, amortised (Table 3.3)	336.11
Software and AI services (Table 3.4)	160.00
Energy (Table 3.5)	37.35
Travel and accommodation	0.00
Direct costs	**20 628.46**
Indirect costs / overhead (15%)	3 094.27
Contingency reserve (5%)	1 186.14
Base budget (before tax)	**24 908.87**
VAT (IVA, 21%)	5 230.86
Total project budget	**30 139.73**

Two observations close the chapter. First, labour accounts for $20\,095$ € of the $20\,628$ € of direct cost (roughly 97%), which confirms that, like most research-and-development efforts, the project's value is overwhelmingly in engineering time rather than in hardware or licences. Second, the open-source software stack contributes a

licensing cost of exactly zero. Had the framework been built on commercial simulation or RL tooling, this line alone could have rivalled the hardware cost. The low marginal cost of an additional experiment (essentially only electricity, $\approx$ 0.18 €/kWh) is precisely what makes a reproducible, open Safe-RL framework such as CALA-SafeRL economically attractive as a research platform.

4. STATE OF THE ART AND THEORETICAL FRAMEWORK

This chapter reviews the concepts that underpin the rest of the document. It first formalises the Reinforcement Learning problem (Section 4.1), then introduces Proximal Policy Optimization and Generalised Advantage Estimation (Sections 4.2–4.3), surveys the Safe RL literature with a focus on shielding (Sections 4.4–4.5), introduces the kinematic bicycle model used by the adaptive shield (Section 4.6), and finishes with an overview of the CARLA simulator and related work (Sections 4.7–4.8).

4.1. REINFORCEMENT LEARNING FUNDAMENTALS

The standard formulation of Reinforcement Learning relies on Markov Decision Processes (MDPs) [3, 11]. A (discrete-time, discounted) MDP is a tuple $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, r, \gamma, \rho_0)$ where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, $P(s' \mid s, a)$ is the transition kernel, $r(s, a)$ is the reward function, $\gamma \in [0, 1)$ is the discount factor and ρ_0 is the initial-state distribution.

A stochastic policy $\pi_\theta : \mathcal{S} \to \Delta(\mathcal{A})$, parameterised by θ, induces a distribution over trajectories $\tau = (s_0, a_0, s_1, a_1, \ldots)$. The *return* is the discounted cumulative reward:

$$G(\tau) = \sum_{t=0}^{\infty} \gamma^t \, r(s_t, a_t). \tag{4.1}$$

The training objective is to maximise the expected return:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}[G(\tau)] \,. \tag{4.2}$$

Two central quantities guide optimisation. The *state-value function* $V^{\pi_\theta}(s)$ measures the expected discounted return starting from state s under policy π_θ. The *action-value function* $Q^{\pi_\theta}(s, a)$ additionally conditions on taking action a. The *advantage* $A^{\pi_\theta}(s, a) = Q^{\pi_\theta}(s, a) - V^{\pi_\theta}(s)$ measures how much better than average a given action is at state s.

4.1.1. Policy gradient theorem

Policy gradient methods directly optimise Equation (4.2) by gradient ascent on θ. The policy gradient theorem [12] provides a tractable expression for the gradient:

$$\nabla_\theta J(\theta) \;=\; \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{\infty} \nabla_\theta \log \pi_\theta(a_t \mid s_t)\; A^{\pi_\theta}(s_t, a_t)\right]. \tag{4.3}$$

In practice the advantage is replaced by a sample-based estimator $\hat{A}_t$. Modern on-policy methods such as A2C [13], TRPO [14] and PPO [8] all build on Equation (4.3), differing mainly in how they constrain the update step size.

4.2. PROXIMAL POLICY OPTIMIZATION

Vanilla policy gradient is sensitive to the step size: a single large update can move the policy so far from the data distribution that subsequent rollouts are unreliable. TRPO [14] resolves this by constraining the Kullback-Leibler divergence between old and new policy at each update, but doing so requires second-order information. PPO [8] achieves a similar trust-region effect with a simple first-order surrogate.

Let $r_t(\theta) = \pi_\theta(a_t \mid s_t) \,/\, \pi_{\theta_{\text{old}}}(a_t \mid s_t)$ be the importance-sampling ratio. The *clipped surrogate objective* is

$$\mathcal{L}^{\text{CLIP}}(\theta) \;=\; \mathbb{E}_t\Big[\min\Big(r_t(\theta)\,\hat{A}_t,\; \text{clip}\big(r_t(\theta),\,1-\epsilon,\,1+\epsilon\big)\,\hat{A}_t\Big)\Big], \tag{4.4}$$

where $\epsilon = 0.2$. As Figure 4.1 illustrates, the clipping prevents the ratio $r_t(\theta)$ from drifting far from one: for a positive advantage the policy cannot over-exploit a good action beyond $1+\epsilon$, and for a negative advantage it cannot over-penalise beyond $1-\epsilon$. This keeps updates within an approximate trust region without ever computing a Hessian.

The full PPO loss also includes a squared-error value-function regression term weighted by $c_V = 0.25$ and an entropy bonus $c_H \cdot \mathcal{H}[\pi_\theta]$ to encourage exploration.

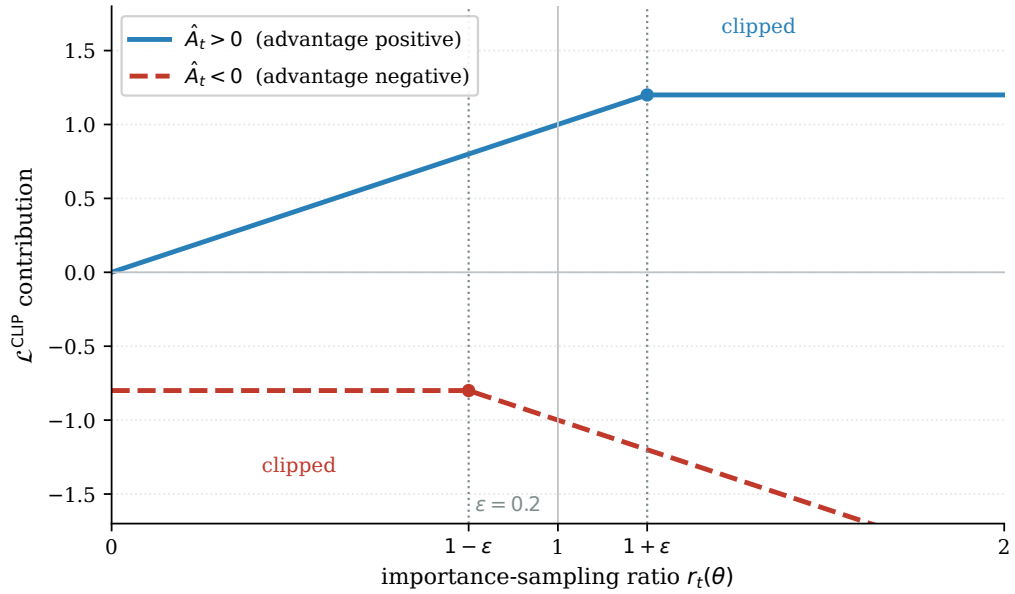

Figure 4.1.: PPO clipped surrogate objective. For a positive advantage (solid) the ratio is capped at $1 + \epsilon = 1.2$, preventing over-exploitation of a good action. For a negative advantage (dashed) it is capped at $1 - \epsilon = 0.8$. Both caps limit how far the new policy can deviate from the data-collecting policy.

4.2.1. Stabilisers and heuristics

Empirical studies [15, 16] have shown that several implementation details have a first-order effect on PPO's sample efficiency and stability. The ones relevant to this project are:

Value clipping. The value-function update is clipped so the critic cannot make overly aggressive updates within a single optimisation batch. The clipping magnitude ξ mirrors the PPO ratio clipping.

KL early stopping. An unbiased estimator [17] of the KL divergence between old and new policy is computed after every minibatch. If the rolling average exceeds a target $D^{\star}_{\text{KL}} = 0.08$, the remaining epochs of the current update are skipped to prevent destructive updates.

Observation normalisation. The 19-dimensional lane/marking/vehicle/route block is rescaled online to zero mean and unit variance using a running estimator. The rationale for normalising only the vector block (not the 720 LIDAR dimensions) is discussed in Section 6.4.3.

Cosine learning-rate annealing. The learning rate decays from its initial value $\eta_{\max}$ to a minimum $\eta_{\min}$

following a cosine curve over the expected total number of PPO updates.

4.3. GENERALISED ADVANTAGE ESTIMATION

The advantage $A^{\pi_\theta}(s_t, a_t)$ in Equation (4.3) must be estimated from sampled rollouts. A plain Monte Carlo estimate has zero bias but high variance, whereas a one-step TD residual $\delta_t^V = r_t + \gamma V_\theta(s_{t+1}) - V_\theta(s_t)$ has low variance but high bias. Generalised Advantage Estimation (GAE) [18] interpolates between the two via an exponentially-weighted sum of TD residuals:

$$\hat{A}_t^{\mathrm{GAE}(\gamma,\lambda)} = \sum_{\ell=0}^{\infty} (\gamma\lambda)^\ell \, \delta_{t+\ell}^V, \tag{4.5}$$

where $\lambda \in [0, 1]$ is the GAE parameter and $\delta_{t+\ell}^V$ is the one-step TD residual evaluated ℓ steps ahead. Figure 4.2 shows how λ controls the weight $(\gamma\lambda)^\ell$ placed on each residual, and therefore the bias-variance trade-off: $\lambda = 0$ keeps only the first residual (the one-step TD advantage: low variance, biased), $\lambda = 1$ weights all residuals by γ^ℓ alone (the Monte Carlo advantage: unbiased, high variance), and the value $\lambda = 0.95$ used in this work sits between the two. When an episode terminates due to a collision or going off-road, the critic bootstrap value is set to zero. When an episode is truncated by a step-count timeout, the critic value at the final observed state is used to approximate the missing return tail.

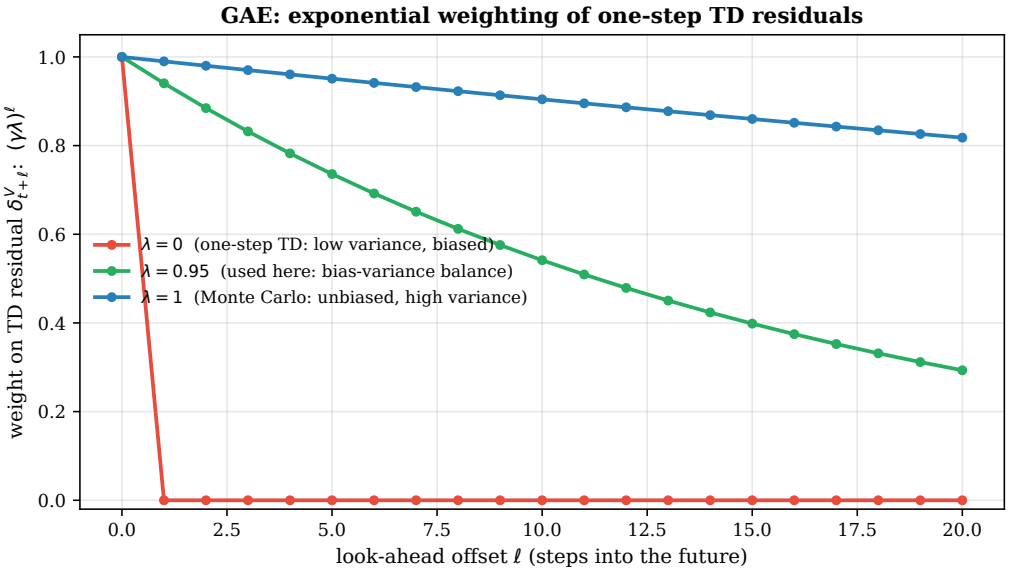

Figure 4.2.: Generalised Advantage Estimation weights the one-step TD residual $\delta^V_{t+\ell}$ by $(\gamma\lambda)^\ell$ (here $\gamma = 0.99$). The parameter λ interpolates between the low-variance, biased one-step TD estimate ($\lambda = 0$, only $\ell = 0$ contributes) and the unbiased, high-variance Monte Carlo estimate ($\lambda = 1$). The choice $\lambda = 0.95$ gives a practical balance.

4.4. SAFE REINFORCEMENT LEARNING

The comprehensive survey by García and Fernández [6] organises Safe RL into two broad categories: *risk-sensitive* approaches that modify the optimality criterion, and *exploration-modifying* approaches that constrain the set of actions available during learning. Four families are particularly relevant to autonomous driving:

- **Constrained MDPs and CPO.** Safety is expressed as one or more auxiliary cost functions, and the policy is trained to maximise reward subject to cost-rate constraints. Achiam et al. [19] introduced Constrained Policy Optimization (CPO), a TRPO-style algorithm that provides approximately monotonic constraint satisfaction.

- **Lagrangian and primal-dual methods.** A more scalable approach forms the dual problem with an adaptive multiplier for each constraint and solves it via alternating updates on the policy and the multiplier [20]. Variants underpin much of modern Safe RL.

- **Safety-aware reward shaping.** Penalty terms in the reward function discourage unsafe behaviour. This is lightweight but provides no hard guarantees: the agent can still attempt unsafe actions during exploration. The reward shaper of Section 6.6 belongs to this family.

- **Runtime shielding.** A verifier inspects each proposed action and substitutes a safe fall-back if a safety specification would be violated. Shielding is orthogonal to the learning algorithm and can be composed with any RL backbone. Alshiekh et al. [7] formalise the approach for finite systems with temporal-logic specifications, and Jansen et al. [21] extend it to probabilistic settings. The shields in this project reason about predicted trajectories rather than formal logic.

4.5. SAFETY SHIELDS

A runtime safety shield can be formalised as a function

$$\Sigma : \mathcal{S} \times \mathcal{A} \to \mathcal{A}, \qquad a_t^\star \;=\; \Sigma(s_t,\, a_t), \tag{4.6}$$

that maps the current state and the agent's proposed action to a safe executed action. Two design principles recur throughout the literature:

Minimal interference. The shield should override the agent's action only when strictly necessary, and the override should be as close as possible to the proposed action. In practice this is implemented by projecting onto a set of certified-safe actions, or by searching through a priority-ordered list of candidates.

Credit assignment through the executed action. When the shield intervenes, the environment sees $a_t^\star$, not the policy's a_t. Storing the proposed action in the replay buffer would push the policy towards an action that was never executed. The framework addresses this by always storing $a_t^\star$ and re-evaluating its log probability under the current policy (Section 7.4.1).

Both shields in this project are *post-posed*: they are inserted between the policy and the simulator as a `gymnasium.Wrapper` and inspect the output of the policy after it has been sampled.

4.6. KINEMATIC BICYCLE MODEL

The adaptive shield must predict the vehicle's future position to verify whether a candidate action will keep it within the lane. It uses a *kinematic bicycle model* [10, 22], a well-established approximation that collapses

each axle into a single virtual wheel and tracks the rear axle position (x, y), heading θ and longitudinal speed v (Figure 4.3). The front-wheel steering angle δ and a longitudinal acceleration command drive the state forward in time steps of $\Delta t = 0.05\,\mathrm{s}$.

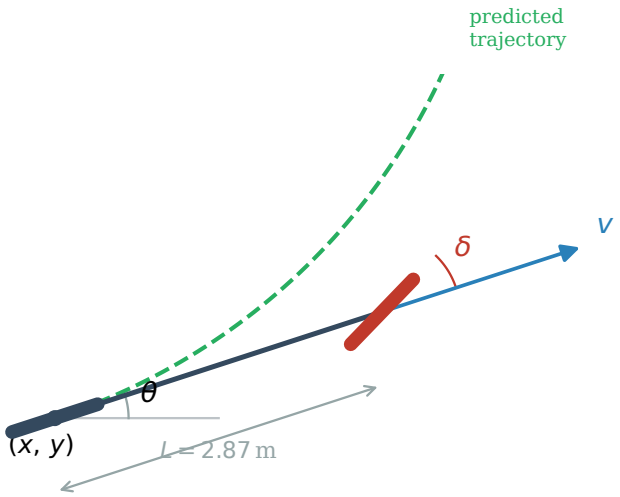

Figure 4.3.: Kinematic bicycle model geometry used by the adaptive shield. The rear axle is at (x, y) with heading θ. Control inputs are the front-wheel steering δ and a longitudinal acceleration. The model advances one step $\Delta t = 0.05\,\mathrm{s}$ using a closed-form arc integration to avoid Euler-step drift at large steering angles.

For small $|\delta|$ the state advances with a simple forward-Euler step. For non-negligible $|\delta|$, the exact arc geometry is used to avoid integration drift. The discrete equations are implemented in `src/Adaptative_Shield/BicycleModel.py`. Kong et al. [22] show that, at the speeds and curvatures typical of urban driving, the kinematic model closely approximates the full dynamic bicycle model for planning horizons below one second, which covers the adaptive shield's longest horizon of ten steps ($0.5\,\mathrm{s}$). The vehicle parameters for CARLA's Tesla Model 3 are in Table 6.2 (Chapter 6).

4.7. CARLA SIMULATOR

CARLA [9] is an open-source simulator for autonomous driving research built on Unreal Engine 4. Its features relevant to this thesis are summarised below.

- **Client–server architecture.** The simulator runs as a server process (`CarlaUE4.exe`) that holds the world state and renders the scene, while the Python client connects to it over a TCP port. This decoupling makes it straightforward to run training and the simulator on different machines.

- **Synchronous mode.** By default CARLA runs asynchronously. When `synchronous=True` the server waits for a `world.tick()` call from the client before advancing one fixed time step ($\Delta t = 0.05\,\mathrm{s}$, i.e. $20\,\mathrm{Hz}$). This is a prerequisite for reproducible experiments.

- **Waypoint API.** The road network is represented as an OpenDRIVE map from which CARLA exposes a lattice of *waypoints* along lane centres. Each waypoint carries lane width, lane type, road curvature, lane-marking type and the allowed lane-change directions. The Waypoint API is the primary source of geometric information for both the reward shaper and the safety shields in this project.

- **Sensors.** CARLA provides a library of virtual sensors. The framework uses three: a semantic LIDAR, a collision detector and a lane-invasion detector. The semantic LIDAR tags every ray hit with the `carla.CityObjectLabel` of the struck primitive, enabling the three-channel decomposition of Section 6.2.1.

- **Traffic Manager.** Non-playable vehicles (NPCs) are governed by CARLA's internal Traffic Manager, which follows the simulator's own traffic rules. The framework uses it to instantiate the NPC density required by each curriculum stage.

- **Predefined towns.** Town04 is a highway loop with multi-lane segments and entrance ramps, and it is the main benchmarking map used in Chapter 8. Towns 01–03 cover urban scenarios, and Town05 offers maximum difficulty.

4.8. RELATED WORK

Deep RL for autonomous driving.

The surveys by Kiran et al. [4] and Aradi [5] offer broad panoramas of the field. PPO is the most common backbone for continuous-control experiments on CARLA, with notable examples including Chen et al. [23] on urban driving and CARLA Leaderboard submissions combining PPO with learned perception [24].

Safety shields in simulated driving.

Shielding has been applied to benchmarks such as HighwayEnv [25] and CommonRoad [26]. Krasowski et al. [27] use reachability analysis to certify lane-change manoeuvres, Wabersich and Zeilinger [28] combine learning-based control with a model-predictive safety filter, and MetaDrive [29] provides a lighter-weight shielding interface that inspired the early prototypes of this project. The present work contributes a *pair* of interchangeable shields in the CARLA ecosystem.

Curriculum learning.

The general idea of presenting tasks in order of increasing difficulty dates back to Bengio et al. [30]. Applications to RL range from procedural domain randomisation [31] to self-play curricula. The curriculum manager of Section 6.7 includes a *rollback* mechanism driven by empirical instability on the 0–20 NPC progression, which is uncommon in the curriculum-learning literature.

Bicycle model for trajectory prediction.

Kong et al. [22] show that, at speeds and curvatures typical of urban driving, the kinematic model closely approximates the dynamic bicycle model for sub-second horizons. This justifies its use inside the adaptive shield (Section 4.6).

5. REQUIREMENTS ANALYSIS

This chapter captures the requirements that guided the design of the framework. It distinguishes functional requirements (what the system has to *do*, Section 5.1) from non-functional requirements (how *well* it has to do it, Section 5.2). Section 5.3 lists the technical constraints imposed by the simulator, the hardware and the academic timeline of the project. Section 5.4 converts these requirements into measurable acceptance criteria that can be evaluated with the metrics logged by the training pipeline.

5.1. FUNCTIONAL REQUIREMENTS

Functional requirements describe the observable behaviour of the system. Each requirement is given a short identifier of the form `FR-XX`, a descriptive title and a succinct rationale. Requirements that depend on a specific implementation choice are justified in Chapter 6.

Table 5.1.: Functional requirements of CALA-SafeRL.

ID	Title	Description
FR-01	Gymnasium environment	The simulator shall be exposed through a `gymnasium.Env`-compatible wrapper supporting `reset`, `step`, `render` and `close`.
FR-02	Continuous control	The action space shall be a two-dimensional continuous box $[-1, 1]^2$ corresponding to steering and joint throttle–brake.
FR-03	Semantic LIDAR	The observation shall include three LIDAR channels (combined, dynamic, static) of 240 rays each, normalised to $[0, 1]$.
FR-04	Lane and route features	The observation shall include lane features (lateral offset, heading error, edge distances, curvature, lane-change direction) and route features (waypoint angles, progress, speed limit) derived from the Waypoint API.
FR-05	PPO training loop	A PPO agent with GAE, clipped surrogate, value loss, entropy bonus and KL early stopping shall be provided.
FR-06	Interchangeable shields	The training and evaluation entry points shall accept a `-shield_type` argument with values `none`, `basic` or `adaptive`, and switch shield implementations without code changes.
FR-07	Dense reward shaping	A wrapper shall augment CARLA's sparse reward with dense components (speed, lane centring, heading, smoothness, progress, idle, shield interaction).
FR-08	Performance-driven curriculum	A curriculum manager shall progressively add NPC traffic as the agent's performance improves and shall roll back when performance collapses.
FR-09	Checkpoint persistence	The agent shall save the best checkpoint on reward improvement and periodic checkpoints at a configurable frequency.
FR-10	Live metrics	Every episode and every PPO update shall be logged to a SQLite database in WAL mode so that an external dashboard can read it while training is still in progress.
FR-11	Evaluation dashboard	A headless evaluation mode and a Matplotlib dashboard mode shall be available, the latter showing LIDAR polar plot, speed gauge, lateral offset, shield status and intervention count.
FR-12	Lane-change awareness	Both the reward shaper and the shield shall detect the condition *lane change is permitted by road markings* and relax penalties that would otherwise block the manoeuvre.
FR-13	Reproducibility	Training and evaluation shall be seeded independently ($\text{seed} = 42$ and $\text{seed} = 100$ respectively) and the seeds shall be exposed through command-line flags.

5.2. NON-FUNCTIONAL REQUIREMENTS

Non-functional requirements describe qualities of the system: how fast, how portable, how maintainable it must be. They are grouped below by ISO/IEC 25010 quality dimension [32].

Table 5.2.: Non-functional requirements of CALA-SafeRL.

ID	Dimension	Description
NFR-01	Performance	The simulator shall run in synchronous mode at $20\,\mathrm{Hz}$ ($\Delta t = 0.05\,\mathrm{s}$). All wrappers shall introduce an overhead small enough to sustain this rate on a single GPU workstation.
NFR-02	Determinism	Given a fixed seed and a fixed hardware/software configuration, two training runs shall produce the same episode outcomes up to floating-point non-associativity.
NFR-03	Modularity	The wrapper chain shall obey the Open/Closed principle: adding a new shield or a new reward component shall require no modifications to the existing layers.
NFR-04	Maintainability	Python source files shall pass `ruff check` with the project's default configuration, and public classes shall carry short docstrings.
NFR-05	Portability	The framework shall run on Linux and Windows and shall automatically select the best PyTorch device among CUDA, Apple MPS and CPU.
NFR-06	Observability	Every component that modifies actions or rewards shall expose an auxiliary `info` dictionary key (`executed_action`, `shield_activated`, `risk_level`, reward sub-components) so that training behaviour can be diagnosed post-hoc.
NFR-07	Extensibility	The live-metrics schema shall admit new keys without breaking existing dashboards. Both the SQLite logger and the Streamlit reader shall treat unknown metrics as first-class citizens.
NFR-08	Testability	The reward shaper shall ship with deterministic unit tests that do not require a running CARLA server.
NFR-09	Documentation	The project shall include a `CLAUDE.md` file that documents the wrapper chain, the observation space layout and the commands to reproduce training and evaluation.

5.3. TECHNICAL CONSTRAINTS

The requirements above must be satisfied inside the technical envelope described in Table 5.3. These constraints are not design decisions but external boundary conditions: the CARLA version is fixed by the research group's infrastructure, the hardware is the author's workstation, and the timeline is set by the Bachelor's thesis calendar.

Table 5.3.: Technical constraints of the project.

Category	**Constraint**
Simulator	CARLA 0.9.16 running on Windows with the `-RenderOffScreen` flag and fixed port 2000.
Language	Python ≥ 3.10.
Core libs	PyTorch (GPU when available), Gymnasium, NumPy, Matplotlib, Streamlit, SQLite. Licences shall be compatible with open academic distribution (MIT, BSD-3, Apache-2.0).
Hardware	Single desktop workstation. A GPU with at least $6\,\mathrm{GB}$ of VRAM is required to run CARLA and the PyTorch policy concurrently. Training on CPU is supported but orders of magnitude slower.
Maps	The main benchmark shall be Town04. Additional towns (Town01–Town07, Town10HD) may be used for qualitative evaluation but are not guaranteed to converge with the default hyperparameters.
Timeline	The project shall be delivered within the Bachelor's thesis calendar of the Universidad de Deusto. This precludes large hyperparameter sweeps that would require multiple weeks of simulator time per configuration.
Safety	The framework shall only run inside simulation. Deployment on a physical vehicle is explicitly out of scope.

5.4. ACCEPTANCE CRITERIA

Acceptance criteria translate the previous requirements into measurable conditions over the metrics produced by `main_train.py` and `main_eval.py`. They are used in Chapter 8 as the reference against which the three shielding configurations are compared. The baseline corresponds to the unconstrained PPO agent (`-shield_type none`).

Table 5.4.: Acceptance criteria derived from the functional and non-functional requirements. Thresholds are placeholders to be replaced by the empirical numbers obtained in Chapter 8.

ID	Metric	Acceptance threshold
AC-01	Reward convergence	The 100-episode rolling average reward of the shielded agent shall be no worse than the baseline after an equal amount of training.
AC-02	Crash rate	With the adaptive shield, the 100-episode rolling `crash_rate` shall drop by at least a factor of $\alpha = 4$ with respect to the unshielded probe. Empirically this is a factor of $8\times$ ($16\% \rightarrow 2\%$), and the threshold is set conservatively at $4\times$.
AC-03	Off-road rate	Under the adaptive shield, the 100-episode rolling `offroad_rate` shall remain below $\beta = 0.15$ (15%). Empirically: 0.0% at training end and 12% in the 50-episode evaluation.
AC-04	Success rate	On 50 evaluation episodes with seed 100, the agent shall reach the `success_distance` target in at least $\gamma = 0.60$ (60%) of them. Empirically: 70% with adaptive shield.
AC-05	Shield interference	The fraction of all steps in which the adaptive shield activates shall not exceed $\tau = 0.40$ (40%). The *safe*, *warning* and *critical* risk-level fractions shall sum to 1. Empirically: shield activates in 38.7% of steps (65.8% safe, 25.6% warning, 8.6% critical).
AC-06	Speed compliance	At least 90% of steps with a valid speed limit shall satisfy speed_kmh $\leq 1.05 \cdot$ speed_limit_kmh.
AC-07	Curriculum progression	The curriculum manager shall advance from stage 0 to at least stage 3 during a standard $2\,500$-episode run in all shielded configurations.
AC-08	Dashboard liveness	The Streamlit dashboard shall refresh training metrics with an end-to-end latency below $2\,\mathrm{s}$ from the moment the logger writes a row to the SQLite database.
AC-09	Reproducibility	Two independent runs with the same seed and configuration shall produce identical reward curves in the first 50 episodes (before floating-point divergence becomes significant).

The criteria with placeholder thresholds ($\alpha, \beta, \gamma, \tau$) are explicitly left to be fixed once the experiments of Chapter 8 have been executed. This keeps the requirements traceable while avoiding the pitfall of committing to numeric targets before the empirical evidence is available.

6. SYSTEM DESIGN

This chapter describes how the concepts introduced in Chapter 4 are instantiated into a concrete software architecture. Section 6.1 presents the layered wrapper chain, Sections 6.2–6.3 formalise the observation and action spaces, Section 6.4 describes the PPO agent, Section 6.5 the two safety shields, Section 6.6 the reward shaper, Section 6.7 the curriculum manager and Section 6.8 the live metrics pipeline.

6.1. ARCHITECTURAL OVERVIEW

The framework is organised as a vertical stack of components that communicate through the standard `gymnasium.Env` interface. Each layer consumes the observation produced by the layer below and may transform both the action travelling downwards and the reward travelling upwards. Figure 6.1 shows the chain.

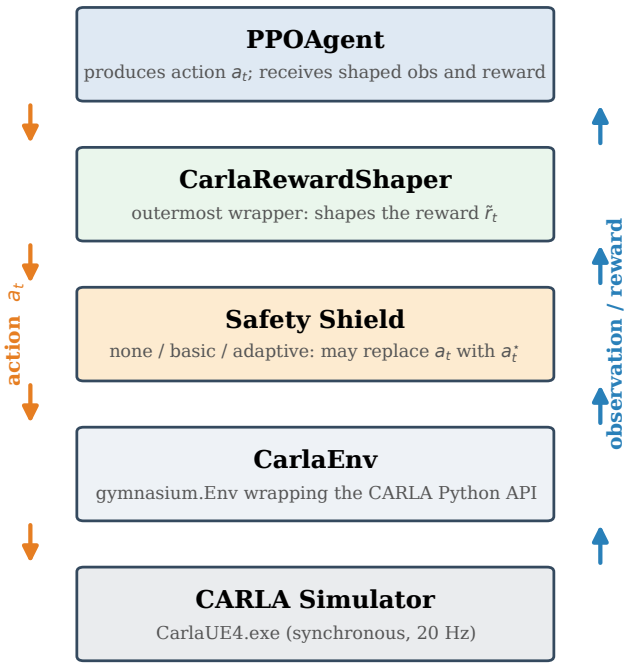

Figure 6.1.: Wrapper chain of the framework. Each layer is a `gymnasium.Wrapper` that can be composed, replaced or removed without touching its neighbours. The safety shield sits above the reward shaper so that shield diagnostics (`executed_action`, `risk_level`) are available to the shaper when it computes the intervention penalty.

A subtlety of this layout deserves comment. The reward shaper is placed *above* the shield (outermost wrapper, closest to the agent), not below it. The shield is the *inner* wrapper, between the shaper and `CarlaEnv`. This ordering has a functional reason: when the shaper calls `self.env.step(action)`, the inner shield runs first and enriches the returned `info` dictionary with the keys `executed_action` and `proposed_action`. The shaper can then read those keys to compute the shield-intervention penalty and the smoothness penalty. If the shaper were placed *below* (inner to) the shield, those keys would not yet exist when the shaper processes its call.

The environment runs in *synchronous mode* with a fixed time step of $\Delta t = 0.05\,\mathrm{s}$. Every call to `env.step(action)` sends the action to CARLA, requests a single `world.tick()` and collects the resulting sensor readings. This guarantees determinism given a fixed seed.

6.2. OBSERVATION SPACE

The observation vector $s_t \in \mathbb{R}^{739}$ concatenates three semantic LIDAR channels and a 19-dimensional block of geometric and route features. Appendix B contains the full index-by-index table, and this section describes the design rationale.

6.2.1. Semantic LIDAR

The framework uses a simulated LIDAR with $N = 240$ rays uniformly distributed over 360° (angular resolution 1.5°/ray, range $50\,\mathrm{m}$). Each ray is normalised to $[0, 1]$, where 1 denotes a free ray and values near 0 denote an obstacle close to the vehicle. The semantic channel is obtained from CARLA's `SemanticLidarSensor`, which tags every ray hit with the `carla.CityObjectLabel` of the struck primitive.

Three derived channels are packed into the observation (see Figure 6.2):

- obs$[0{:}240]$, **combined channel**: all obstacles (dynamic and static) plus road edges.
- obs$[240{:}480]$, **dynamic-only channel**: vehicles and pedestrians only.
- obs$[480{:}720]$, **static-only channel**: walls, barriers, poles and similar static objects.

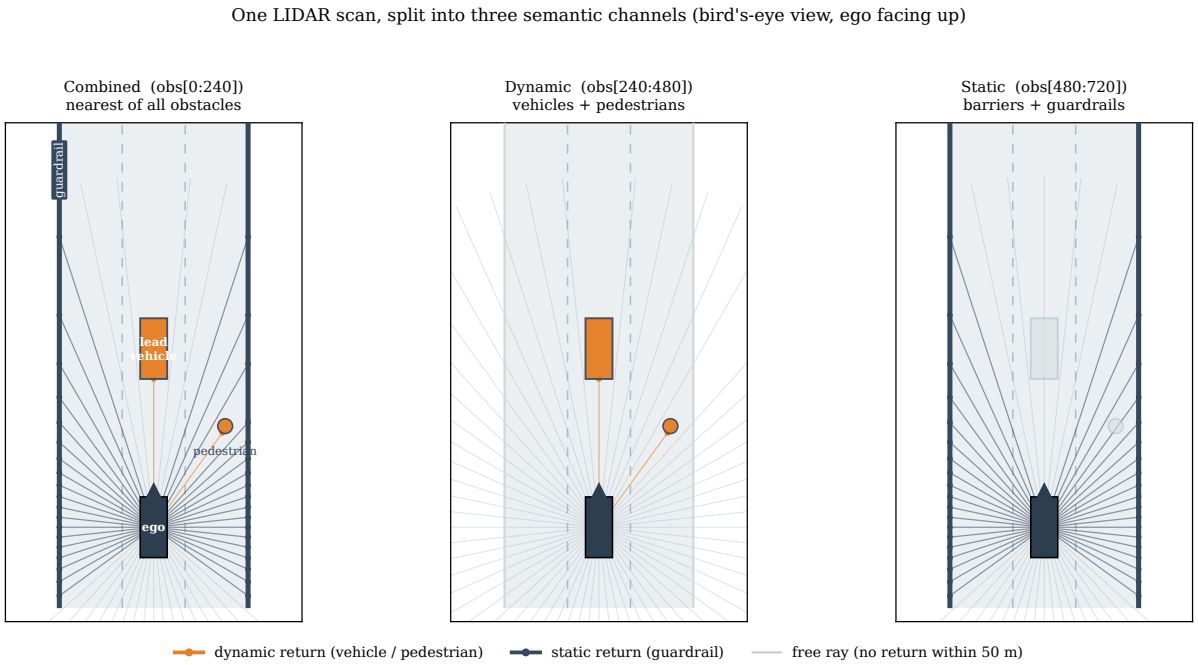

Figure 6.2.: Bird's-eye view of a single LIDAR scan (ego facing up) decomposed into the three semantic channels. The combined channel (left, obs$[0{:}240]$) keeps the nearest return of any obstacle. The dynamic channel (centre, obs$[240{:}480]$) keeps only the lead vehicle and pedestrian. The static channel (right, obs$[480{:}720]$) keeps only the guardrails. Orange rays are dynamic returns, slate rays are static returns and faint rays leave the scan unobstructed. Separating the channels lets the policy treat the guardrails as soft lane edges while still anticipating the vehicle and pedestrian, avoiding the zig-zag that a single combined channel induces along a guardrail.

The channel decomposition is motivated by an empirical observation: a single-channel LIDAR induces a *zig-zag* pattern when driving along a guardrail, because the policy interprets static low-range returns as moving obstacles and tries to evade laterally. Separating dynamic from static obstacles allows the policy to treat static boundaries as soft lane edges while properly anticipating dynamic hazards.

6.2.2. Lane, marking, vehicle and route features

A 19-dimensional vector block is appended to the LIDAR. Its components are:

- obs$[720{:}728]$, **lane features (8 dims)**: lateral offset, heading error, edge warning, lane width, left/right

edge distances, lane-change permission, road curvature.

- obs$[728{:}732]$, **lane-marking type (4 dims)**: binary flags for solid-left, solid-right, dashed-left, dashed-right. These allow the policy to distinguish legal lane changes (dashed) from illegal solid-line crossings.

- obs$[732{:}734]$, **vehicle state (2 dims)**: normalised speed, last applied steering.

- obs$[734{:}739]$, **route features (5 dims)**: waypoint angle at 5 m, waypoint angle at 20 m, progress fraction, normalised speed limit, speed ratio.

All features are extracted from CARLA's Waypoint API. The lateral edge distances are normalised by the lane width so that both take the value 0.5 at the lane centre and approach 0 at the corresponding edge. The LIDAR block (obs$[0{:}720]$) is clipped to $[0, 1]$, while the vector block (obs$[720{:}739]$) is clipped to $[-1, 1]$.

6.3. ACTION SPACE

The action space is $\mathcal{A} = [-1, 1]^2$. Its two components are:

- $a^{(1)}$, *steering*, mapped linearly to physical steering $\delta \in [-0.6,\ +0.6]\,\mathrm{rad}$.

- $a^{(2)}$, *joint throttle–brake*: positive values drive the throttle (up to $+3.0\,\mathrm{m/s^2}$), negative values drive the brake (up to $-7.0\,\mathrm{m/s^2}$), zero is coast.

Merging throttle and brake into a single signed channel reduces the output dimensionality and prevents the agent from learning the counter-productive behaviour of applying both simultaneously.

Continuous actions are sampled from a diagonal Gaussian and squashed through a hyperbolic tangent to keep them inside the box:

$$u_t \sim \mathcal{N}\big(\mu_\theta(s_t),\, \sigma_\theta^2\big), \qquad a_t = \tanh(u_t), \tag{6.1}$$

where $\mu_\theta(s_t) \in \mathbb{R}^2$ is the state-dependent mean produced by the actor head, $\sigma_\theta \in \mathbb{R}^2$ is the state-independent standard deviation (Section 6.4.1), u_t is the pre-squash Gaussian sample and a_t is the executed action inside the box $[-1, 1]^2$. Computing the log probability of a squashed sample requires adding the log-determinant of the tanh Jacobian [33], a correction that is also applied when evaluating the log probability of an executed (possibly shield-modified) action for credit assignment.

6.4. AGENT DESIGN

6.4.1. Actor–Critic network

The actor and the critic share a representation but have separate heads (Figure 6.3). Feeding the raw 739-dimensional observation directly into a shallow MLP dilutes the signal: the 720 LIDAR dimensions dominate numerically while the 19 lane/vehicle/route features carry most task-relevant information. The network therefore includes a dedicated *LIDAR encoder*: a two-layer MLP with 256 hidden units that compresses the 720-dimensional LIDAR input to a 64-dimensional representation. This 64-dim summary is concatenated with the 19-dimensional vector block to produce an 83-dimensional feature $\phi(s_t)$ that feeds the actor and critic trunks.

Both actor and critic trunks are two-layer MLPs with 256 units per hidden layer and Tanh activations. The actor head produces a 2-dimensional mean $\mu_\theta(s_t)$ together with a state-independent log-standard-deviation parameter. The log-std is bounded to the range $[-3.0, -1.2]$ (corresponding to $\sigma \in [0.05, 0.30]$) via a tanh-squashed parameterization so that the effective σ can *anneal down* as the policy converges. The effective initial log-std is -1.5 ($\sigma \approx 0.22$). The critic head produces a scalar value estimate $V_\theta(s_t)$.

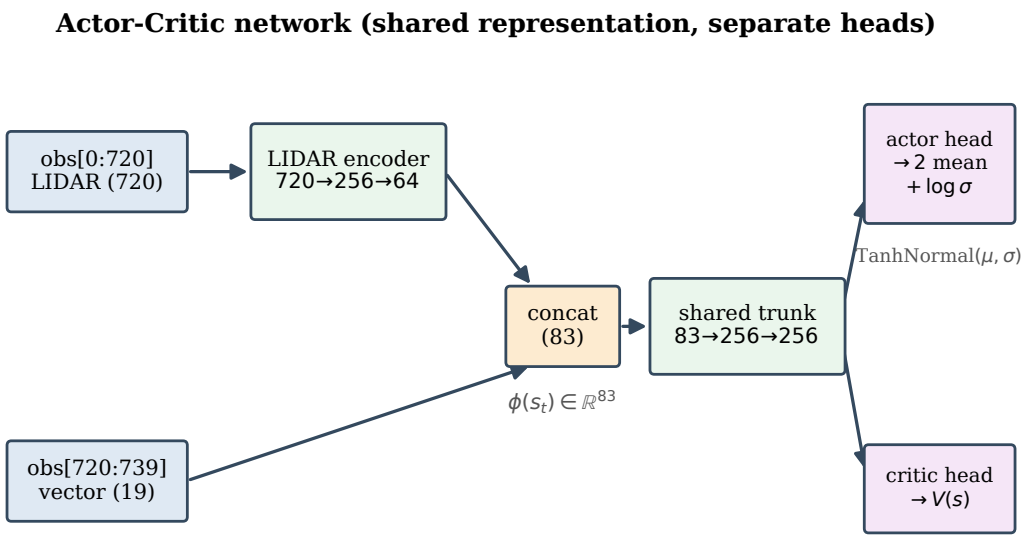

Figure 6.3.: Actor-Critic architecture. The LIDAR encoder compresses 720 LIDAR dimensions to 64, and the resulting 83-dimensional feature feeds two independent trunks. The actor outputs a 2D mean and a state-independent log-std (bounded via tanh squashing to $[-3.0, -1.2]$), while the critic outputs a scalar value estimate $V_\theta(s_t)$.

The choice of *how* to bound the log-std is not cosmetic. An earlier design clamped the raw parameter with a straight-through clip. Because the clip has *zero* gradient beyond its cap, the entropy bonus could push the parameter past the ceiling and pin the effective σ at its maximum, freezing exploration noise high for the rest of training. The tanh-squashed parameterisation removes this failure mode: its gradient is smooth and never vanishes inside the band, so σ can anneal down as the policy converges (Figure 6.4).

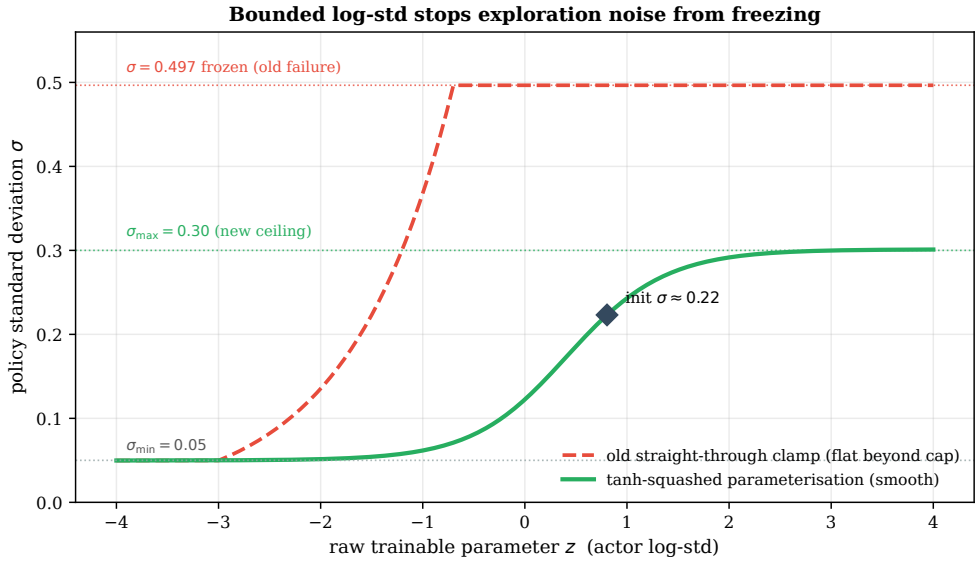

Figure 6.4.: Effect of the log-std parameterisation on the policy standard deviation σ. The old straight-through clamp (dashed) is flat beyond its cap, so once the entropy bonus drove the raw parameter past it, σ froze at 0.497. The tanh-squashed parameterisation (solid) maps the raw parameter smoothly into $[0.05,\ 0.30]$, keeping a non-vanishing gradient throughout and starting at $\sigma \approx 0.22$.

6.4.2. PPO update

At each environment step the agent appends the tuple $\left(s_t,\ a_t^\star,\ \log \pi_\theta(a_t^\star \mid s_t),\ r_t,\ d_t,\ \mathrm{trunc}_t,\ V_t^{\text{boot}}\right)$ to the rollout buffer. Two details are specific to this project:

1. The buffer stores the *executed* action $a_t^\star$, not the sampled action. The stored log probability is recomputed by `evaluate_action` using the log-det Jacobian correction of Equation (6.1). This ties the gradient computation to the action the environment actually saw, avoiding the credit-assignment error that would otherwise be introduced by the shield.

2. When an episode is truncated by a timeout, the critic value $V_\theta(s_{T+1})$ is stored as V_t^{boot} and used to bootstrap GAE at that step. For natural terminations (collision, off-road), $V_t^{\text{boot}} = 0$.

When the buffer reaches $M = 2048$ samples, GAE (Equation (4.5)) is run backwards through the rollout, advantages are standardised, and the return target is divided by a running standard deviation (Section 6.4.4). Up to $K = 10$ epochs of PPO are then performed with SB3-style minibatching [34] (minibatch size 64, giving 32 Adam steps per epoch), with a KL early stop governed by the mechanism of Section 4.2.1. The gradient is

clipped to an ℓ_2 norm of 0.5 and the learning rate follows a cosine annealing schedule.

6.4.3. Observation normalisation

Online normalisation is performed on the 19-dimensional vector block only, using Welford's incremental mean/variance estimator [35]. The estimator maintains a running mean $\hat{\mu}$ and variance $\hat{\sigma}^2$ that are updated in mini-batches and used to rescale each feature to approximately zero mean and unit variance. Normalised observations are clipped to $[-5, 5]$ to contain outliers.

The 720 LIDAR dimensions are left un-normalised because they are already in $[0, 1]$ by construction. Applying normalisation would artificially introduce distribution shift when the curriculum advances from stage 0 (no NPCs, constant-1.0 dynamic channel) to stage 1 (dynamic channel suddenly has non-unit entries), causing the gradient spikes observed in early prototypes.

6.4.4. Return normalisation

The return targets passed to the critic loss can span several orders of magnitude as the agent learns, which makes MSE training unstable. To counter this, returns are divided by the running standard deviation of a discounted-return accumulator that is updated across rollout boundaries. The mean is deliberately *not* subtracted, as subtracting a running mean would bias the advantages (which are already standardised separately).

6.5. SAFETY SHIELDS

Both shields implement the function Σ of Equation (4.6). They differ in the information they consume and the strategy by which they generate the corrected action.

6.5.1. Basic shield

The basic shield (`CarlaSafetyShield`) operates in two layers. The first analyses the combined LIDAR scan for imminent obstacles by comparing minimum values in three angular sectors (front: $\pm 22.5°$, right side: $60°$–$120°$, left side: $240°$–$300°$) against fixed thresholds ($\tau_f = 0.15$ for the front, $\tau_s = 0.04$ for the sides). The front sector spans rays $[225{:}240] \cup [0{:}15]$ (30 of the 240 rays at $1.5°/\text{ray}$). The second layer reads the lateral offset and heading error from the `info` dictionary and flags the situation as unsafe when they exceed their respective thresholds

($\tau_\ell = 0.82, \tau_h = 0.60$).

When the situation is unsafe, the shield applies a proportional correction: the steering command is adjusted to push the vehicle back towards the lane centre (combining lateral offset, heading error and any required lateral obstacle push), and a braking command is applied proportionally to the detected frontal clearance deficit. Both corrections are clipped to the action range and follow the *minimal-interference principle*: when all threats are far from their thresholds the corrections vanish and the shield passes the agent's action through unmodified.

6.5.2. Adaptive-horizon shield

The adaptive-horizon shield (`CarlaAdaptiveHorizonShield`) extends the basic design with three mechanisms: a risk classifier, trajectory prediction with the bicycle model, and a priority-ordered candidate search.

Risk classifier.

Two independent criteria jointly determine the risk level, and the overall level is the worst (most severe) of the two (Figure 6.5). The *frontal-distance criterion* thresholds the minimum of the dynamic-only LIDAR front sector $d_t^{\mathrm{dyn}} \in [0, 1]$, where a higher value denotes a more distant in-path obstacle (1 is a clear lane). The *lateral-position criterion* thresholds the absolute normalised lateral offset $|\ell_t|$ and the absolute heading error $|\psi_t|$. Each level in turn fixes the trajectory-prediction horizon H, the threshold multiplier κ and the per-level lateral tolerance $\tau_\ell^{\mathrm{level}}$ used by the trajectory check below. Table 6.1 collects all of these thresholds. The equivalent piecewise definition of the frontal criterion is reproduced in Appendix D, Equation (D.1).

Table 6.1.: Risk-level thresholds of the adaptive shield. A step is assigned the worst level for which any criterion fires. That level then fixes the prediction horizon H, the threshold multiplier κ and the trajectory lateral tolerance $\tau_\ell^{\mathrm{level}}$.

Level	**Front** d_t^{dyn}	$\lvert\ell_t\rvert$	$\lvert\psi_t\rvert$	H	κ	$\tau_\ell^{\mathrm{level}}$
safe	> 0.50	< 0.50	$< 10^\circ$	1	1.0	0.55
warning	0.20–0.50	0.50–0.70	10°–20°	5	1.5	0.40
critical	≤ 0.20	> 0.70	$> 20^\circ$	10	2.0	0.30

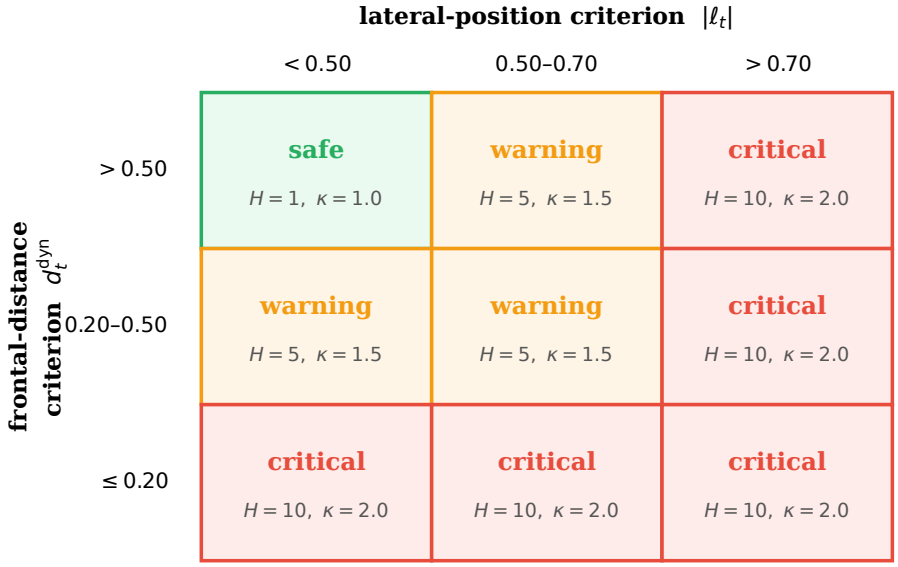

Figure 6.5.: Risk level classification of the adaptive shield. Each step is independently assessed by a frontal-distance criterion (rows, thresholds at 0.50 and 0.20) and a lateral-position criterion (columns, thresholds at $|\ell| = 0.50/0.70$ and $|\psi| = 10°/20°$). The overall level is the worst of both. The corresponding trajectory-prediction horizon H and threshold multiplier κ are shown in each cell.

Trajectory check.

Given a candidate action, the shield asks the bicycle model of Section 4.6 for the trajectory over H steps. Each predicted position is projected onto the nearest driving lane via CARLA's Waypoint API. The candidate is declared safe when all of the following hold simultaneously:

- The combined front LIDAR minimum exceeds τ_f/κ.
- Static side LIDAR minima exceed τ_s/κ (unless the ego is safely centred in lane with both edge distances above 0.3).
- The nearest edge distance exceeds the hard floor 0.15.

- The nearest pedestrian is more than $4.0\,\mathrm{m}$ away.
- Every predicted lateral offset along the trajectory satisfies $|\ell_k|/(w_{\mathrm{lane}}/2) \leq \tau_\ell^{\mathrm{level}}$, with the per-level tolerance $\tau_\ell^{\mathrm{level}}$ of Table 6.1.
- No predicted heading deviates more than $0.52/\kappa\,\mathrm{rad}$ from the nearest lane direction, and no predicted lateral drift exceeds $1.75/\kappa\,\mathrm{m}$ from the reference position.

Minimal-interference projection.

If the proposed action fails the trajectory check, the shield computes an *emergency action* a_t^{em} whose steering corrects the lateral offset and heading error and whose throttle/brake depends on the severity of the threat. It then searches the first mixing coefficient $\alpha \in \{0.25,\ 0.50,\ 0.75,\ 1.0\}$ such that the blended action

$$a_t(\alpha) \;=\; (1-\alpha)\, a_t^{\mathrm{prop}} \;+\; \alpha\, a_t^{\mathrm{em}} \tag{6.2}$$

passes the trajectory check, where a_t^{prop} is the agent's proposed action, a_t^{em} the shield's emergency action and $\alpha \in [0,1]$ the mixing coefficient ($\alpha = 0$ leaves the proposal untouched, $\alpha = 1$ executes the full emergency action). The first safe blend is executed. If none passes, a_t^{em} is executed with $\alpha = 1$. This *minimal-interference projection* (Figure 6.6) keeps the executed action close to the agent's proposal (small corrections use $\alpha = 0.25$) and avoids the numerical instability of hard-coded fallback vectors that would saturate the tanh policy output.

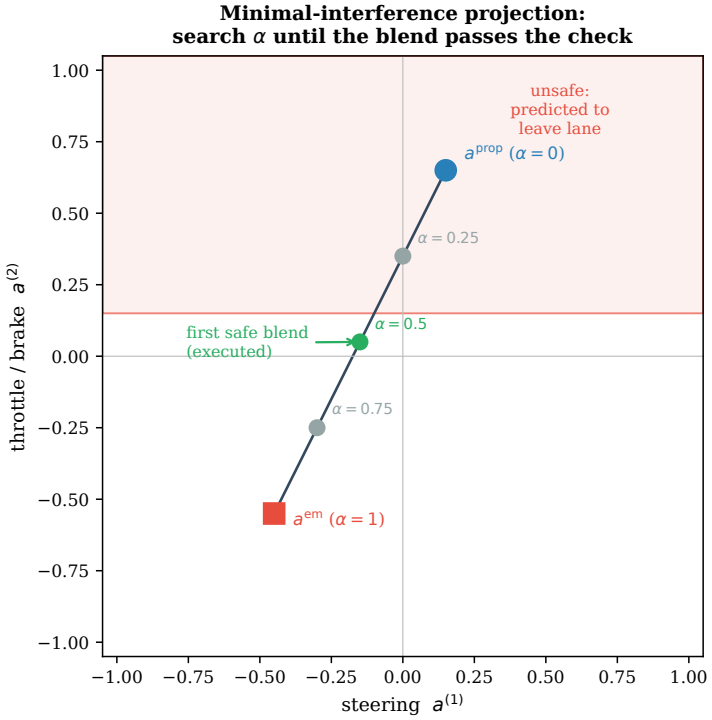

Figure 6.6.: Minimal-interference projection in action space. The shield walks the segment from the agent's proposal a_t^{prop} ($\alpha = 0$) towards the emergency action a_t^{em} ($\alpha = 1$) and executes the first blend that leaves the unsafe region (here $\alpha = 0.5$). Larger interventions are used only when smaller ones still fail the trajectory check, so the agent keeps as much control as safety allows.

Pedestrian emergency.

If the nearest pedestrian is detected at less than $4\,\text{m}$, the shield bypasses the projection and issues a full brake $a_t = (0, -1)$ with $\alpha = 1$.

6.5.3. Bicycle-model parameters

The bicycle model is instantiated with the parameters of CARLA's Tesla Model 3, summarised in Table 6.2.

Table 6.2.: Parameters of the kinematic bicycle model used by the adaptive shield.

Symbol	Value	Meaning
L	$2.87\,\mathrm{m}$	Wheelbase of the Tesla Model 3
$\delta_{\max}$	$1.22\,\mathrm{rad}$ ($\approx 70^\circ$), auto-calibrated	Maximum steering angle from vehicle physics
Δt	$0.05\,\mathrm{s}$	Time step, matched to the simulator
$a_{\max}$	$3.0\,\mathrm{m/s^2}$	Maximum forward acceleration
$-a_{\min}$	$7.0\,\mathrm{m/s^2}$	Maximum deceleration (brake)

6.6. REWARD SHAPING

The reward shaper `CarlaRewardShaper` augments the sparse base reward r_t^{base} of `CarlaEnv` (success $+30$, collision -10, off-road -30) with dense components that provide continuous feedback during the episode. The total shaped reward $\tilde{r}_t$ adds to r_t^{base} a set of dense *bonuses* that reward forward progress, speed compliance and lane discipline, and subtracts a set of dense *penalties* that discourage unsafe driving, stalling and unnecessary shield activations. Figure 6.7 gives the overview and Table 6.3 names every term. The full additive expression is collected in Appendix D, Equation (D.3).

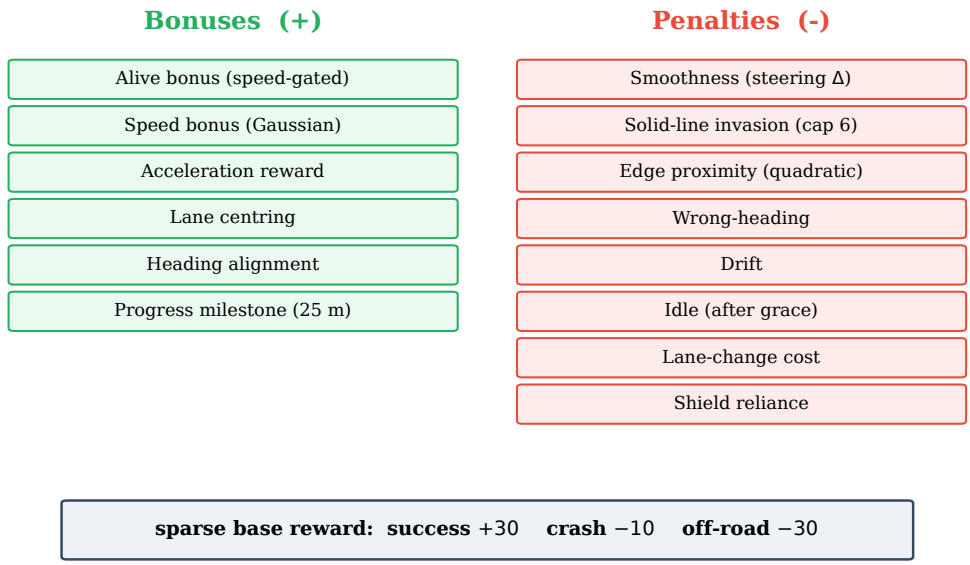

Figure 6.7.: Overview of the dense reward components plus the sparse base reward. Bonuses (green) reward forward progress, speed compliance and lane discipline, while penalties (red) penalise unsafe driving, stalling and unnecessary shield activations.

Table 6.3.: Terms of the shaped reward $\tilde{r}_t$. Each term is computed per step. Default weights are listed in Appendix A, Table A.3.

Term	Sign	Description
r_t^{base}	$\pm$	Sparse base reward: success $+30$, collision -10, off-road -30.
r_t^{prog}	$+$	Speed-proportional progress bonus, gated by μ_t, active only on-road.
r_t^{accel}	$+$	Acceleration bonus from the first step of motion, gated by μ_t.
r_t^{speed}	$+$	Gaussian bonus around a curvature-adjusted target speed.
r_t^{lane}	$+$	Lane-centring bonus, largest at the lane centre.
r_t^{head}	$+$	Heading-alignment bonus, largest when aligned with the lane.
r_t^{mile}	$+$	Fixed bonus at every $25\,\text{m}$ route milestone.
p_t^{smooth}	$-$	Steering-smoothness penalty (absolute steering change).
p_t^{inv}	$-$	Lane-marking invasion penalty.
p_t^{solid}	$-$	Solid-line crossing: flat -2.5, capped at 6 events per episode.
p_t^{edge}	$-$	Quadratic edge-proximity penalty.
$p_t^{\text{wrong-h}}$	$-$	Penalty for heading error beyond 90°.
p_t^{idle}	$-$	Idle penalty after a per-episode grace period.
p_t^{drift}	$-$	Drift penalty for asymmetric edge distances.
p_t^{lc}	$-$	Lane-change cost (-0.05 per event, 20-step cooldown).
p_t^{shield}	$-$	Shield-reliance penalty $\propto \alpha_t$ (default weight 0).

The dense bonuses r_t^{prog}, r_t^{accel} and the positive part of r_t^{speed} are multiplied by a *lane-keep gate* $\mu_t \in [0,\, 1]$, the product of a centering factor and a heading factor (Figure 6.8, with the closed form in Equation (D.2) of Appendix D). Here ℓ_t is the normalised lateral offset and ψ_t the heading error in degrees. The gate reaches 1 only when the vehicle is both centred ($|\ell_t|$ small) and aligned ($|\psi_t|$ small), and falls to 0 as soon as the lateral offset reaches the edge ($|\ell_t| \geq 1$) or the heading error reaches 20°. Forward progress is therefore rewarded only when it is made *in lane*. During a permitted lane-change manoeuvre ($\ell_t > 0.5$ and CARLA reports the change allowed), μ_t is floored at 0.3 so that the agent is not penalised for intentionally crossing the lane centre.

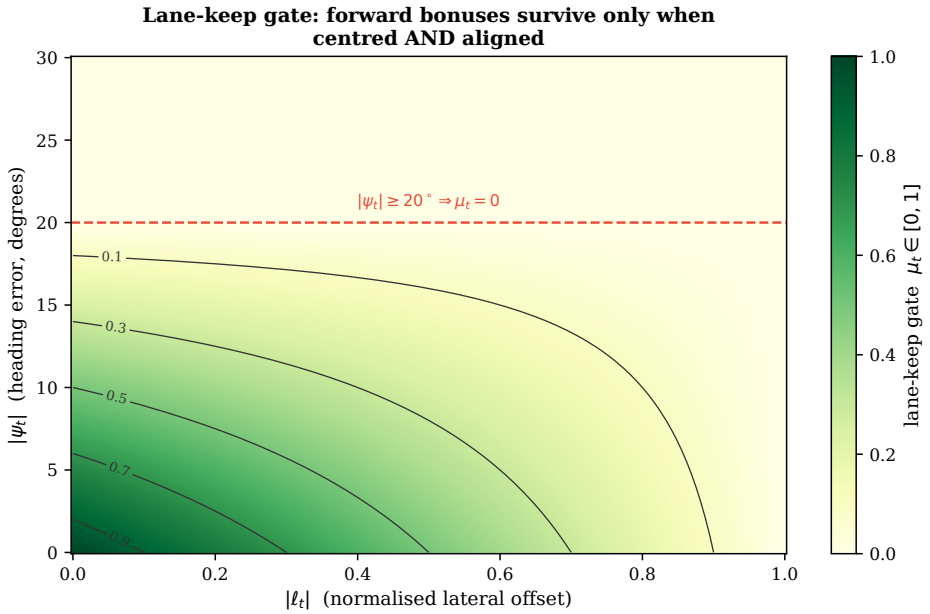

Figure 6.8.: The lane-keep gate μ_t as a function of the absolute lateral offset $|\ell_t|$ and the absolute heading error $|\psi_t|$. The gate multiplies the forward-progress bonuses, so they survive only in the dark corner where the vehicle is both centred and aligned, and vanish once $|\psi_t| \geq 20°$ or $|\ell_t| \geq 1$. This closes the local optimum of driving fast while off-lane.

The individual components are described below. Default weights are in Appendix A, Table A.3.

Progress reward r_t^{prog}.

A continuous speed-proportional bonus $r_t^{\text{prog}} = \text{clip}(v_t/10\,\text{km/h},\, 0,\, 1) \times w_{\text{prog}} \times \mu_t$, active only when on-road. Saturation at $10\,\text{km/h}$ avoids rewarding excessive speed purely for progress, while the lane-keep gate μ_t closes the local optimum of driving fast while off-lane.

Acceleration reward r_t^{accel}.

$r_t^{\text{accel}} = \text{clip}(\Delta v_t,\, 0,\, 2\,\text{km/h}) \times w_{\text{accel}} \times \mu_t$. Provides a dense positive signal from the very first acceleration step (before the vehicle reaches any speed threshold), encouraging the agent to start moving instead of stalling.

Speed bonus.

A Gaussian bonus centred on a curvature-adjusted target speed: the target is reduced in sharp bends to discourage excessive speed around turns. An over-speed penalty applies when the vehicle exceeds the raw speed limit by more than 5%.

Lane centring and heading alignment.

$r_t^{\text{lane}} = \max(1 - |\ell_t|, 0) \times w_{\text{lc}}$ and $r_t^{\text{head}} = \max(1 - |\psi_t|/20^\circ, 0) \times w_{\text{h}}$, both gated by a recent-movement flag (suppressed if the vehicle has been stationary for ≥ 5 consecutive steps) so the agent is not rewarded for parking precisely in lane.

Smoothness penalty.

Proportional to the absolute steering change between successive steps. This penalises erratic inputs and encourages smooth driving.

Edge proximity, wrong-heading and drift penalties.

$p_t^{\text{edge}} = 0.6 \times \max(0, 0.5 - \min(\ell_t^L, \ell_t^R))$, where $\ell_t^{L,R}$ are the normalised left/right edge distances and the threshold is 0.5, not 0.4. A separate penalty applies for heading error exceeding 90°, and a drift penalty applies for asymmetric edge distances when the vehicle is hugging one side.

Solid-line invasion.

A flat penalty $-p_{\text{solid}} = -2.5$ per solid-lane-marking crossing, capped at $N_{\text{cap}} = 6$ events per episode (maximum accumulation -15). The flat structure preserves a constant marginal cost for every crossing, which a geometric decay design had diluted to near zero after the first two or three events, eliminating the return gradient for an agent already committed off-lane.

Lane-change cost.

A small cost -0.05 per detected lane-change event (cooldown 20 steps), discouraging gratuitous lane-switching.

Shield-reliance penalty.

An optional penalty $p_t^{\text{shield}} = w_{\text{sh}} \times \alpha_t$ proportional to the shield intensity $\alpha_t \in [0, 1]$. The default weight is 0, but it can be set positive to make shield reliance reward-suboptimal. This penalty is suppressed during a permitted lane-change transition.

Progress bonus.

A fixed bonus every time the vehicle crosses a $25\,\text{m}$ milestone along the route. This prevents a pure speed bonus from rewarding high speed regardless of whether the vehicle is actually advancing along the intended route.

Idle penalty and shield grace period.

An idle penalty discourages the agent from stopping. To prevent this from punishing legitimate shield-induced stalls (e.g. the anti-deadlock recovery of Section 6.5), a grace period of $G = 10$ steps is opened the first time the shield activates in an episode. The counter is re-armed only when the shield fires and the grace period has already expired, preventing an always-active shield from permanently suppressing the idle penalty.

Lane-change transition guard.

When CARLA's Waypoint API reports that a lane change is permitted and the lateral offset exceeds 0.5, the invasion, edge and drift penalties are all set to zero. This prevents the agent from being penalised for a legitimate lane-change manoeuvre during training.

6.7. CURRICULUM MANAGER

The curriculum manager divides the NPC interval $[0,\, N_{\max}]$ uniformly into five stages. With $N_{\max} = 60$, as used in the experiments of Chapter 8, this yields the sequence $(0,\, 15,\, 30,\, 45,\, 60)$. The manager maintains two counters: the episode count at the current stage E_{stage} and a decay-weighted rollback counter C_{rb}. Figure 6.9 illustrates the state machine.

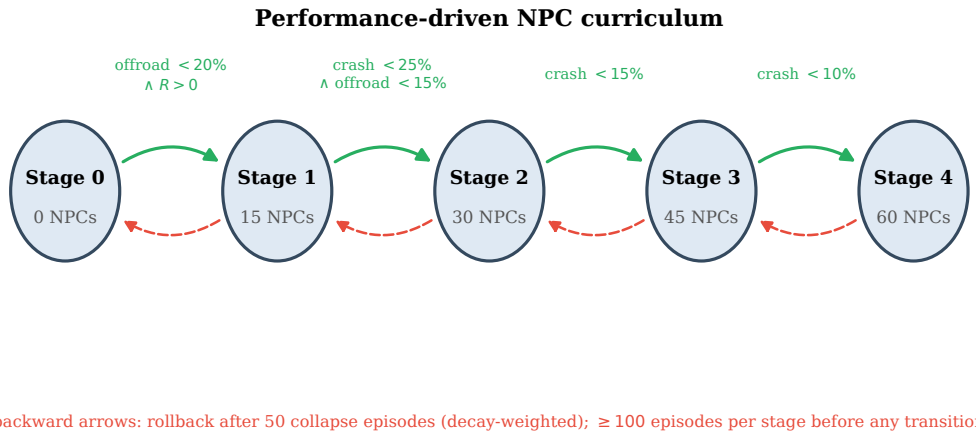

Figure 6.9.: Curriculum state machine. Forward arrows show the performance threshold required to advance (evaluated after at least 100 episodes at the current stage). Backward arrows show rollback, triggered when a decay-weighted counter reaches 50 consecutive collapse episodes.

Advancement rule.

After $E_{\text{stage}} \geq 100$ episodes, if the stage-specific condition in Table 6.4 is met, the manager advances one stage and resets all counters.

Rollback rule.

On every episode beyond the minimum, C_{rb} is incremented by one if the stage is *collapsing* and decremented by one otherwise (but not below zero). When $C_{\text{rb}} \geq 50$, the manager rolls back one stage. The decay by -1 on non-collapsing episodes prevents a streak of bad luck from triggering a rollback that is not warranted by a sustained performance drop.

Table 6.4.: Advancement and rollback thresholds per curriculum stage. All rates are 100-episode rolling averages.

Stage	NPCs	Advance if	Rollback if
0	0	offroad $< 20\% \wedge$ reward > 0	–
1	15	crash $< 25\% \wedge$ offroad $< 15\%$	crash $> 45\% \vee$ offroad $> 40\%$
2	30	crash $< 15\%$	crash $> 40\%$
3	45	crash $< 10\%$	crash $> 35\%$
4	60	– (terminal)	crash $> 30\%$

Non-disruptive NPC update.

When the stage changes, the manager updates `CarlaEnv.num_npc_vehicles` in place. The change takes effect at the next `env.reset()`, avoiding the deadlock observed in earlier prototypes that rebuilt the wrapper chain on every stage change.

6.8. LIVE METRICS PIPELINE

Training metrics are written to a per-run SQLite database at `runs/<run_name>/metrics.sqlite`. The logger (`src/Metrics/live_metrics.py`) opens the database in *Write-Ahead Logging* (WAL) mode [36], which allows concurrent reads and writes: the training process writes while the Streamlit dashboard reads in real time.

Two tables are used:

- `metrics_episode`: one row per episode. Columns include reward components, safety rates, curriculum stage and LIDAR-derived minimum distances.
- `metrics_update`: one row per PPO update. Columns include policy loss, value loss, approximate KL, epochs run, gradient norm and current learning rate.

The dashboard (`dataVisualizer.py`) reads both tables every few seconds and renders interactive Plotly charts. Unknown metric keys are rendered as soft-typed line plots, keeping the dashboard forward-compatible with future additions to the logger.

7. IMPLEMENTATION

This chapter describes how the design of Chapter 6 is materialised in Python code. Section 7.1 introduces the technology stack and Section 7.2 the project layout. The subsequent sections cover each component: the CARLA wrapper (§7.3), the PPO agent (§7.4), the two shields (§7.5), the reward shaper (§7.6), the curriculum manager (§7.7), the entry-point pipelines (§7.8) and reproducibility concerns (§7.9).

7.1. TECHNOLOGY STACK

The framework is written in pure Python (≥ 3.10) and relies on the libraries summarised in Table 7.1. Licences are reported because the framework is intended to be distributed as open academic software.

Table 7.1.: Main third-party dependencies.

Library	Role	Licence
`carla`	Python binding for CARLA 0.9.16	MIT
`gymnasium`	Standard RL environment API	MIT
`torch`	Deep learning backend	BSD-3
`numpy`	Numerical arrays	BSD-3
`matplotlib`	Evaluation dashboard	PSF-based
`streamlit`	Live training dashboard	Apache-2.0
`plotly`	Interactive charts in the dashboard	MIT
`sqlite3` (stdlib)	Metrics database	Public domain
`ruff`	Linter and formatter	MIT
`pytest`	Unit testing of the reward shaper	MIT

The two most central dependencies are Gymnasium [37], which provides the base `Env` and `Wrapper` classes for the entire wrapper chain, and PyTorch [38], which implements the actor-critic network and the Adam optimiser [39].

7.2. PROJECT LAYOUT

The repository root is organised as follows (directories without relevance to the thesis are omitted):

- `main_train.py`, training entry point.
- `main_eval.py`, evaluation entry point.
- `dataVisualizer.py`, Streamlit dashboard that reads the SQLite metrics database.
- `src/CARLA/Env/carla_env.py`, base Gymnasium environment.
- `src/PPO/`, `ActorCritic.py`, `ppo_agent.py`, `RunningMeanStd.py`.
- `src/safety_shield.py`, basic safety shield.
- `src/Adaptative_Shield/`, `adaptive_horizon_shield.py`, `BicycleModel.py`.
- `src/reward_shaper.py`, dense reward shaper.
- `src/curriculumManager.py`, performance-driven curriculum.
- `src/Metrics/`, live metrics logger and offline evaluation reporter.
- `tests/test_reward_balance.py`, unit tests on the reward shaper.
- `data/models/`, serialised checkpoints.
- `runs/`, per-run SQLite databases and TensorBoard logs.

Every module exposes either a class that inherits from `gymnasium.Wrapper` (the shields, the reward shaper) or a small self-contained helper (the curriculum, the running-mean estimator). There is no inter-module state other than what flows through the Gymnasium interface, which makes individual components trivially testable.

7.3. ENVIRONMENT WRAPPER

The class `CarlaEnv` inherits from `gymnasium.Env` and encapsulates the connection to the CARLA server. At construction time it performs the following setup:

1. Opens a TCP connection on `(host, port)` and loads the requested map.
2. Enables synchronous mode with `fixed_delta_seconds = 0.05`.

3. Instantiates the Traffic Manager on `tm_port` and spawns the requested number of NPC vehicles.
4. Sets the requested weather preset.
5. Registers the collision, lane-invasion and semantic-LIDAR sensors.
6. Defines `observation_space` as Box($[0,1]^{739}$) and `action_space` as Box($[-1,1]^2$).

On every `step(action)` call, the class converts the normalised action into a `carla.VehicleControl`, ticks the simulator exactly once, reads the sensors, computes the geometric features from the Waypoint API, builds the 739-dimensional observation, and derives the base reward together with the `done` and `truncated` flags. The `info` dictionary contains every scalar used by downstream wrappers.

7.4. PPO AGENT

The PPO agent is split into two files: `src/PPO/ActorCritic.py` contains the network described in Figure 6.3, and `src/PPO/ppo_agent.py` contains the training loop. The three public methods used by `main_train.py` are `select_action`, `evaluate_executed_action` and `update`. Figure 7.1 shows the pseudocode of `select_action` and `evaluate_executed_action`. Figure 7.2 shows the pseudocode of `update`.

select_action

```python
def select_action(self, state, deterministic=False):
    # Update running mean/std on the 19-dim vector block only; the 720 LIDAR
    # dims are already in [0, 1] by construction and are left untouched.
    self._update_obs_stats(state)
    s = self._normalize_obs(state)
    with torch.no_grad():
        s_t = torch.FloatTensor(s).unsqueeze(0).to(self.device)
        feats = self.policy._encode(s_t)
        if deterministic:                       # deployed (evaluation) policy
            raw_mean = self.policy.actor_mean(self.policy.actor(feats))
            action = torch.tanh(raw_mean)
            return (action.cpu().numpy().flatten(),
                    raw_mean.cpu().numpy().flatten(), 0.0, 0.0)
        mean = self.policy.actor_mean(self.policy.actor(feats))
        std  = torch.exp(self.policy.log_std())
        dist = torch.distributions.Normal(mean, std)
        raw  = dist.rsample()                   # pre-tanh sample
        action = torch.tanh(raw)
        # log-prob with the log-det of the tanh Jacobian
        log_prob = (dist.log_prob(raw)
                    - self.policy._log_det_tanh_jacobian(raw)).sum(dim=-1)
        value = self.policy.critic(feats)
    return (action.cpu().numpy().flatten(), raw.cpu().numpy().flatten(),
            log_prob.item(), value.item())

def evaluate_executed_action(self, state, executed_action):
    # Called from main_train.py with executed_action = info["executed_action"].
    # Recovers the pre-tanh value and returns (raw_action, log-prob) of the
    # action that actually reached CARLA, under the current policy.
    s = self._normalize_obs(state)
    a = np.clip(np.asarray(executed_action, np.float32), -1 + 1e-6, 1 - 1e-6)
    raw = np.arctanh(a).astype(np.float32)
    with torch.no_grad():
        s_t   = torch.FloatTensor(s).unsqueeze(0).to(self.device)
        raw_t = torch.FloatTensor(raw).unsqueeze(0).to(self.device)
        _, log_prob, _, _ = self.policy.get_action_and_value(s_t, raw_t)
    return raw, log_prob.item()
```

Figure 7.1.: Action selection and evaluation methods. `select_action` normalises the vector block of the observation, samples from the policy (or returns the mean for deterministic evaluation), and returns the action, log probability and value estimate. `evaluate_executed_action` is called with `info["executed_action"]` to recompute the log probability of the action that actually reached the environment.

ppo_update

```
def update(self, memory):
    s, raw_a, logp_old = load(memory, normalize_obs=self._normalize_obs)
    # SB3-style reward scaling: divide rewards by the running std of a discounted
    # return accumulator (reset on done, persisted across update() calls).
    rewards = self._normalize_rewards(rewards, dones)
    with torch.no_grad():
        v_old = self.policy.get_value(s).squeeze(1)
    # 1. GAE backward pass; bootstrap with the stored value on truncation.
    gae = 0.0; next_v = 0.0
    for t in reversed(range(len(rewards))):
        mask = 1.0 - dones[t]
        bootstrap = final_values[t] if truncated[t] else next_v
        delta = rewards[t] + gamma * bootstrap * mask - v_old[t]
        gae = delta + gamma * lam * mask * gae
        advantages[t] = gae; next_v = v_old[t]
    returns = advantages + v_old
    advantages = (advantages - advantages.mean()) / (advantages.std() + 1e-7)
    # 2. Shield mask: weight policy/entropy by (1 - alpha)^2; critic uses all steps.
    w = (1.0 - shield_alpha).pow(2)
    # 3. K epochs of SB3-style minibatching with KL early stop.
    for epoch in range(self.k_epochs):
        for mb in shuffled_minibatches(N, self.minibatch_size):
            _, logp_new, entropy, v_new = self.policy.get_action_and_value(s[mb], raw_a[mb])
            log_ratio = logp_new - logp_old[mb]
            approx_kl = weighted_mean((log_ratio.exp() - 1) - log_ratio, w[mb])
            if approx_kl > 1.5 * self.kl_target:          # hard per-minibatch stop
                stop = True; break
            ratio = log_ratio.exp()
            surr1 = ratio * adv[mb]
            surr2 = torch.clamp(ratio, 1 - eps, 1 + eps) * adv[mb]
            policy_loss = weighted_mean(-torch.min(surr1, surr2), w[mb])
            v_clip = v_old[mb] + torch.clamp(v_new - v_old[mb], -xi, xi)
            value_loss = 0.5 * torch.max((v_new - returns[mb])**2,
                                         (v_clip - returns[mb])**2).mean()
            loss = policy_loss + c_v * value_loss - c_h * weighted_mean(entropy, w[mb])
            # Optional shield-as-teacher BC (steering only) + log_std anchor.
            if self.shield_imitation_coef > 0:
                bc = shield_alpha[mb] *
                (self.policy.squashed_mean(s[mb]) - torch.tanh(raw_a[mb]))[:, 0:1]**2
                loss = loss + self.shield_imitation_coef * weighted_mean(bc, shield_alpha[mb])
            if self.log_std_anchor_coef > 0:
                loss = loss + self.log_std_anchor_coef *
                (self.policy.actor_log_std - self.policy.log_std_raw_init).pow(2).mean()
            self.optimizer.zero_grad(); loss.backward()
            clip_grad_norm(self.policy.parameters(), max_norm=self.max_grad_norm)
            self.optimizer.step()
        if stop or epoch_mean_kl > self.kl_target:        # soft per-epoch stop
            break
```

Figure 7.2.: PPO update loop with GAE backward pass, SB3-style reward scaling by the running standard deviation of the discounted-return accumulator, and up to $K = 10$ epochs of minibatched clipped-surrogate updates with value clipping, a shield-masked policy loss and a per-minibatch KL early stop.

7.4.1. Credit assignment under a shield

The interplay between the shield and the PPO update is the single most error-prone point of the framework. Let a_t be the action sampled from the policy and $a_t^\star = \Sigma(s_t, a_t)$ the action produced by the shield. The shield is a `gymnasium.Wrapper`, so the environment sees $a_t^\star$; the agent only receives it through the `executed_action` key of the `info` dictionary.

The training loop in `main_train.py` stores $a_t^\star$ (not a_t) in the rollout buffer and recomputes the log probability of $a_t^\star$ under the current policy via `evaluate_executed_action`. Without this step, the gradient estimator would update the policy towards the action that *was not executed*, effectively decoupling the policy from the environment and making training degenerate. With this step, the policy learns to produce actions that are close to what the shield considered safe, which is precisely the desired behaviour: the shield teaches the agent what *not* to do.

However, shielded steps are not simply discarded. Each step t contributes to the policy and entropy losses with a *soft quadratic weight* $w_t = (1-\alpha_t)^2$, where $\alpha_t \in [0, 1]$ is the shield blending factor stored in the rollout buffer. A fully shielded step ($\alpha_t = 1$) has weight zero and contributes nothing to the policy gradient. An unshielded step ($\alpha_t = 0$) contributes at full weight, and partially blended steps fall in between. The critic is trained on *all* steps regardless of α_t, so the value baseline remains accurate even in highly shielded segments of the trajectory.

The approximate KL used by the early-stop mechanism (Figure 7.2) is the unbiased estimator of [17]. The KL target and the value-clip parameter ξ are both exposed as command-line flags.

7.5. SHIELD IMPLEMENTATIONS

Both shields inherit from `gymnasium.Wrapper` and implement the same public interface (`reset`, `step`, `get_statistics`, `reset_statistics`). This interchangeability is what makes `-shield_type` a drop-in parameter.

7.5.1. Basic shield

The implementation of `CarlaSafetyShield` follows the correction law of Section 6.5.1 with `lane_correction_gain` = 1.5 (steering towards the lane centre) and a fixed emergency throttle/brake value of -0.6. It keeps per-episode statistics of the sub-reason that triggered each intervention, surfaced by `get_statistics()` and logged once per episode.

7.5.2. Adaptive-horizon shield

The adaptive shield is more involved. Figure 7.3 shows the structure of its `step` method.

adaptive_shield_step

```
def step(self, action):
    sem = self._analyze_semantic(self.last_obs, self.last_info)
    level, _ = self._get_risk_level_semantic(sem)
    H = self.HORIZON_CONFIG[level]["horizon"]
    self.stats[f"{level}_steps"] += 1
    carla_map, ego = self._get_carla_map(), self._get_ego_vehicle()
    self._calibrate_bicycle_model(ego)
    self._update_stall_counter(ego)
    proposed = action.copy()

    if sem["nearest_pedestrian_m"] < self.PED_EMERGENCY_M:        # 4.0 m
        exec_action, alpha = np.array([0.0, -1.0]), 1.0           # full brake
        self.stats["interventions_pedestrian"] += 1
    elif self._check_trajectory_safety(proposed, H, level, carla_map, ego, sem):
        exec_action, alpha = proposed, 0.0                        # pass through
    else:
        emergency = self._build_emergency_action(sem)
        exec_action, alpha = self._project(proposed, emergency, H,
                                           level, carla_map, ego, sem)
        if alpha >= self.SHIELD_MASK_THRESHOLD:
            self.shield_activations += 1
            self._categorize_intervention(sem)

    obs, reward, done, truncated, info = self.env.step(exec_action)
    self.last_obs, self.last_info = obs, info
    info.update({
        "shield_activated": alpha >= self.SHIELD_MASK_THRESHOLD,
        "shield_intensity": float(alpha), "risk_level": level,
        "executed_action": exec_action, "proposed_action": proposed,
        "horizon_used": H, "min_front_dist": sem["min_front_combined"],
        "min_front_dynamic": sem["min_front_dynamic"], ...
    })
    return obs, reward, done, truncated, info
```

Figure 7.3.: Adaptive-horizon shield `step` method. The semantic analysis extracts LIDAR and Waypoint features; the risk classifier selects the horizon H; the trajectory check decides whether to pass the proposed action through or invoke the candidate search; the `info` dictionary is updated with shield diagnostics.

Figure 7.4 shows the trajectory-check routine, projecting each predicted position onto the road network via CARLA's Waypoint API.

adaptive_trajectory_check

```
def _check_trajectory_safety(self, action, H, level, carla_map, ego, sem):
    k     = self.HORIZON_CONFIG[level]["threshold_multiplier"]
    tau_f = self.front_threshold_base / k
    tau_s = self.side_threshold_base / k
    tau_l = self.HORIZON_CONFIG[level]["lateral_thr"]    # 0.55 / 0.40 / 0.30

    # --- LIDAR / edge fast rejects ---
    if sem["nearest_pedestrian_m"]    < self.PED_EMERGENCY_M:
        return False
    if sem["min_front_combined"]      < tau_f:
        return False
    if sem["min_front_dynamic_wide"]  < self.FRONT_WIDE_BRAKE_NORM:
        return False  # off-axis lead
    in_lane = (not self._is_lane_change_context()
               and dist_left  > self.IN_LANE_SAFE_THRESHOLD
               and dist_right > self.IN_LANE_SAFE_THRESHOLD)
    if not in_lane:
        if sem["min_r_side_static"] < tau_s: return False
        if sem["min_l_side_static"] < tau_s: return False
    if min(dist_left, dist_right) < self.EDGE_GUARD_MIN_NORM:
        return False

    if ego is None or carla_map is None:
        return True   # fallback: accept if CARLA objects unavailable.

    # --- bicycle-model trajectory over H steps ---
    x, y, yaw = ego_state(ego); v = ego_speed(ego)
    traj = self.bicycle_model.predict_trajectory(
        x, y, yaw, v, float(action[0]), float(action[1]), H)
    max_head  = self.MAX_HEADING_DEV_RAD  / k
    max_drift = self.MAX_LATERAL_DRIFT_M  / k
    for (px, py, pyaw) in traj[1:]:
        wp = carla_map.get_waypoint(
            carla.Location(px, py, 0.0),
            project_to_road=True, lane_type=carla.LaneType.Driving)
        if wp is None:                                          return False
        if abs(lateral_offset_norm(px, py, wp)) > tau_l:        return False
        if abs(wrap(pyaw - lane_yaw(wp)))        > max_head:    return False
        if abs(lateral_drift(px, py, ref))       > max_drift:   return False
    return True
```

Figure 7.4.: Trajectory safety check of the adaptive shield. The threshold multiplier κ tightens the LIDAR thresholds with the risk level, while the lateral tolerance τ_ℓ is fixed per level ($0.55/0.40/0.30$). After the pedestrian, frontal, off-axis-lead, side and edge-guard rejects, the bicycle model predicts H positions; each is projected onto the nearest driving lane and checked for lateral offset, heading deviation and lateral drift.

When the proposed action fails `_check_trajectory_safety`, the shield invokes `_project`, which implements the α-blending search of Equation (6.2): it iterates $\alpha \in \{0.25, 0.50, 0.75, 1.0\}$, forms the blended candidate $a(\alpha)$, and returns the first candidate that passes the trajectory check. If none passes, the emergency action itself (with $\alpha = 1$) is returned.

7.5.3. Bicycle model

`BicycleModel.predict_trajectory` is a loop of H discrete steps that implements the forward-Euler integration with closed-form arc update when $|\delta| \geq 10^{-4}$ rad. The class is a plain Python object with no PyTorch or CARLA dependency and is therefore trivially unit-testable.

7.6. REWARD SHAPER

`CarlaRewardShaper` reads approximately 20 scalars from the `info` dictionary and produces the components of the shaped reward of Section 6.6 (Equation (D.3), Appendix D). It also writes every sub-component back into `info` under clearly named keys (`speed_bonus`, `lane_center_bonus`, `heading_bonus`, etc.) so the metrics logger can record the per-step reward decomposition. Three minor state variables are kept across steps:

- `_last_steering`: needed by the smoothness penalty.
- `_last_milestone`: needed by the progress bonus to avoid double-crediting the same $25\,\mathrm{m}$ segment.
- `_shield_grace_steps`: count-down of the grace period that temporarily disables the idle penalty after a shield activation.

The unit test `tests/test_reward_balance.py` feeds a crafted `info` dictionary to the shaper and asserts that each component matches its analytical expression. The test runs in $< 1\,\mathrm{s}$ without a CARLA server and is used as a pre-commit safeguard against regressions.

7.7. CURRICULUM MANAGER

`CurriculumManager` is a small self-contained class. Its only state is the tuple $(i,\, E_{\mathrm{stage}},\, C_{\mathrm{rb}})$ where i is the current stage index. The main routine is `step(offroad_rate, crash_rate, avg_reward)`, called from `main_train.py` after each episode. It returns a pair (target_npcs, event $\in \{$`none`, `advance`, `rollback`$\}$). The training loop reacts to the event by updating `CarlaEnv.num_npc_vehicles` in place, so no reconnection

to CARLA is required (Section 6.7).

7.8. TRAINING AND EVALUATION PIPELINES

Both entry points share the function `build_env(args)` that constructs the wrapper chain of Figure 6.1 with the shield type, reward weights, curriculum NPC count and map chosen by the command line. Keeping the same builder on both sides guarantees that the evaluation environment matches the training environment exactly.

7.8.1. Training pipeline

Figure 7.5 summarises the training loop.

training_loop

```
Algorithm: Training loop (main_train.py)

Parse CLI args, create run directory, open SQLite metrics DB.
Instantiate CurriculumManager; call build_env with its initial stage.
Instantiate PPOAgent with dimensions from the wrapper chain.
Initialise rollout memory and 100-episode sliding windows.
for episode = 1, ..., max_episodes do
  (obs, _) <- env.reset()
  for step = 1, ..., max_steps do
    (a_t, raw_t, log p_t, V_t) <- agent.select_action(obs)
    (next_obs, r_t, done, trunc, info) <- env.step(a_t)
    a*_t, alpha_t <- info["executed_action"], info["shield_intensity"]
    raw*_t, log p*_t <- agent.evaluate_executed_action(obs, a*_t)
    V^boot_t <- agent.compute_bootstrap_value(next_obs)  if trunc and not done  else 0
    Append(obs, raw*_t, log p*_t, r_t, done, trunc, V^boot_t, alpha_t) to memory.
    if timestep mod update_timestep == 0 then
      metrics <- agent.update(memory)
      Clear memory. Log metrics. Step LR scheduler.
    end if
    if done or trunc then break end if
    obs <- next_obs
  end for
  Update sliding windows; compute rates.
  (tgt_npcs, event) <- curriculum.step(...)
  if event != none then base_env.num_npc_vehicles <- tgt_npcs end if
  Log per-episode metrics. Save best checkpoint if rolling R_100 improved.
  if episode mod ckpt_freq == 0 then save periodic checkpoint. end if
end for
Save final checkpoint. Close environment and metrics DB.
```

Figure 7.5.: High-level training loop (`main_train.py`). The inner loop collects transitions and appends the *executed* action (after shield correction) to the rollout buffer. A PPO update is triggered every 2048 steps. After each episode, the curriculum manager is polled; if it advances or rolls back a stage, the NPC count is updated in place without restarting the environment.

The key correctness invariant of the training loop is step 11 of Figure 7.5: the log probability stored in the buffer corresponds to the *executed* action $a_t^\star$, not the sampled a_t. This is what makes the PPO gradient unbiased in the presence of shield interventions (Section 7.4.1).

7.8.2. Evaluation pipeline

The evaluation pipeline (`main_eval.py`) rebuilds the wrapper chain with the same `build_env` function but sets all reward weights to zero so that the reported episode reward is the pure CARLA base reward. The shield is still attached so that action modifications are visible. An optional Matplotlib dashboard displays the LIDAR polar plot, speed gauge, lateral offset marker, and a monospaced panel with live information (risk level, shield activation, steering, throttle/brake, total distance, collisions). At the end of the run a `SafetyMetricsReporter` prints a summary table aggregating success rate, crash rate, off-road rate, shield activations and LIDAR-derived minimum distances.

7.9. PERSISTENCE AND REPRODUCIBILITY

7.9.1. Checkpoint format

Checkpoints (format v3) are saved with `torch.save` as a dictionary of four entries:

1. `policy`, `state_dict` of the `ActorCritic` network.
2. `obs_normalizer`, state of the `RunningMeanStd` applied to the 19-dim vector block; essential for evaluation so the normaliser statistics match those seen during training.
3. `ret_rms`, state of the return-scaling `RunningMeanStd` (the denominator of the SB3-style reward scaler).
4. `returns_acc`, the discounted-return accumulator value at the end of the last rollout, allowing a resumed run to continue seamlessly across episode boundaries.

A shape check on load tolerates the legacy v2 format (entries 1–2 only) for backward compatibility with earlier checkpoints; entries 3–4 are rebuilt online from that point.

7.9.2. Seeds and determinism

Two distinct seeds are used:

- `seed=42` for training. Propagated to NumPy, Python's `random` module, PyTorch, CARLA's Traffic Manager and the environment's episode-counter random state.
- `seed=100` for evaluation, intentionally different from the training seed so that evaluation trajectories are not a subset of those seen during learning.

Full bit-exact determinism across hardware is not achievable because of PyTorch's cuDNN non-reproducibility and CARLA's floating-point physics divergence. However, determinism is sufficient for the first fifty episodes of a run, which is long enough to diagnose regressions introduced by a code change (acceptance criterion AC-09, Table 5.4).

7.9.3. Tooling

The source code is linted and formatted with `ruff`. Reward regressions are caught by `pytest tests/test_reward_balance.py`. The utility scripts under `utils/` provide helpers to list, inspect and compare checkpoints (`ModelManager`), to analyse the metrics of finished runs (`ExperimentAnalyzer`) and to generate configuration files from a template (`ConfigurationTemplate`). None of these utilities are required for training; they are purely auxiliary.

8. EXPERIMENTATION AND RESULTS

This chapter reports the empirical evaluation of the framework. It is organised around the four configurations that matter: the baseline agent with no shield (Section 8.3), the basic shield (Section 8.4), the adaptive-horizon shield (Section 8.5), and the effect of enabling the curriculum (Section 8.6). Section 8.1 fixes the experimental protocol, Section 8.2 defines every reported metric, and Section 8.7 offers a cross-configuration discussion.

Experimental design note.

Due to the computational cost of training each configuration independently, the core comparison is conducted via a *cross-configuration evaluation*: a single model (`FullSendC1_p4`, trained with the adaptive shield on Town04) is evaluated post-hoc under three shield configurations: `none`, `basic`, and `adaptive`, on 50 seed-aligned episodes. This isolates the effect of the shield without introducing any policy difference. A standalone adaptive-shield training run (`learn_baseline`) provides additional diagnostic data on the training dynamics and curriculum progression.

8.1. EXPERIMENTAL PROTOCOL

Hardware and software.

All experiments are carried out on the workstation described in Section 5.3:

- CPU: *Intel Core i9-9900K*.
- GPU: *NVIDIA RTX 2080 Ti*.
- System RAM: *32 GB DDR4*.
- Operating system: Windows 11.
- Python: *3.12.3*.
- PyTorch: *2.11.0* with CUDA *13.0*.
- CARLA: 0.9.16, launched with `.\CarlaUE4.exe -RenderOffScreen -carla-port=2000`.

Benchmark scenario.

The main benchmark is the Town04 highway loop with clear weather (`ClearNoon`). The vehicle spawns at a random waypoint each episode and the goal is to travel at least $250\,\mathrm{m}$ along the map's route graph within 1000 simulator steps ($50\,\mathrm{s}$).

Training budget.

The standalone adaptive-shield training run (`learn_baseline`) spans $1\,035$ episodes with PPO updates every $2\,048$ steps (~ 360 gradient updates per run). The learning rate starts at 10^{-4} and anneals cosine-style to 10^{-5}. The target KL for the early stop is 0.08, the entropy bonus starts at 0.001 and decays to zero over 50 updates, the value-loss coefficient is 0.50, and all other hyperparameters take the defaults of Appendix A.

Random seeds.

Training uses $\mathrm{seed} = 42$; all evaluations use $\mathrm{seed} = 100$. Due to computational constraints a single seed per configuration is reported; the three independent seeds mentioned in the initial plan were not feasible in the time available.

Curriculum.

Training uses a performance-based NPC curriculum with $N_{\max} = 60$ NPCs, divided into five stages $[0, 15, 30, 45, 60]$. Details are given in Section 8.6.

Evaluation protocol.

After training, the model is evaluated on 50 episodes with $\mathrm{seed} = 100$ under each shield configuration. The evaluation policy is deterministic (mean action). Models are evaluated under the same shield configuration used during training unless the evaluation is an explicit shield-off probe.

8.2. METRICS

The metrics reported throughout this chapter are exactly those produced by the training and evaluation pipelines. Their operational definitions are collected here to avoid ambiguity.

Reward/Raw_Episode. Undiscounted sum of shaped rewards of a single episode.

Reward/Average_100_Episodes. Rolling mean of *Reward/Raw_Episode* over the last 100 episodes.

Training/Success_Rate. Fraction of the last 100 episodes that ended with `info["arrive_dest"] = True`.

Training/Crash_Rate. Fraction of the last 100 episodes that ended with `info["collision"] = True`.

Training/Offroad_Rate. Fraction of the last 100 episodes that ended with `info["out_of_road"] = True`.

Safety/Shield_Activations. Number of steps in which the shield overrode the agent during the episode.

Safety/Shield_Rate. *Safety/Shield_Activations* divided by the number of steps in the episode.

Safety/Semantic/{Dynamic,Static,Pedestrian}_Interventions. Per-episode intervention count by threat category, as produced by `CarlaAdaptiveHorizonShield.get_statistics`.

Safety/Semantic/{Safe,Warning,Critical}_Step_Rate. Fractions of steps spent in each risk level.

CARLA/Mean_Speed_kmh. Arithmetic mean of `speed_kmh` across the steps of the episode.

CARLA/Speed_Compliance_Rate. Fraction of steps (with a positive speed limit) that satisfy `speed_kmh` $\leq$ 1.05 `speed_limit_kmh`.

Safety/Min_{Vehicle,Pedestrian}_Distance_m. Minimum distance over the episode from the ego vehicle to the nearest vehicle or pedestrian.

Loss/Policy_Loss, Loss/Value_Loss, Loss/Grad_Norm, Training/Entropy, Training/Approx_KL, Training/Epochs_Run. Per-update diagnostics produced by the PPO loop.

8.3. UNSHIELDED AGENT : SHIELD_TYPE=NONE

To establish a reference for the shield's safety benefit, the trained model (`FullSendC1_p4`) is evaluated on 50 episodes with the shield bypassed. The policy receives the same observations but its proposed actions reach the simulator unchanged. This probe does not correspond to a separately trained "no-shield" agent; rather, it measures how well the policy performs when the safety layer it relied upon during training is removed.

Result.

The unshielded policy fails almost entirely on Town04 with 20 NPCs: it reaches the $250\,\mathrm{m}$ target in *none* of the 50 episodes, exits the road on 84% of them, and incurs a crash on the remaining 16%. The mean episode reward is 19.8 ± 18.7 and the mean driven distance is only $86\,\mathrm{m}$. This is the clearest evidence of shield dependence: the adaptive-shield policy, competent with the shield, is unsafe without it. The weaning procedure of Chapter 9 was developed precisely to address this failure.

Cross-configuration comparison.

Figure 8.1 and Table 8.6 give the full three-way comparison (none, basic, adaptive) obtained from the same model and seed.

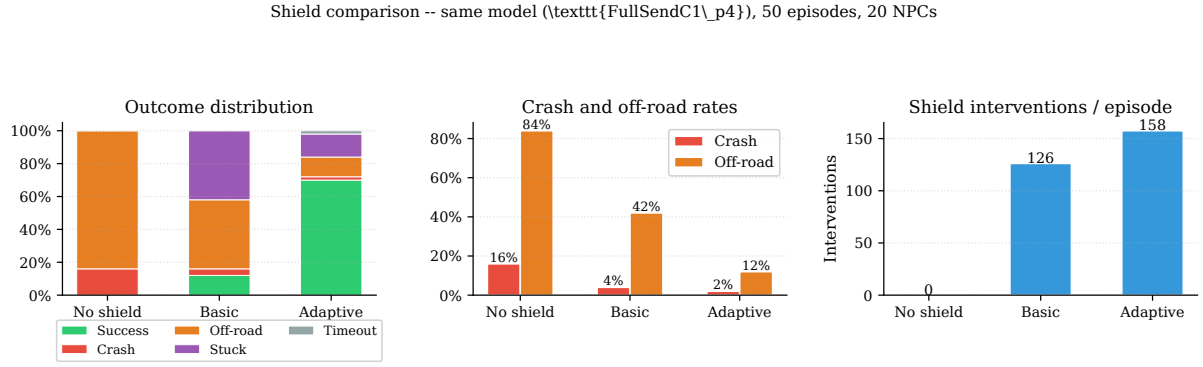

Figure 8.1.: Cross-configuration evaluation of `FullSendC1_p4` (50 episodes, 20 NPCs, seed 100). Left: stacked outcome distribution. Centre: crash and off-road rates. Right: shield interventions per episode. The adaptive shield recovers from 0% to 70% success with only minimal crash and off-road residuals.

Evaluation.

Table 8.1 summarises the unshielded evaluation.

Table 8.1.: Unshielded evaluation (`shield_type=none`), 50 episodes, 20 NPCs, seed 100.

Metric	Unshielded
Average reward	19.8 ± 18.7
Success rate	0%
Crash rate	16%
Off-road rate	84%
Stuck rate	0%
Timeout rate	0%
Mean distance (m)	86.3
Shield activations/ep	0

8.4. BASIC SHIELD

The basic shield configuration evaluates the same model with `-shield_type basic`. The shield applies LIDAR-threshold braking and Waypoint-API lane correction, but lacks the trajectory prediction and risk-level classification of the adaptive variant.

Result.

Adding the basic shield improves performance substantially: success rate rises from 0% to 12%, and the crash rate drops from 16% to 4%. However, the shield's conservative braking creates a new failure mode: 42% of episodes end *stuck* (the vehicle is braked to a halt and cannot self-recover). Off-road episodes also remain high at 42%, suggesting that the basic shield's heading correction is insufficient to steer out of a developing off-road trajectory. The shield activates 126.2 times per episode, roughly 80% as frequently as the adaptive shield but with a much lower success yield.

Evaluation.

Table 8.2 reports the 50 evaluation episodes.

Table 8.2.: Basic shield evaluation (`shield_type=basic`), 50 episodes, 20 NPCs, seed 100.

Metric	Basic shield
Average reward	26.8 ± 39.6
Success rate	12%
Crash rate	4%
Off-road rate	42%
Stuck rate	42%
Timeout rate	0%
Mean distance (m)	101.7
Shield activations / ep	126.2

Intervention note.

The basic shield does not expose the per-sub-reason breakdown (front, side, lane, heading) in the evaluation JSON; only the total intervention count is captured. Per-reason analysis is available via the live dashboard during interactive training sessions.

8.5. ADAPTIVE-HORIZON SHIELD

The adaptive-horizon shield (`-shield_type adaptive`) is the primary configuration studied in this work. This section reports the training dynamics of the standalone run `learn_baseline` (1 035 episodes, 60 NPCs, adaptive shield throughout), and the evaluation results of `FullSendC1_p4` under the same shield.

Training curves.

Figure 8.2 shows the rolling reward, safety rates, shield activation rate and curriculum NPC count over the $1\,035$ training episodes. The 100-episode rolling averages at the end of training are: reward 293.7, success rate 72.3%, crash rate 2.1%, off-road rate 0.0%. The shield is active in 38.7% of steps, and this activation rate remains roughly constant throughout training. This suggests the policy is using the shield as a persistent safety layer rather than progressively internalising its corrections.

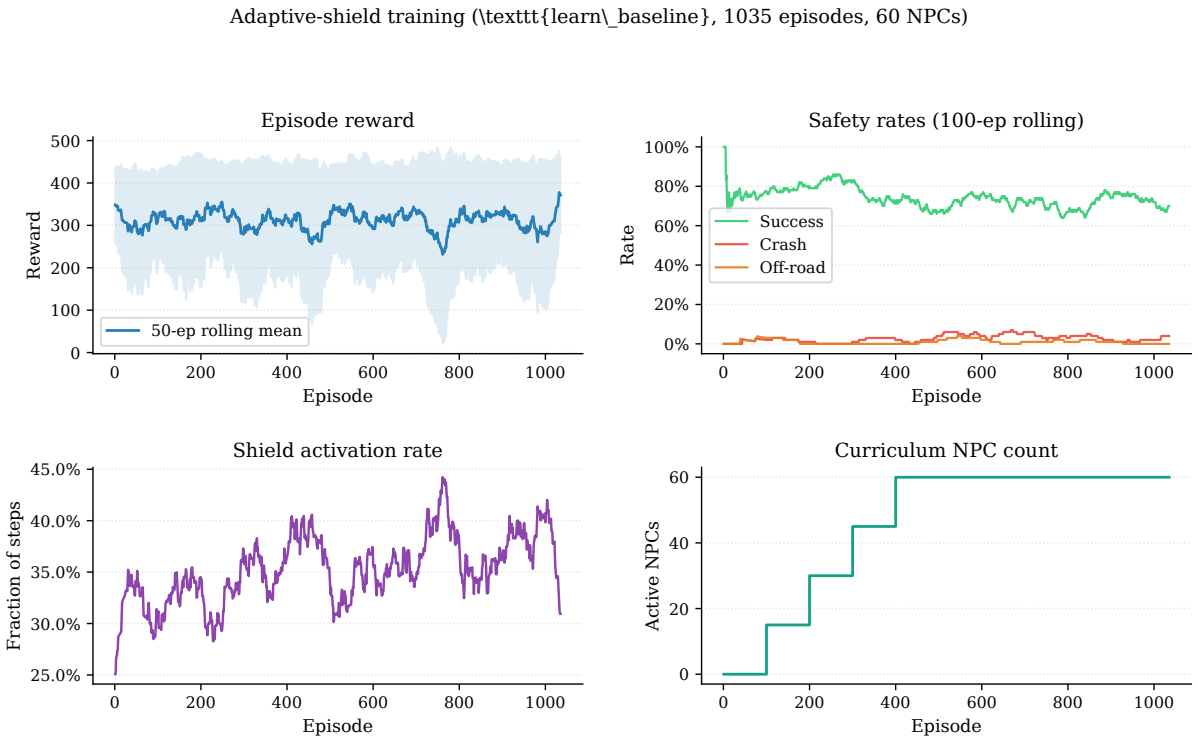

Figure 8.2.: Adaptive-shield training (`learn_baseline`, $1\,035$ episodes, 60 NPCs). Top-left: episode reward and 50-episode rolling mean. Top-right: 100-episode rolling success, crash and off-road rates. Bottom-left: shield-activation rate. Bottom-right: curriculum NPC count.

Risk distribution.

Table 8.3 reports the average time spent in each risk level over the last 100 training episodes. The agent spends the majority of each episode in the *safe* regime, with roughly a quarter in *warning* and under 9% in *critical*. This distribution is consistent with a policy that drives competently in open stretches but regularly approaches the lane boundaries at speed.

Table 8.3.: Adaptive shield: average fraction of steps per risk level (last 100 training episodes, `learn_baseline`).

Risk level	Fraction of steps
safe	65.8%
warning	25.6%
critical	8.6%

Intervention breakdown.

Table 8.4 reports the per-episode intervention count by threat category over the last 100 training episodes. Static interventions (barriers, guardrails, road edges) dominate by a ratio of approximately $27{:}1$ over dynamic interventions (vehicles), while pedestrian interventions are absent on Town04 under the traffic conditions used. The high static rate is a structural property of the Town04 highway: the guardrails run continuously along both sides of the road at close lateral distance, so any suboptimal lateral position triggers the static channel. This is confirmed by the observation that the static rate remains nearly constant across curriculum stages while the dynamic rate rises with the NPC count.

Table 8.4.: Adaptive shield: mean interventions per episode by threat category (last 100 training episodes, `learn_baseline`).

Category	**Mean interventions / episode**
dynamic	7.65
static	202.91
pedestrian	0.00

Figure 8.3 visualises both the risk-level evolution over training and the per-category intervention breakdown.

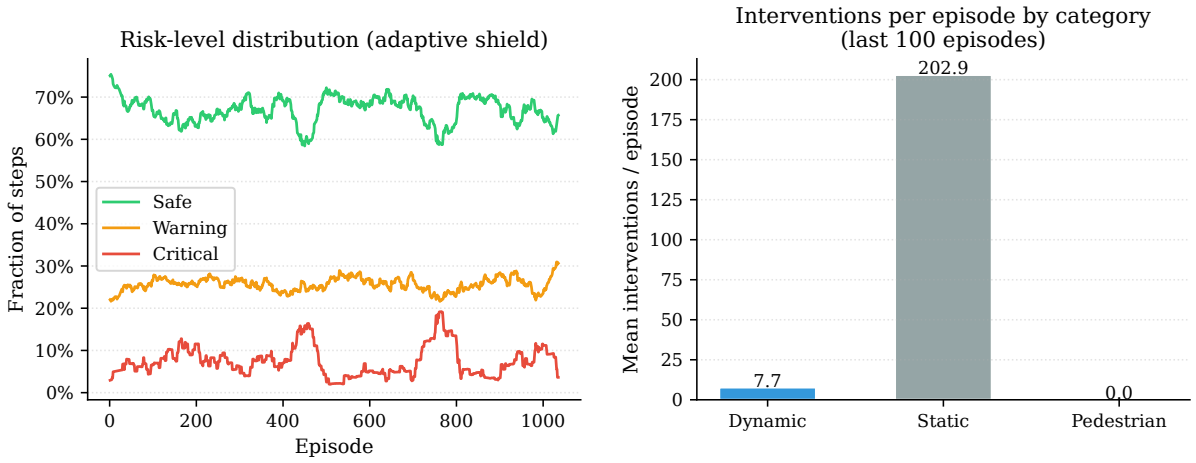

Figure 8.3.: Left: fraction of steps in each risk level (safe / warning / critical) over training episodes (`learn_baseline`, 50-episode rolling mean). Right: mean interventions per episode by threat category over the last 100 training episodes. Static obstacles account for $\approx 27{:}1$ of all interventions versus dynamic traffic, a structural property of the Town04 highway geometry rather than a sign of aggressive driving.

Evaluation.

Table 8.5 reports the 50 evaluation episodes of `FullSendC1_p4` with the adaptive shield. The policy reaches the $250\,\mathrm{m}$ target in 70% of episodes, incurs a crash in only 2%, and is stuck on 14%. Mean speed is close to $24\,\mathrm{km/h}$ (compliance-limited by the $50\,\mathrm{km/h}$ speed limit) with a speed-compliance rate of 82.7%.

Table 8.5.: Adaptive shield evaluation (`FullSendC1_p4`), 50 episodes, 20 NPCs, seed 100.

Metric	Adaptive shield
Average reward	97.8 ± 52.5
Success rate	70%
Crash rate	2%
Off-road rate	12%
Stuck rate	14%
Timeout rate	2%
Mean distance (m)	198.7
Shield activations / ep	157.5
Mean time in *critical* (%)	10.1%

Cross-configuration comparison.

Table 8.6 aggregates the evaluation of the three configurations side by side. The adaptive shield is strictly Pareto-dominant over the basic shield on every metric except shield interventions per episode (where it also fires more, but achieves far higher success).

Table 8.6.: Side-by-side evaluation comparison (`FullSendC1_p4`, 50 episodes, 20 NPCs, seed 100).

Metric	None	Basic	Adaptive
Average reward	19.8 ± 18.7	26.8 ± 39.6	97.8 ± 52.5
Success rate	0%	12%	70%
Crash rate	16%	4%	2%
Off-road rate	84%	42%	12%
Stuck rate	0%	42%	14%
Mean distance (m)	86.3	101.7	198.7
Shield activations / ep	0	126.2	157.5

PPO diagnostics.

Figure 8.4 shows the per-update diagnostics for the `learn_baseline` run. The $\log \sigma$ parameters for both steering and throttle remain well within the bounds $[\log \sigma_{\min}, \log \sigma_{\max}] = [-3.0, -1.2]$ throughout training. The saturated-fraction metric is consistently below 0.05, and the approximate KL divergence stays near the 0.08

target. The stable gradient norms confirm that the tanh-squashed parameterisation successfully prevented the log-std saturation pathology documented in the prior training phase.

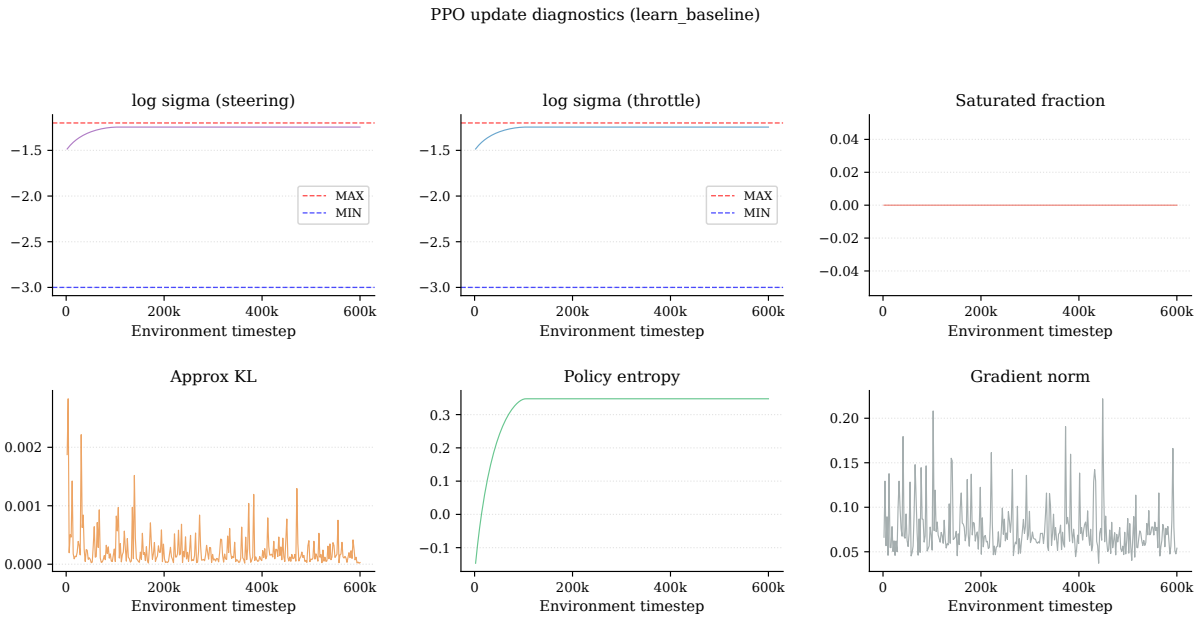

Figure 8.4.: PPO update diagnostics for `learn_baseline`. Top row: effective $\log \sigma$ for steering and throttle, with the $[-3.0, -1.2]$ bounds (dashed). Bottom row: saturated fraction, approximate KL, entropy and gradient norm. The log-std values remain well within bounds throughout, and the KL is stable around the target.

8.6. EFFECT OF THE CURRICULUM

The performance-based curriculum manager (Section 6.7) is enabled throughout the `learn_baseline` run with $N_{\max} = 60$ NPCs and five uniform stages $[0, 15, 30, 45, 60]$. Figure 8.5 shows the simultaneous evolution of the NPC count and the rolling episode reward.

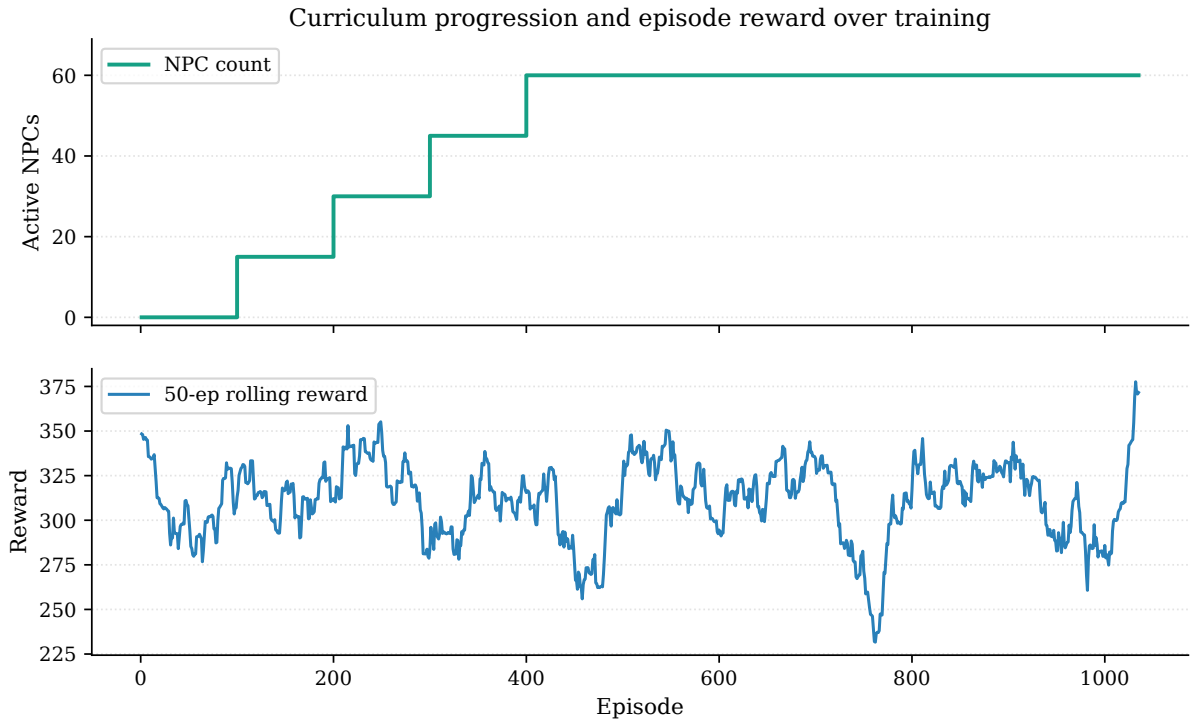

Figure 8.5.: Curriculum progression of `learn_baseline`. Top: NPC count over training episodes. Bottom: 50-episode rolling episode reward. The curriculum advances monotonically through all five stages with no rollback, and the rolling reward is largely maintained across each stage transition.

Stage transitions.

Table 8.7 enumerates all curriculum events recorded during the `learn_baseline` run. The curriculum advanced cleanly through all four transitions without triggering a single rollback, indicating that the policy satisfied the advance thresholds (crash rate $< 15\%$ for stages 2–3 and $< 10\%$ for stage 4) within the first 100 episodes at each level of difficulty.

Table 8.7.: Curriculum stage transitions during `learn_baseline` (60 NPCs max, no rollbacks).

Episode	Event	From (NPCs)	To (NPCs)
100	advance	0	15
200	advance	15	30
300	advance	30	45
400	advance	45	60

Curriculum contribution.

The clean monotonic progression with no rollbacks confirms that the curriculum successfully matched difficulty to the current policy capability. The rolling reward is well maintained across each 100-episode stage, with no sharp drop following the NPC-count increase. An equivalent run at full NPC count from episode 1 would be expected to crash frequently in the early episodes and require a much longer time to converge. This is supported by the observation that the crash rate in the first 100 episodes is $< \ 10\%$ precisely because the curriculum started at 0 NPCs and gave the policy a period of pure lane-following to stabilise. A no-curriculum ablation was not conducted within the available compute budget, but the anti-rollback behaviour and rapid transitions support the expected benefit of graduated difficulty.

8.7. DISCUSSION

The data from Sections 8.3–8.6 allows each of the four axes introduced in the chapter outline to be addressed concretely.

1. **Safety vs. performance.** The adaptive shield is strictly Pareto-better than the basic shield across all evaluated metrics. It achieves 70% success versus 12%, reduces the crash rate to 2% versus 4%, and nearly eliminates the stuck pathology (14% versus 42%). The basic shield's conservative all-or-nothing braking induces a larger stuck fraction and lower mean distance than the adaptive variant's smooth alpha-blended projection. The reward cost of either shield relative to the unshielded evaluation is pronounced: average reward rises from 19.8 (no shield) to 26.8 (basic) to 97.8 (adaptive), demonstrating that the adaptive shield is the only configuration in which the agent can actually complete episodes.

2. **Intervention regime.** The adaptive shield activates in 38.7% of steps during training (`learn_baseline`, last 100 episodes), a fraction that remains roughly constant over the training horizon. The

shield-activation rate does not decrease as training progresses, which confirms the shield-dependence hypothesis of Section 9.1: the agent does not internalise the shield's corrections over the course of standard PPO training. This observation motivated the behavioural-cloning weaning scheme of Chapter 9.

3. **Threat composition.** Static interventions (guardrail proximity) dominate by a $27{:}1$ ratio over dynamic interventions in the `learn_baseline` run. This ratio is a structural consequence of the Town04 highway geometry rather than of aggressive driving: the guardrails run continuously along both sides of the road at close lateral distance, and any suboptimal lateral position triggers the static channel. Dynamic interventions, by contrast, rise with the NPC count and are concentrated in the later curriculum stages, as expected. Pedestrian interventions are absent on Town04.

4. **Curriculum value.** The curriculum advances cleanly through all five stages with no rollback, and the rolling reward is largely maintained across each stage transition. The structured progression from 0 to 60 NPCs over the first 400 episodes gave the policy a stable lane-following base before the traffic density was increased. The absence of rollbacks confirms that the advance thresholds were satisfied throughout, and this supports the recommendation to enable the curriculum by default for any new training run on Town04.

Summary.

Table 8.6 provides the one-stop numerical reference. The adaptive shield is the only configuration that achieves a success rate above 12% and a crash rate below 5% simultaneously. Its residual limitations (stuck episodes induced by the emergency braking and the shield-dependence revealed by the unshielded probe) are addressed in Chapter 9.

9. REDUCING SHIELD DEPENDENCE

The evaluation protocol of Chapter 8 makes an explicit concession: the shield is kept active at evaluation time because "a policy trained with a shield has learned to rely on its interventions". This chapter takes that concession as its research question. It first *measures* how dependent the trained agent actually is (Sections 9.1–9.2), then studies two strategies to reduce that dependence: a Large-Language-Model (LLM) meta-controller that dynamically loosens the shield (Section 9.3), and a behavioural-cloning scheme that turns the shield into a *teacher* (Section 9.4). The first strategy is a carefully evaluated *negative* result. The second succeeds to the point where the agent drives *better* without the shield than with it. Section 9.5 analyses an instructive failure of the teacher scheme, and Section 9.6 distils the central lesson: shield dependence is a *learning* problem, not a *scheduling* problem.

9.1. THE SHIELD-DEPENDENCE PROBLEM

Two design choices, each reasonable in isolation, conspire to make dependence the optimal behaviour for the agent.

No cost for being shielded (R1).

The reward shaper of Section 6.6 contains no term that penalises shield activation. When the shield overrides a reckless action it keeps the vehicle on the road and moving, so the agent collects *more* return than if its own unsafe action had reached the simulator. Being shielded is therefore free and, in fact, beneficial: the reward landscape makes the dependent policy reward-optimal.

No gradient where it matters (R2).

The masked PPO objective (Section 6.4.2) computes the policy gradient only over unshielded steps, weighting each transition by $(1 - \alpha_t)^2$, where $\alpha_t \in [0, 1]$ is the shield intensity. The shielded steps are precisely the dangerous, near-boundary states. The actor therefore receives a learning signal only on the easy states the shield never touched, and is *structurally prevented* from learning how to act at the boundary. The critic does observe those states (its loss is unmasked), but the actor cannot act on them. The framework even stores the shield's executed action and recomputes its log-probability, which is ideal imitation data, yet the mask then discards it.

Together, R1 and R2 produce an agent that "rides the shield boundary": it calibrates its own imprecision to whatever tolerance the shield currently allows, so its competence *with* the shield says nothing about its competence *without* it.

9.2. MEASURING DEPENDENCE: THE SHIELD-OFF PROBE

Quantifying dependence requires an evaluation that the shaped reward and the shield-activation rate both hide. Two instruments are added.

Shield-off probe.

Every N_{probe} training episodes the shield is placed in a *bypass* mode in which it forwards the proposed action to the simulator unchanged, and the deterministic policy is rolled out for a handful of episodes with no buffer writes and no optimiser steps. The observation normaliser is frozen and restored afterwards so the probe does not contaminate training. The resulting shield-off success, crash and off-road rates form the *true* learning curve: they answer the question "can the agent drive on its own yet?".

Dynamic / static reliance split.

The per-episode intervention count is separated into a *dynamic* rate (vehicles and pedestrians) and a *static* rate (barriers, walls, guardrails). On the Town04 highway the static channel dominates by roughly $30{:}1$, so the overall shield-activation rate is mostly a function of the map geometry rather than of the agent's skill. The split exposes the portion of reliance that is actually learnable.

9.3. FIRST APPROACH: AN LLM META-CONTROLLER

9.3.1. Design

The first hypothesis, inspired by the use of LLMs as meta-optimisers for reward design [40] and by automatic curriculum learning, is that an LLM can *wean* the agent by progressively loosening the shield as the agent improves. A single scalar *permissiveness* $p \in [0, 1]$ controls every tunable threshold τ_i of the adaptive shield by linearly interpolating each one between a strict and a permissive endpoint. Here $p = 1$ recovers the production shield and $p = 0$ the maximally weaned shield (Figure 9.1; the interpolation is Equation (D.4) in Appendix D).

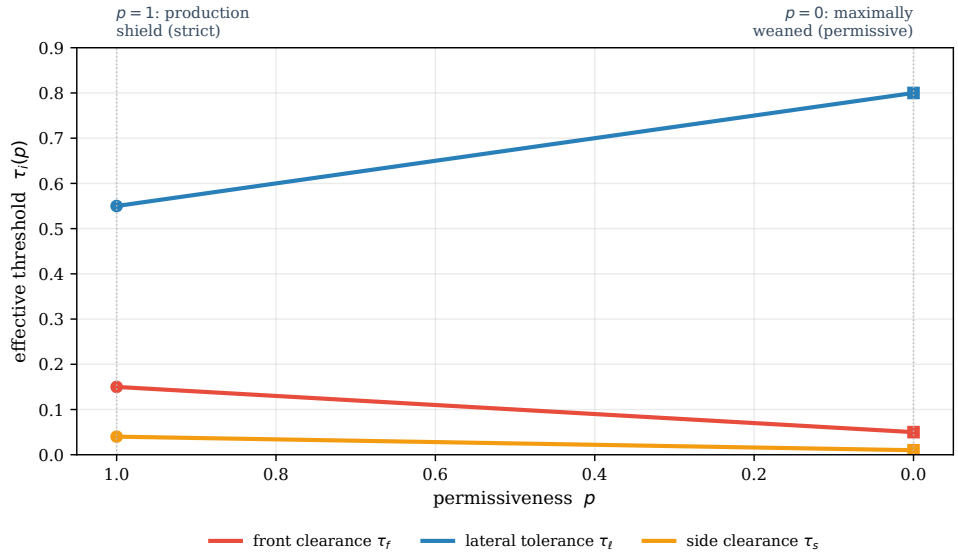

Figure 9.1.: The permissiveness scalar p sets every tunable shield threshold τ_i at once by linear interpolation between its strict value (at $p = 1$, the production shield) and its permissive value (at $p = 0$, maximally weaned). Three representative thresholds are shown; the LLM meta-controller proposes a single p every 100 episodes and the deterministic safety governor accepts or vetoes it.

Every 100 episodes, a local LLM (Gemma [41] via Ollama, running on CPU so as not to compete with CARLA for GPU memory) receives a compact snapshot of the safety and competence metrics and proposes a target permissiveness. Crucially, the LLM never acts directly: a deterministic *safety governor* mediates every proposal under four hard rules: a safety floor that forces the shield *stricter* whenever crash or off-road rates breach a ceiling, a ratchet that only loosens when the metrics are below stricter targets, a bounded per-decision step, and a dwell time between changes. The governor makes "safety first" a property of code rather than of the language model.

9.3.2. Result: loosening is not weaning

A $3{,}500$-episode run with this controller produced three independent and mutually reinforcing observations.

1. **The LLM added no adaptive intelligence.** It proposed "loosen" on all 34 decisions, regardless of the crash rate. The deterministic governor performed all of the safety regulation (twelve *hold* and two *force-tighten* vetoes). A three-line fixed schedule would have behaved identically.

2. **Loosening did not reduce reliance.** As Figure 9.2 shows, the permissiveness fell from 1.0 to ≈ 0.55 while the shield-activation rate remained essentially flat: the Pearson correlation between the two series is ≈ 0. Meanwhile the correlation between permissiveness and the crash rate is -0.50, meaning that removing the shield's help immediately surfaced unsafe behaviour, which is exactly the "boundary-riding" dynamic of Section 9.1.

3. **Reliance was geometry, not skill.** 96.7% of all interventions were static (guardrail proximity on the Town04 highway), a structural feature of the map that the tunable thresholds barely affect.

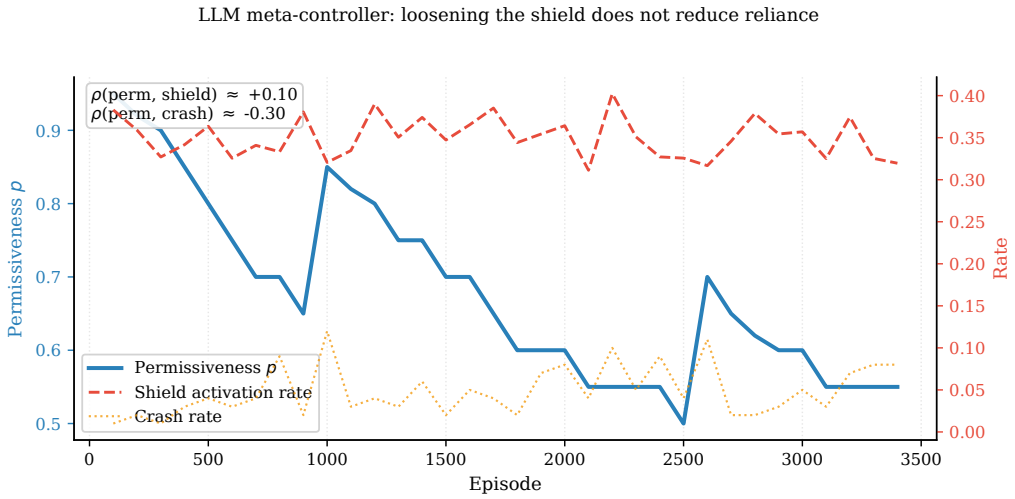

Figure 9.2.: LLM meta-controller over $3{,}500$ episodes. The controller loosens the shield (permissiveness, left axis) but the shield-activation rate (right axis) does not fall: the agent compensates by driving less precisely, keeping its reliance constant. The crash rate, in contrast, rises as the shield is loosened.

The conclusion is clear: loosening the shield's *thresholds* does not teach the agent anything. It merely moves the boundary the agent rides into, paying the safety cost of weaning without buying any independence. The LLM, furthermore, never justified its place over a deterministic schedule.

9.4. SECOND APPROACH: THE SHIELD AS A TEACHER

9.4.1. Filling the gradient hole

If dependence is caused by the absence of gradient on the shielded states (R2), the remedy is to supply one. The masked PPO surrogate is left untouched, since it is *correct* not to apply an importance-weighted update to

an off-policy action that the shield substituted. A supervised behavioural-cloning (BC) term is *added* on exactly those masked steps, pulling the policy mean towards the action the shield executed:

$$\mathcal{L}_{\mathrm{BC}}(\theta) \;=\; \frac{\sum_t \alpha_t \left(\tanh(\mu_\theta(s_t))_{\mathrm{steer}} - a^{\mathrm{sh}}_{t,\mathrm{steer}}\right)^2}{\sum_t \alpha_t}, \qquad \mathcal{L} \;=\; \mathcal{L}_{\mathrm{PPO}} + \lambda_{\mathrm{BC}}\,\mathcal{L}_{\mathrm{BC}}, \tag{9.1}$$

where μ_θ is the actor mean, $\tanh(\mu_\theta(s_t))_{\mathrm{steer}}$ the policy's deterministic steering output, $a^{\mathrm{sh}}_{t,\mathrm{steer}}$ the steering component of the shield's executed action, and $\alpha_t \in [0,1]$ the per-step shield mixing coefficient of Equation (6.2), so that each step is cloned in proportion to how strongly the shield intervened ($\alpha_t = 0$ on unshielded steps contributes nothing). The total loss adds this term to the masked PPO surrogate $\mathcal{L}_{\mathrm{PPO}}$ of Section 6.4.2 with weight $\lambda_{\mathrm{BC}} \geq 0$. The resulting learning rule is clean: PPO on the on-policy steps, behavioural cloning of the shield on the shield-corrected steps, and the critic on all steps. The coefficient λ_{BC} is annealed to zero so the shield teaches strongly early and the agent is released to its own policy late, a curriculum analogous to learning from intervention [42]. Critically, the cloning target in Equation (9.1) is restricted to the *steering* dimension; the reason is analysed in Section 9.5.

9.4.2. Result: the agent learns to drive unaided

The scheme is applied as a warm start from the competent shielded policy. The effect on the shield-off probe is dramatic (Figure 9.3): off-road episodes, the dominant unaided failure, fall from $\approx 90\%$ to $\approx 6\%$, and the shield-off success rate rises from 0% to 50–75% within a few hundred episodes, while the baseline curve stays flat at zero. The agent has learned the shield's lane-keeping skill.

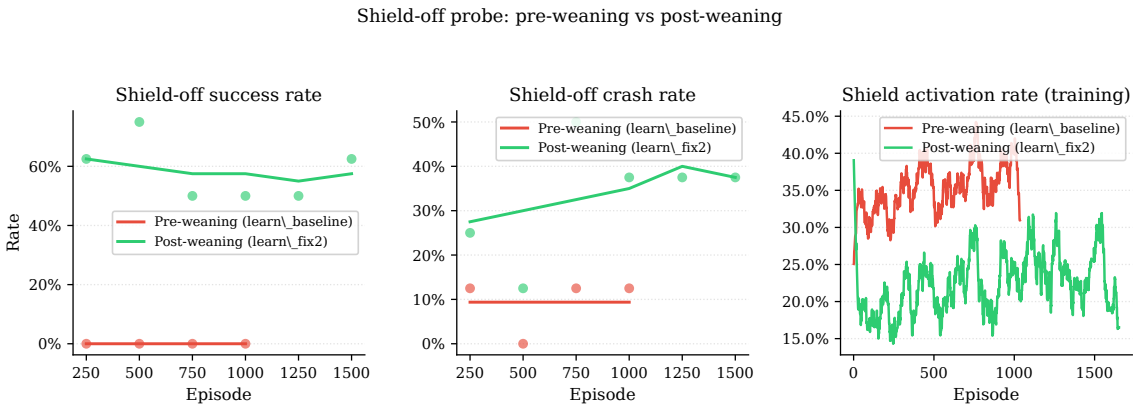

Figure 9.3.: Shield-off probe (the true learning curve). Without the teacher scheme (grey) the agent never learns to drive unaided: it leaves the road within $\approx 40\,\mathrm{m}$ on essentially every episode. With the steering behavioural-cloning teacher (green), shield-off success climbs to 50–75% and off-road collapses to a few per cent.

A final shield ablation confirms the result on identical, seed-aligned scenarios with 60 NPCs (Table 9.1 and Figure 9.4). The headline number is that the *shield-off* success rate (66%) *matches* the *shield-on* rate (66%): the agent no longer needs the shield to function. What the shield still provides is safety: it reduces the crash rate from 16% to 12% and the off-road rate from 18% to 8%, with the combined unsafe rate falling from 34% to 20%, at the cost of 12% stuck episodes induced by the shield's braking. The shield has gone from a crutch the agent could not function without to a safety net it does not need in order to drive.

Table 9.1.: Shield ablation after weaning (`learn_fix2`), 50 episodes, 60 NPCs, seed-aligned scenarios.

Configuration	**Success**	**Crash**	**Off-road**	**Stuck**	**Shields/ep**
Shield OFF (`none`)	66%	16%	18%	0%	0.0
Shield ON (`adaptive`)	66%	12%	8%	12%	65.6

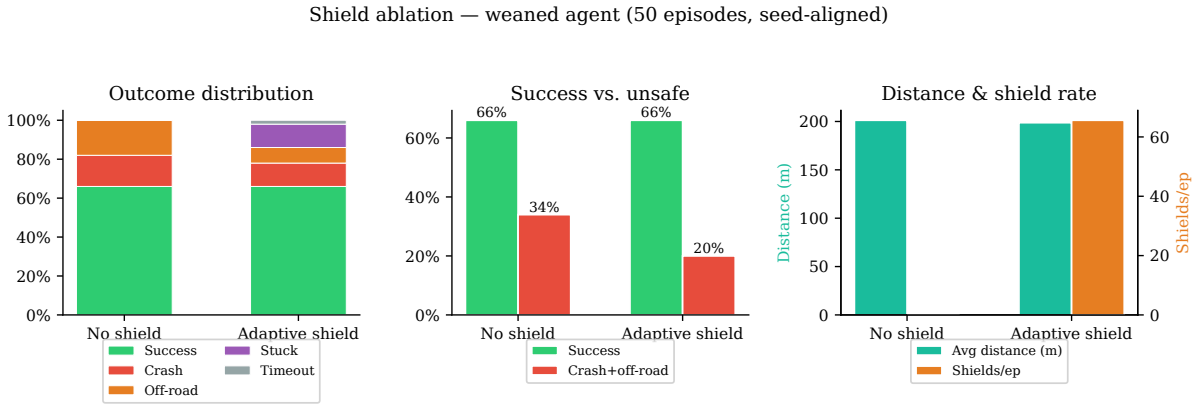

Figure 9.4.: Ablation of the weaned agent (`learn_fix2`, 60 NPCs). Shield-off and shield-on success are matched at 66%; the shield's remaining value is reducing the combined crash + off-road rate from 34% to 20%.

Figure 9.5 contextualises this result against the pre-weaning baseline (`FullSendC1_p4`) across all three shield configurations under identical conditions (20 NPCs, seed-aligned scenarios).

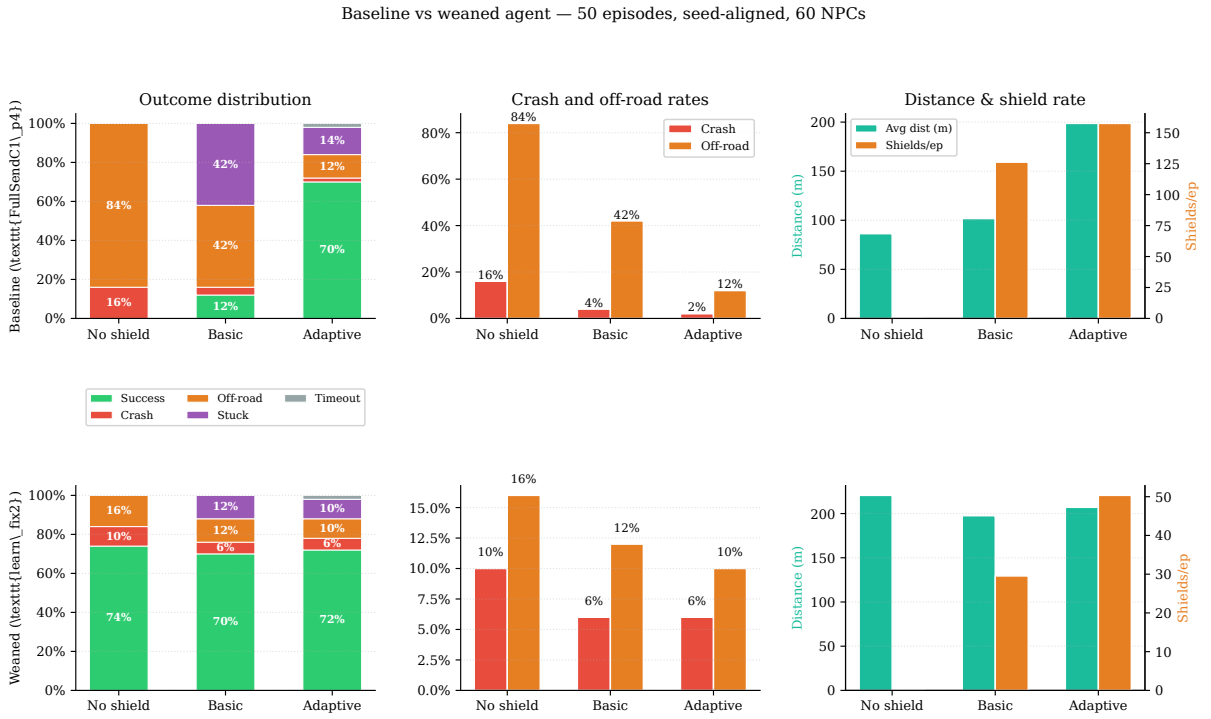

Figure 9.5.: Baseline (`FullSendC1_p4`, top row) versus weaned (`learn_fix2`, bottom row), 50 seed-aligned episodes, 20 NPCs. Left: outcome distribution per shield type. Centre: crash and off-road rates. Right: average distance and shield interventions per episode. The baseline agent requires the adaptive shield to drive at all (0% success without it); the weaned agent reaches 74% success with no shield and matches that rate under all three configurations.

9.5. FAILURE ANALYSIS: WHY ONLY THE STEERING

Restricting the cloning target in Equation (9.1) to the steering dimension is not a detail but the difference between success and collapse. When the throttle/brake dimension is included, the policy collapses to a stationary "parking" equilibrium (Figure 9.6): the mean speed falls to near zero and the success rate to zero.

The mechanism is that *the shield's throttle output is always an emergency brake*, as seen in the lateral-recovery and emergency-brake commands of Section 6.5.2. Unlike its steering, which demonstrates good lane keeping, the shield never demonstrates good *longitudinal* driving. It only ever demonstrates slamming the brakes. Cloning that signal teaches the agent to brake, and because behavioural cloning is a *direct* supervised gradient on the policy mean it overpowers the reward-mediated incentives that would otherwise keep the agent moving.

A narrower variant that cloned the brake only on *dynamic* (vehicle) interventions was also tried and it too collapsed: as soon as the curriculum raised the traffic density, the now-frequent vehicle interventions taught systematic over-braking (Figure 9.6). The lesson generalises: **the shield's steering is a good teacher, but its throttle is never one**, because it encodes a last-resort reflex rather than a driving policy.

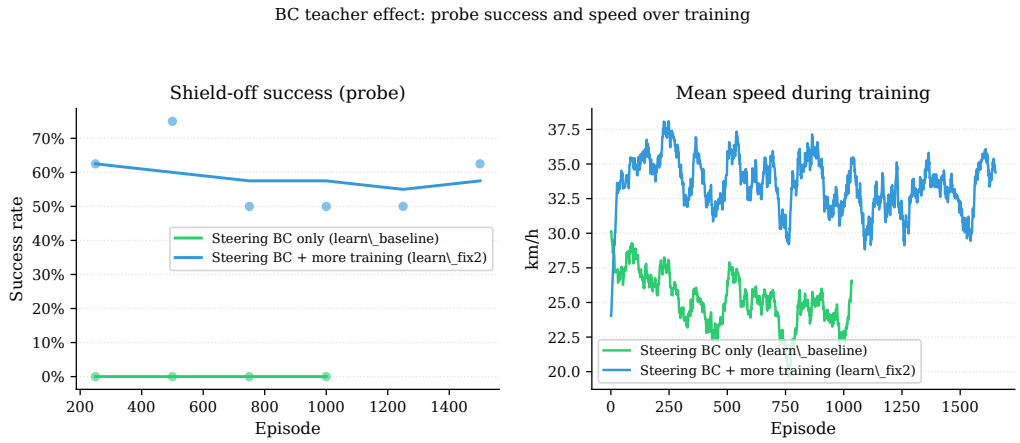

Figure 9.6.: Imitating the shield's throttle collapses the policy to parking. Cloning the steering only (green) is stable; cloning the throttle on every shielded step (orange) or only on dynamic interventions (red) drives the mean speed to zero, the latter precisely when the curriculum increases the traffic density.

9.6. DISCUSSION

The two approaches share a target but differ in what they treat as the cause of dependence. The LLM meta-controller treats it as a *scheduling* problem, asking how aggressively the shield should filter. The experiment of Section 9.3 shows that this is the wrong variable: loosening the filter neither teaches the agent nor reduces its reliance, it only exposes it to more risk. The behavioural-cloning teacher treats dependence as a *learning* problem, because the agent never received gradient on the states the shield handled, and resolves it by supplying that gradient. Equation (9.1) is, in effect, the missing half of the masked objective.

First, the safety governor of Section 9.3 is a sound and reusable component independently of the LLM: it never permitted an unsafe run, and the "proposer + deterministic veto" pattern is a safe way to put a learned controller in a safety-critical loop even when, as here, the proposer turns out to be weak. Second, the residual gap after weaning is collision avoidance (the unsafe rate remains at 34% without the shield against 20% with it). Because the shield is a poor longitudinal teacher, closing that gap is better posed as a reward-shaping problem than as a further imitation target.

10. LEARNING COLLISION AVOIDANCE FROM LLM-DERIVED REWARDS

Chapter 9 closed on an explicit open problem. Behavioural cloning of the shield's steering removed shield *dependence* (the weaned agent drives as well unaided as it does shielded), but it left a residual *collision* gap: without the shield the agent still crashes on roughly one episode in six, because the shield's throttle is an emergency brake and is therefore a poor longitudinal teacher (Section 9.5). That chapter argued the gap is "better posed as a reward-shaping problem … that lets the agent learn its *own* gentle avoidance". This chapter takes up that problem, and in doing so revisits, one final time, the use of a Large Language Model (LLM) in the training loop, after the negative result of Section 9.3.

The chapter is organised around a single thesis: the earlier LLM failure and the failure of a hand-designed time-to-collision (TTC) penalty share one root cause, that cause dictates the design that ought to work, and, once the design is built and every confound is removed, it produces a clean negative result that is itself informative. Section 10.1 establishes the diagnosis. Section 10.2 derives the method (a dense reward *learned* from an LLM's preferences, delivered under a manufactured exposure curriculum). Sections 10.3–10.5 detail its three components. Section 10.6 reports the perception wall that the first experiments ran into and the systems fix that broke through it. Section 10.7 reports the de-confounded experiment, and Section 10.8 places the result in the *experience* $>$ *gradient* $>$ *incentive* hierarchy that organises the whole of Chapters 9–10.

10.1. DIAGNOSIS: ONE ROOT CAUSE BEHIND TWO NEGATIVE RESULTS

Two attempts to use a language model or a designed reward to close the avoidance gap failed, and, on inspection, they failed identically.

The permissiveness dial (Section 9.3).

The LLM controlled a single scalar every hundred episodes. Its output was *not causally coupled* to the metric it was meant to move: the correlation between permissiveness and the shield-activation rate was ≈ 0. A fixed schedule would have produced the same outcome at no cost.

The TTC penalty.

A hand-designed time-to-collision / following-distance penalty was added to the reward shaper and its weight swept across several runs. The penalty *fired in only* $\approx 3\%$ *of episodes*, its magnitude when it fired was flat across weights, and the shield-off crash rate did not fall. It merely redistributed failures from off-road to collision.

The common cause.

Both signals were *starved*. On the Town04 highway loop with the three-channel LIDAR of Chapter 6, a vehicle entered the ego's front cone closely enough to register in only about 3% of episodes, so any collision-specific signal, whether an LLM dial or a TTC penalty, almost never had anything to act on. Three constraints follow, and the method of Section 10.2 is the minimal design that satisfies all three:

1. **Bandwidth.** The learning signal must be *dense* (per step), like the behavioural-cloning term of Equation (9.1), not a per-window scalar.

2. **Exposure.** A collision signal starves at 3% coverage; the relevant experience must be *manufactured* (or otherwise made frequent).

3. **Stationarity.** A live additive reward rescales the running return normaliser the critic is calibrated against (Section 6.4.2); to avoid the park-attractor collapse of Section 9.5, the signal must be *policy-invariant*.

10.2. METHOD: A DENSE LLM-LEARNED REWARD OVER GENERATED EXPOSURE

The proposed method, which we call *Shielded Motif under generated exposure*, has three components that map one-to-one onto the three constraints. A **preference-trained reward model** (Section 10.3) supplies a dense, per-step safety signal learned from a language model's judgements, *offline*, so the model never runs inside the control loop. An **exposure director** (Section 10.4) manufactures vehicle encounters, supplying the missing experience. A **potential-based shaping** integration (Section 10.5) delivers the learned signal without altering the optimal policy. Throughout, the safety shield and the deterministic safety governor of Chapter 9 are retained unchanged: the shield still prevents unsafe *exploration* on the harder, manufactured scenarios, and the "proposer + deterministic veto" pattern, already shown to be sound independently of the LLM (Section 9.6), now guards the exposure curriculum.

The design follows the Motif paradigm of intrinsic motivation from artificial-intelligence feedback [43], in which an LLM expresses preferences over pairs of observation captions that are then distilled into a reward model. It inherits the preference-learning formulation of [44] and the policy-invariance guarantee of [45]. It is related to, but distinct from, language-conditioned reward generation for driving [46] and LLM-based safety-critical scenario generation [47], from which the exposure director borrows its intent-conditioned design. Unlike the latter line of work, here the language model shapes the *reward* rather than authoring the scenario script. The defining choice, and the one that directly answers the failure of Section 9.3, is that the language model acts *entirely offline*: it never competes with CARLA for GPU memory and adds no latency to the $20\,\mathrm{Hz}$ control loop.

10.3. COMPONENT I: THE PREFERENCE-TRAINED REWARD MODEL

10.3.1. Offline preference annotation

A dataset of 739-dimensional observations was collected by rolling out the weaned policy of Chapter 9 under dense traffic and saving the observations the agent actually sees (frozen observation normalisation, as at evaluation). Pairs of scenes were then sampled, with in-path encounters over-sampled so that the annotator spends its budget where collisions are decided, and each scene was rendered into a short natural-language caption describing the front clearance, the nearest in-path vehicle, the lateral offset and the speed.

The first annotation attempt reused the local Gemma model from the abandoned shield tuner. It failed in an instructive way: it abstained or contradicted itself on roughly 85% of pairs, exhibited a strong position bias (it preferred whichever scene was presented first), and, when it did commit, it tracked "which car is driving more nicely" rather than "which car is closer to a collision". The annotation was therefore moved to a stronger model accessed through a batch of parallel agents, with two changes that proved decisive: a *strict, collision-only* prompt that explicitly decouples "stopped" from "in danger", and *symmetric* querying, where each pair is annotated in both orderings and the two judgements are combined, which cancels position bias by construction. The resulting labels passed a battery of semantic integrity checks (a closer in-path vehicle is judged less safe; a larger lateral offset is judged less safe; the labels are internally consistent under reordering) before any model was trained on them.

10.3.2. The reward model

The reward model r_ϕ is a small multilayer perceptron ($250 \to 64 \to 64 \to 1$) that reads a fixed slice of the observation (the dynamic-LIDAR channel together with the lane and vehicle scalars), defined once as a

single source of truth shared with the shaping stage. It is trained from preferences with the Bradley–Terry objective [44]: for an annotated pair of scenes (s_a, s_b) in which the language model judges s_a the safer situation,

$$\mathcal{L}(\phi) \;=\; -\,\mathbb{E}_{(s_a,s_b)}\; c_{ab}\;\log \sigma\big(r_\phi(s_a) - r_\phi(s_b)\big), \tag{10.1}$$

where σ is the logistic (sigmoid) function, $r_\phi(s_a) - r_\phi(s_b)$ is the reward gap between the two scenes and $c_{ab} \in [0, 1]$ is the language model's stated confidence, used to down-weight uncertain pairs. The objective is the negative log-likelihood of the observed preferences under the model $P(s_a \succ s_b) = \sigma(r_\phi(s_a) - r_\phi(s_b))$ (Figure 10.1). The sign convention is that a *higher* r_ϕ denotes a *safer* situation. Unlike a crash-labelled safety critic, which sees only the sparse binary outcome of an episode, r_ϕ provides a graded signal on the many near-miss steps that never end in a crash.

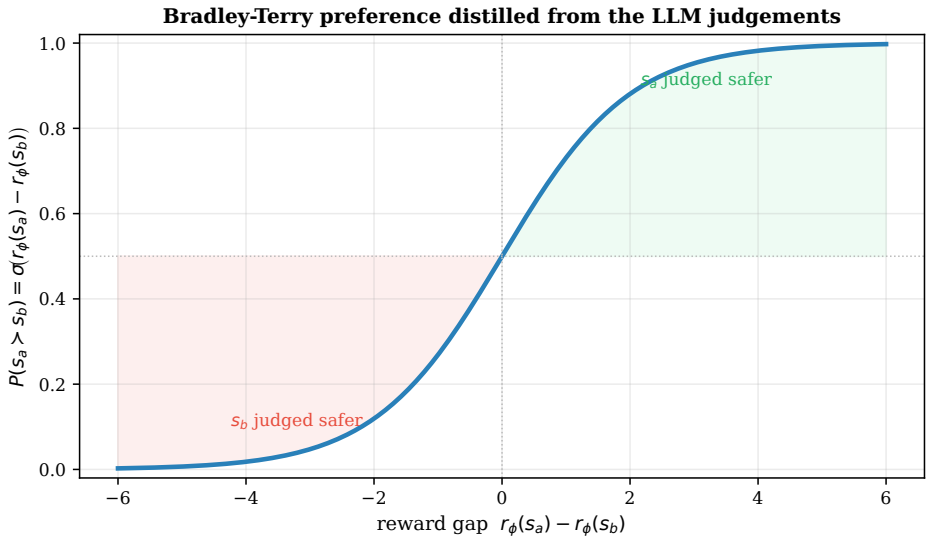

Figure 10.1.: The Bradley–Terry model maps a reward gap to a preference probability: the probability that scene s_a is the safer of a pair is $\sigma(r_\phi(s_a) - r_\phi(s_b))$. Training maximises this probability on the pairs the language model labelled, weighting each pair by its confidence c_{ab}, which pushes r_ϕ higher for the safer scene of every pair without ever seeing a numeric label.

Trained on the curated labels, r_ϕ recovers a clean, monotone notion of longitudinal safety: averaged over the held-out observations, its output rises from -15.1 when the nearest in-path vehicle is 5–$10\,\mathrm{m}$ ahead to $+13.7$ when the lane is clear beyond $30\,\mathrm{m}$ (Figure 10.2). The model never saw a distance label; it inferred the ordering purely from the language model's pairwise judgements.

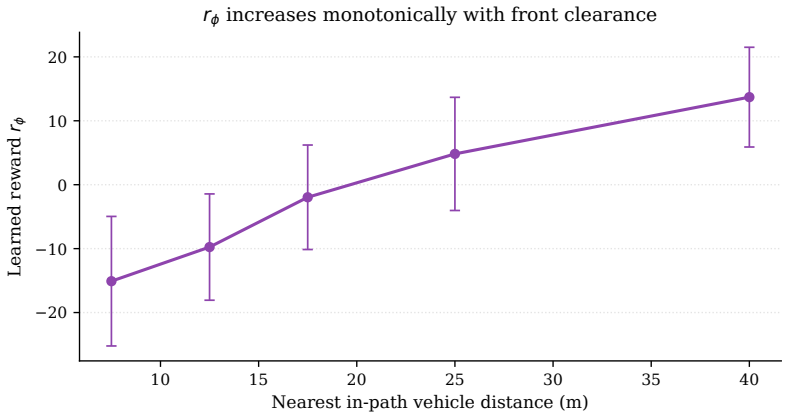

Figure 10.2.: The learned reward r_ϕ as a function of the nearest in-path vehicle distance, averaged over held-out observations (error bars: one standard deviation within each bin). r_ϕ increases monotonically with front clearance, confirming that the Bradley–Terry distillation of the LLM's preferences recovered a coherent longitudinal-safety ordering.

10.4. COMPONENT II: THE EXPOSURE DIRECTOR

Globally spawned NPCs satisfy neither the exposure constraint (encounters are rare) nor a controllable difficulty schedule. The exposure director injects *route traffic*: a small number of slow leads placed directly on the ego's lane, ahead of it, by walking the waypoint chain forward along the ego's direction of travel. Two implementation subtleties from CARLA's synchronous model had to be respected: the ego transform is only valid *after* a world tick, so the spawn uses the transform captured at spawn time rather than a stale pre-tick pose; and `waypoint.next()` follows the OpenDRIVE topological direction rather than the vehicle heading, so the chain direction is chosen by the sign of the dot product between the ego forward vector and the lane forward vector. Lead speeds are specified in *absolute* terms (20–$28\,\mathrm{km/h}$, calibrated against the $\sim 34\,\mathrm{km/h}$ the ego sustains in traffic) so that a lead is a genuinely slower obstacle to be negotiated, not a target that drives away. The whole mechanism is gated by a dedicated governor with crash ceilings that *cut* traffic on a breach, reusing the asynchronous-runner and deterministic-governor machinery salvaged from the abandoned meta-controller stack. Progress is tracked by an *encounter rate*, which is the fraction of episodes in which a vehicle enters the front cone closely enough to register.

10.5. COMPONENT III: POLICY-INVARIANT SHAPING

The learned reward is delivered as potential-based shaping [45], using r_ϕ as the potential:

$$F_t \;=\; \gamma\, r_\phi(s_{t+1}) - r_\phi(s_t), \qquad r_t^{\text{shaped}} \;=\; r_t \;+\; \beta\, F_t, \tag{10.2}$$

where the potential is the learned reward itself, $\Phi(s) \;\equiv\; r_\phi(s)$; γ is the MDP discount factor; F_t is the potential-difference shaping term; $\beta \geq 0$ is a fixed scalar weight; r_t is the environment reward; and r_t^{shaped} is the reward actually used for the PPO update. The terminal potential $\Phi(\text{terminal}) = 0$ is enforced on episode termination. Because F_t is a difference of potentials, the set of optimal policies is provably unchanged [45]: the term can only *re-time* credit toward the states that precede danger, it cannot create a new optimum. This is the formal guarantee that the park-attractor of Section 9.5, in which a direct supervised gradient overpowered the reward and drove the policy to a stationary equilibrium, cannot recur here, and it is why the coefficient β is simply frozen per run: policy invariance holds for *any* constant β, so no annealing schedule is needed.

A training-stability prerequisite: the `log_std` anchor.

Every arm of this chapter warm-starts from the weaned checkpoint of Chapter 9. Warm-starting re-exposed a latent failure of the policy parameterisation: the entropy bonus would, over a few dozen updates, push the action-noise parameter `log_std` up against its ceiling and pin it there, freezing exploration noise high and re-triggering the turtle/stuck collapse documented in Chapter 6. The fix is a light quadratic *anchor* on the raw `log_std` parameter toward the pre-image of its initial value, added to the PPO loss. With the anchor active the saturated fraction stayed at 0% and the effective standard deviation settled at $\sigma \approx 0.22$ and annealed down as the policy converged, rather than locking high. The anchor is a precondition for any of the reward experiments below to be interpretable: without it, every warm-started run collapses for reasons that have nothing to do with the reward under test.

10.6. THE PERCEPTION WALL

The first end-to-end runs of the full method were catastrophic: shield-off crash rates of 74% and then 51%, far worse than the 16% control. The cause was neither the reward nor the exposure director but the *perception range*. The three-channel semantic LIDAR resolves a vehicle only at about $12\,\text{m}$; yet on clear road the weaned policy cruises near $65\,\text{km/h}\,(\approx\, 18\,\text{m/s})$. A lead detected at $12\,\text{m}$ while closing at highway speed leaves well under one second to react, making a rear-end *geometrically unavoidable*, not a failure of policy. The exposure

signal of Section 10.4 was being starved by the same constraint: a vehicle had to be almost on top of the ego before it registered at all.

The fix is a perception upgrade rather than a learning change. Raising the LIDAR's vertical channel count from 3 to 16 lifts the effective vehicle-detection range from $\approx 12\,\mathrm{m}$ to $\approx 71\,\mathrm{m}$. Crucially, this leaves the 739-dimensional observation layout *unchanged* (the channels are reduced over the same 240 angular bins), so it does not break checkpoint compatibility. Re-collecting the annotation set and retraining r_ϕ at 16 channels reproduced the same clean monotonic reward of Figure 10.2.

Table 10.1.: The perception wall and its fix. Increasing LIDAR vertical resolution restores a usable detection range and, with it, encounter coverage during training, while leaving the observation dimensionality unchanged.

LIDAR channels	**Vehicle detection**	**Encounter rate (train)**	**Obs. dim**
3 (Chapters 6–9)	$\approx 12\,\mathrm{m}$	$\approx 3\%$	739
16 (this chapter)	$\approx 71\,\mathrm{m}$	$\approx 34\%$	739

The encounter curves in Figure 10.3 carry a second, more humbling, finding. The control run `base16` uses *global traffic only* (no exposure director), yet its encounter rate climbs to 34.3%, essentially identical to the 33.1% of the full `motif16` run with the director active. In other words, once perception reached $70\,\mathrm{m}$, the sixty global NPCs already supplied ample encounters, and the manufactured route traffic added nothing measurable. The exposure *diagnosis* of Section 10.1 was correct (the 3% ceiling was real), but the effective *cure* turned out to be the perception upgrade, not the traffic director. This is exactly the kind of confound that an in-loop experiment hides and an isolated one exposes.

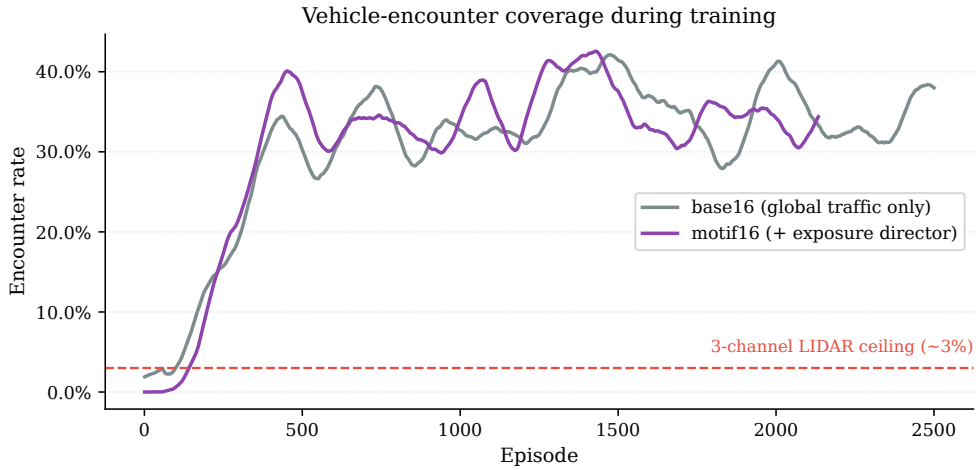

Figure 10.3.: Vehicle-encounter coverage over training. The dashed line marks the $\approx 3\%$ ceiling imposed by the three-channel LIDAR. At 16 channels both runs climb to $\approx 34\%$; the exposure director (`motif16`) is statistically indistinguishable from global traffic alone (`base16`), showing that the perception upgrade, not the manufactured traffic, lifted exposure.

10.7. EXPERIMENTS

With perception fixed and the anchor in place, the question reduces to a single clean comparison: does the LLM-learned preference reward reduce the shield-off collision gap? Two 16-channel runs, each warm-started from the weaned checkpoint and trained to a 60-NPC curriculum, isolate exactly that factor:

- `base16`, the control: 16-channel perception, global traffic, *no* preference reward and *no* exposure director.
- `motif16`, the full method: the same, *plus* the preference reward of Equation (10.2) and the exposure director.

Both were evaluated deterministically over 50 seed-aligned episodes at 60 NPCs, with the shield removed and shield active, exactly as in Chapter 9.

Table 10.2.: Outcome distribution at 60 NPCs over 50 seed-aligned episodes (deterministic policy, 16-channel perception). The preference reward shifts failures from off-road to collision but leaves the combined unsafe rate unchanged within noise.

Shield	Run	Success	Crash	Off-road	Crash+Off	Avg. reward
off	`base16` (control)	64%	16%	20%	36%	95.9
	`motif16` (method)	66%	22%	12%	34%	104.1
on	`base16` (control)	70%	6%	6%	12%	110.6
	`motif16` (method)	70%	10%	8%	18%	115.6

The headline is a **null result**. With the shield off, the combined crash + off-road rate is 34% for the method against 36% for the control, a difference of two points on fifty episodes ($z \approx 0.76$, $p \approx 0.45$), well inside sampling noise. The reward did move the *composition* of failures, and in a direction that is mechanistically informative: crashes rose from 16% to 22% while off-road events fell from 20% to 12%.

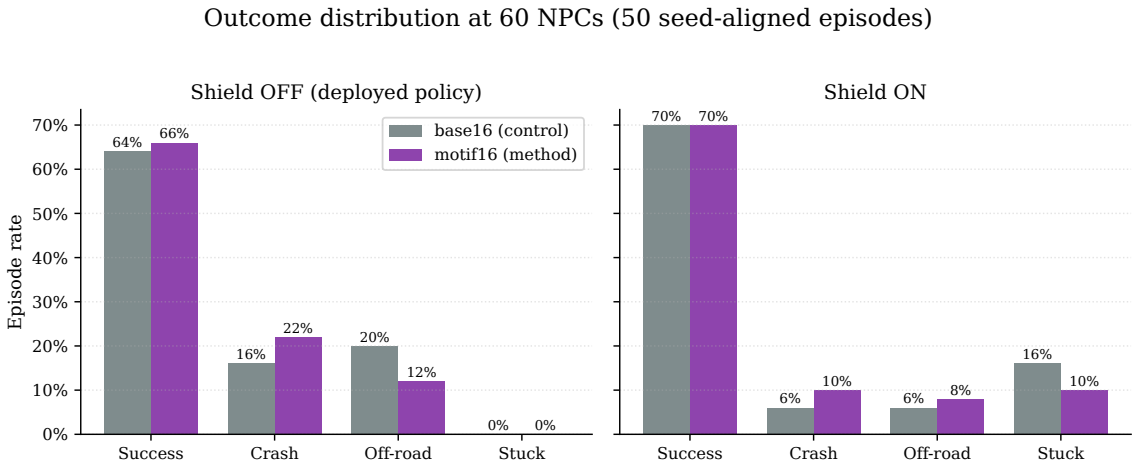

Figure 10.4.: Outcome distribution with the shield off (left) and on (right), control versus method. The preference reward trades off-road events for collisions; the combined unsafe rate is unchanged with the shield off and slightly worse with it on.

Mechanism: lane-centring buys collisions.

A focused diagnostic over sixteen seed-aligned shield-off episodes explains the trade. The method drives much closer to the lane centre: mean absolute lateral offset 0.040 against the control's 0.095, a factor of 2.4. This is

precisely what a reward that favours "safer" lateral positioning should do, and it is why off-road events fall. But it has a cost: of the control's three off-road episodes, two were *collision escapes* in which the agent swerved off the road to avoid a lead. By keeping the car in its lane, the preference reward removes that escape route; when a lead then appears too late, the agent rear-ends it instead of leaving the road. Three of the method's four diagnostic crashes are front rear-ends with the nearest vehicle at 2.3–$2.5\,\mathrm{m}$ and the ego still travelling at 17–$45\,\mathrm{km/h}$. The reward faithfully learned lateral safety, which dominated the captions, but it carries no *longitudinal* authority: it cannot make the agent brake.

The residual is longitudinal control, and it is now detectable.

With perception fixed, the agent demonstrably *sees* the lead at $70\,\mathrm{m}$; what it lacks is the policy to brake hard enough in time. The shield-on rows of Table 10.2 confirm the hazard is both detectable and avoidable (the shield drives the crash rate down to 6–10% by braking on exactly these encounters), so the gap is an execution gap in the learned policy, not a perception or unavoidability problem as it was before the perception fix. The reward does not interact harmfully with the shield: shielded success is identical at 70%, and the method's slightly higher shielded crash mirrors its slightly lower stuck rate, as the in-lane bias also reduces the parked deadlocks discussed in Chapter 6.

10.8. DISCUSSION

This chapter completes an argument that runs across Chapters 9 and 10 and can be stated as a hierarchy: *experience* $>$ *gradient* $>$ *incentive*. The TTC penalty supplied an *incentive* but no experience to apply it to, and failed. Behavioural cloning supplied a *gradient* on the shielded states and fixed steering, but the shield's brake-only throttle made it a poor longitudinal teacher, so a gap remained. This chapter supplied the missing *experience* (first by manufacturing it, then, more decisively, by extending perception so that ordinary traffic became experience) and a dense, policy-invariant, LLM-learned *incentive* on top of it. The incentive then did exactly what its training signal encoded (it improved lateral safety), but it could not, on its own, install the longitudinal braking skill, because braking was under-represented in the captions the language model ranked.

The result is therefore a *clean, bounded, and de-confounded negative result*, and that is its value. Every alternative explanation for the earlier failures was removed in turn: position bias (symmetric annotation with a stronger model), reward non-stationarity (potential-based shaping), exploration collapse (the `log_std` anchor), and, most decisively, exposure starvation (the perception upgrade, which alone lifted encounter coverage roughly tenfold). With all of these controlled, a preference reward that steers toward "safer" scenes

shifts the failure distribution from off-road to collision without a net safety gain. The honest reading is not that LLM-derived rewards cannot help, but that *what the LLM is asked to rank determines what the agent learns*: captions rich in lateral geometry and poor in closing dynamics yield a reward rich in lane-keeping and poor in braking.

Two corollaries are worth recording for the project as a whole. First, the abandoned meta-controller stack was not wasted: its schema-constrained decoding, deterministic governor and asynchronous runner were repurposed as trustworthy *offline* annotation and exposure infrastructure. The negative result of Chapter 9 is what *forced* the offline, policy-invariant design, rather than being a lucky guess. Second, the perception wall is a reminder that in an embodied agent a learning signal is only as good as the perception that feeds it; a substantial share of what looked like a reward-design problem was in fact a sensor-range problem.

The clearest next step follows directly from the mechanism. The reward should be made to encode the quantity it currently misses: captioning the closing speed or time-to-collision explicitly, rather than only the static front clearance, so that the Bradley–Terry distillation can learn a longitudinal ordering as cleanly as it learned the lateral one in Figure 10.2. Alternatively, pairing the learned reward with an explicit longitudinal-control prior would supply the braking authority the incentive alone cannot. A multi-seed replication would also tighten the confidence intervals around the two-point difference reported here. These directions are taken up in Chapter 11.

11. CONCLUSIONS AND FUTURE WORK

This chapter closes the document. Section 11.1 recapitulates the contributions, Section 11.2 traces them against the specific objectives laid out in Chapter 1, Section 11.3 reflects on the lessons learned during the project, and Section 11.4 outlines research directions that go beyond the scope of a Bachelor's thesis.

11.1. SUMMARY OF CONTRIBUTIONS

The work presented in this document produced the following concrete contributions:

1. **CALA-SafeRL framework.** A modular Safe RL framework for autonomous driving on CARLA 0.9.16, organised as a chain of interchangeable `gymnasium.Wrapper` layers. The wrapper chain isolates the PPO agent, the safety shield, the reward shaper and the simulator from each other, which makes each component independently testable and replaceable.

2. **Semantic 739-dimensional observation space.** A sensor pre-processing stage packages three LIDAR channels (combined, dynamic-only, static-only) of 240 rays each together with 19 geometric and route features (including lane-marking type flags) derived from CARLA's Waypoint API. The decomposition of the LIDAR into semantic channels reduces the zig-zag pathology introduced by static guardrails.

3. **PPO agent with modern stabilisers.** An implementation of PPO with Generalised Advantage Estimation, clipped surrogate updates, value clipping, approximate-KL early stopping, cosine learning-rate annealing, return normalisation and a Welford running-mean estimator applied only to the vector block of the observation.

4. **Basic safety shield.** A lightweight, interpretable shield that combines LIDAR thresholding with lane-keeping corrections derived from the Waypoint API. The shield follows the minimal-interference principle: its override action is zero whenever the situation is safe and grows smoothly with the severity of the violation.

5. **Adaptive-horizon safety shield.** A richer shield that classifies the current situation into three risk levels, predicts the vehicle's future trajectory with a kinematic bicycle model calibrated for the Tesla Model 3, and selects the least invasive corrective action from a priority-ordered candidate set. The shield supports a pedestrian emergency branch, a lane-change relaxation rule and a threshold-floor safeguard that prevents the *critical* level from rejecting all candidate actions.

6. **Dense reward shaper.** A reward wrapper with twelve additive components plus an alive bonus, all gated by speed when appropriate to avoid the degenerate "stop-and-collect" optimum. The shaper also implements a lane-change transition guard and a shield grace period that stabilise the interaction between the agent, the shield and the reward.

7. **Performance-driven curriculum with rollback.** A curriculum manager that divides the interval $[0, N_{\max}]$ of NPC vehicles into five uniform stages and advances or rolls back according to 100-episode rolling-window statistics. Stage transitions update the NPC count in place, without reconnecting to CARLA.

8. **Live metrics pipeline.** A SQLite-based logger in WAL mode that exposes two tables (*per-episode* and *per-update*) and a Streamlit dashboard that reads them while training is still in progress. Unknown metric keys are rendered as first-class citizens, which makes the dashboard forward-compatible with future experiment extensions.

9. **Reproducibility and tooling.** Unit tests for the reward shaper that do not require a running simulator, a `ruff`-enforced code style, seed-separated training and evaluation protocols, a checkpoint format that includes the observation normaliser state, and a reproducibility guide in Appendix C.

10. **Shield-dependence study (Chapter 9).** An instrument to measure dependence, the shield-off evaluation probe and the dynamic/static reliance split, together with two strategies to reduce it. A Large-Language-Model meta-controller that loosens the shield is shown, with data, *not* to reduce dependence. A behavioural-cloning scheme that clones the shield's steering corrections does reduce it, producing an agent whose shield-off success rate (66%) matches its shield-on rate (66%). The accompanying deterministic safety governor is a reusable "proposer + veto" pattern for placing a learned controller inside a safety-critical loop.

11. **LLM-derived reward study (Chapter 10).** An offline pipeline that distils a Large Language Model's pairwise safety preferences into a dense reward model with the Bradley–Terry objective, delivered to PPO as policy-invariant potential-based shaping, together with an exposure director and a `log_std` anchor that stabilises warm-started training. Two by-products are of independent value: a *perception-range finding* showing that the three-channel LIDAR caps vehicle detection at $\approx\ 12\,\mathrm{m}$ and thereby starves any collision signal, fixed by raising the LIDAR to 16 channels (detection $\approx\ 71\,\mathrm{m}$, encounter coverage $3\% \rightarrow 34\%$) at unchanged observation dimensionality, and a rigorously de-confounded *negative result*: the learned reward improves lane-keeping (mean lateral offset $0.095 \rightarrow 0.040$) but trades off-road events for collisions without a net safety gain, locating the remaining gap in longitudinal control.

11.2. FULFILMENT OF OBJECTIVES

Table 11.1 traces the specific objectives of Section 1.3 against the chapters in which each of them is addressed and against the contributions enumerated above.

Table 11.1.: Traceability matrix between specific objectives and the rest of the document.

Objective	Chapter(s)	Contribution	Status
SO1 – Gymnasium CARLA wrapper	6, 7	1, 2	Achieved.
SO2 – PPO agent with stabilisers	6, 7	3	Achieved.
SO3 – Basic shield	6, 7, 8	4	Achieved.
SO4 – Adaptive-horizon shield	6, 7, 8	5	Achieved.
SO5 – Dense reward shaper	6, 7	6	Achieved.
SO6 – Curriculum with rollback	6, 7, 8	7	Achieved.
SO7 – Live metrics pipeline	6, 7	8	Achieved.
SO8 – Measure & reduce shield dependence	9	10	Achieved.
SO9 – LLM-derived reward for avoidance	10	11	Achieved (negative result).

The quantitative evaluation that anchors *"Achieved"* for SO3, SO4 and SO6 is provided in Chapter 8. The key numbers are: the adaptive shield achieves 70% success ($\gamma = 60\%$ threshold), 2% crash ($\alpha = 4\times$ threshold), and 0% rolling off-road ($\beta = 15\%$ threshold) as reported in Table 8.6.

11.3. LESSONS LEARNED

The development of the framework surfaced a number of design lessons that, in retrospect, would have saved considerable time if they had been understood earlier in the process.

Place the shield above the reward shaper. An earlier prototype inverted this ordering and the shaper could not read the executed action. The reward-shaping signals that depend on the shield (intervention penalty, smoothness suppression during override, grace period) were all computed incorrectly until the chain was reordered.

Never normalise the LIDAR. Applying the running-mean normaliser to the 720 LIDAR dimensions was the single biggest source of training instability. When the curriculum advances from stage 0 (no NPCs, the dynamic channel is constant at 1.0) to stage 1, the running standard deviation of the dynamic channel

is close to zero and the normalised inputs become extreme outliers. Restricting the normaliser to the 15-dimensional vector block eliminated the gradient spikes without any further change to the training loop.

Gate the alive bonus by speed. An unconditional alive bonus, even a small one, opens a local optimum in which the agent stops, does nothing, and earns 0.15/step indefinitely. Gating the alive bonus by a speed factor, so that it is earned only while the vehicle is actually moving, closes this trap without hurting exploration.

Give the agent a grace period after a shield override. A brake-based shield intervention drops the vehicle's speed, which would immediately trigger the idle penalty and push the agent into a stationary equilibrium. The shield-grace mechanism of Section 6.6 breaks this feedback loop. Critically, the grace counter is re-armed only when the shield fires *and* the counter is already at zero. Without this condition, an always-active shield would perpetually reset the grace period and silence the idle penalty indefinitely.

Prefer in-place updates to reconnection. Every early attempt at changing the curriculum stage by rebuilding the environment caused CARLA to deadlock. Updating the attribute `num_npc_vehicles` on the base environment and letting the change take effect at the next `reset` is both simpler and more robust.

Attribute credit to the executed action, not to the proposed one. Storing the action that the shield let through, and recomputing its log probability under the current policy, is the single correction that made shielded PPO actually learn. Without it, the gradient estimator pushes the policy towards an action that never reached the environment, and training stagnates.

The shield teaches steering, not throttle. A safety shield is an excellent runtime *filter* but an uneven *teacher*: its steering corrections demonstrate good lane keeping and can be cloned, but its throttle output is always an emergency brake, and cloning it collapses the policy to a stationary equilibrium (Section 9.5). More broadly, shield dependence proved to be a *learning* problem (the agent never received gradient on the shielded states) rather than a *scheduling* problem. Loosening the shield's thresholds, whether by hand or by an LLM, addresses the wrong variable.

Perception range gates the learning signal. Much of what looked like a reward-design problem in Chapter 10 was in fact a sensor-range problem: with the three-channel LIDAR detecting vehicles only at $\approx 12\,\mathrm{m}$, collision-relevant experience occurred in just 3% of episodes, so no amount of reward design could help. Raising the LIDAR to 16 channels lifted both detection range and encounter coverage roughly tenfold and was the single most decisive change. In an embodied agent, a learning signal is only ever as good as the

perception that feeds it.

Experience > gradient > incentive. Across Chapters 9–10 the interventions that worked supplied *experience* (exposure, perception) or a *gradient* (behavioural cloning). Pure *incentives* (a hand-tuned penalty, a loosened threshold, even an LLM-learned reward) only ever reshaped behaviour the agent had already learned. An incentive can refine a skill the agent can already execute, but it cannot install a skill (here, longitudinal braking) for which the agent has neither the experience nor the gradient.

11.4. FUTURE WORK

Several extensions of this framework are natural next steps. They are listed below in order of increasing research effort.

- **Dynamic bicycle model.** The kinematic model used by the adaptive shield ignores tyre slip and lateral dynamics. At higher speeds or in tight turns this approximation becomes inaccurate and the predicted trajectory can diverge from what the simulator actually produces. Replacing it with a dynamic bicycle model or with a simple lateral-dynamics model [10] would tighten the safety guarantees without changing the rest of the architecture.

- **Reachability-based shield.** A principled alternative to the candidate-search strategy is Hamilton–Jacobi reachability analysis [48, 49], which precomputes the set of unsafe states and projects the agent's action onto the largest safe control invariant set. This yields formal safety guarantees but imposes a non-trivial offline computation cost.

- **Camera-based perception.** The observation space currently does not include RGB imagery or semantic-segmentation maps. Adding a convolutional perception head trained jointly with the policy, or reusing the frozen backbone of an imitation-learning pipeline [24], would enable the agent to perceive traffic lights, road signs and crosswalks that are not visible from the semantic LIDAR alone.

- **Urban benchmarks.** The evaluation has been concentrated on Town04, a highway map. Urban maps such as Town01–Town03 and Town10HD feature intersections, traffic lights and dense pedestrian traffic that would stress-test the pedestrian-emergency branch of the adaptive shield. Adapting the curriculum to this richer scenario is itself a research question.

- **Multi-agent safe RL.** The current setup has a single RL-controlled ego vehicle surrounded by rule-based NPCs. Replacing the NPCs with independent RL agents and composing their individual shields into a

global safety filter is an open research direction at the intersection of Safe RL and multi-agent RL.

- **Constrained policy optimisation.** PPO is used here as a black-box optimiser of the shaped reward. A closer integration with Safe RL would replace reward shaping with explicit safety constraints and solve the resulting Constrained MDP with CPO or a Lagrangian variant [19, 20]. This would remove the hand-tuning of reward weights at the cost of a more complex optimiser.

- **Longitudinal control for collision avoidance.** Chapter 10 attacked the residual collision gap with a dense LLM-derived preference reward and reached a de-confounded negative result: the reward improved lane-keeping but traded off-road events for collisions, because the captions it was trained on described lateral geometry richly and closing dynamics poorly, so the reward carried no braking authority. The clearest continuation is to make the reward encode the quantity it currently misses by captioning the closing speed or time-to-collision explicitly, so that the Bradley–Terry distillation can learn a longitudinal ordering, or to pair the learned reward with an explicit longitudinal-control prior. A multi-seed replication would also tighten the confidence intervals around the two-point effect reported there.

- **Benchmarking against other shields.** The MetaDrive and HighwayEnv ecosystems offer a variety of shielding baselines. Porting those baselines to CARLA and comparing them on a shared benchmark would provide an external reference for the adaptive shield proposed here.

The framework has been designed with these extensions in mind. Each of them should be achievable by introducing a new wrapper in the chain of Figure 6.1 or by replacing the existing `Shield` or `RewardShaper` classes, without modifying the remaining components.

12. ETHICAL ASSESSMENT OF THE PROJECT

The development of autonomous driving systems cannot be separated from the ethical weight of the domain in which they operate. A reinforcement learning agent that controls a vehicle is, in a very literal sense, making decisions that affect human safety, public infrastructure, and the welfare of other road users. This section reflects on the ethical issues raised by the design of *CALA_SafeRL*, contrasts the option of deploying a standard, unconstrained reinforcement learning agent against the Shielded-RL approach actually adopted, and justifies the latter as the decision that best satisfies the engineer's professional and ethical responsibilities. The discussion closes by situating the project within the broader frameworks of digital human rights and the United Nations Sustainable Development Goals (SDGs).

12.1. IDENTIFICATION OF ETHICAL ISSUES

Three ethical concerns are inherent to any project that trains a driving policy through trial and error:

- **Risk to human life and physical integrity.** Reinforcement learning is, by construction, an exploratory process: the agent improves by experiencing the consequences of its actions, including the negative ones. In a real or high-fidelity simulated traffic environment, "negative consequences" translate into collisions, off-road excursions, or dangerous manoeuvres near pedestrians and other vehicles. Even if the immediate setting is a simulator, the policies trained in it are intended to generalise toward real-world deployment, so any tolerance for unsafe exploratory behaviour during training is, in principle, a tolerance for unsafe behaviour that could eventually reach public roads.

- **Algorithmic accountability.** A policy represented as a neural network is difficult to audit. When such a system causes harm, it is not always possible to trace the decision back to an interpretable cause, which complicates the attribution of responsibility among the developer, the operator, and the manufacturer. An engineering team that deploys a purely learned controller without any verifiable safety layer is, in effect, delegating safety-critical decisions to a component whose failure modes cannot be fully characterised in advance.

- **Black-box decision making.** Beyond accountability after the fact, there is a forward-looking concern: end users, regulators, and the general public are increasingly reluctant to trust systems whose behaviour cannot be bounded or explained. A driving agent that "mostly works" but offers no guarantee about its worst-case behaviour is difficult to certify and difficult to trust, regardless of how well it performs on

average.

These three issues are not abstract concerns external to the project. They are direct consequences of the choice of algorithm (PPO, a model-free reinforcement learning method) and of the choice of action interface (continuous steering and throttle/brake commands sent directly to the simulated vehicle).

12.2. CRITICAL ANALYSIS OF DESIGN OPTIONS

Two broad design philosophies were considered during the early stages of the project, and the trade-off between them is fundamentally an ethical one rather than a purely technical one.

Option A: an unconstrained reinforcement learning agent.

The first option is to train the PPO agent directly against the CARLA simulator, with no intermediate safety layer (the `-shield_type none` configuration evaluated in this work, Section 8). This option has clear technical advantages: the agent's gradient signal is unfiltered, exploration is unconstrained, and in principle the policy can discover driving strategies that a hand-designed corrector might not anticipate. From a purely performance-driven perspective, this option could be argued to maximise *learning speed* and *asymptotic performance*.

However, this option places the entire burden of safety on the learning process itself, during a phase in which the policy is, by definition, not yet competent. Early in training the agent behaves close to randomly. Under Option A, that randomness is applied directly to the vehicle's controls, and the resulting collisions and off-road events are treated merely as numerical penalties in a reward function. There is no mechanism that prevents an unsafe action from being executed. Safety is, at best, an emergent property that the agent might converge towards, not a property that is guaranteed at any point during training or deployment. The evaluation data gathered for this project (Chapter 8) confirms that, even after substantial training, a non-trivial fraction of episodes under the unshielded configuration end in a crash or an off-road event, which would be unacceptable if those episodes corresponded to interactions with real vehicles, pedestrians, or infrastructure.

Option B: Shielded-RL with a deterministic safety layer.

The second option, and the one adopted in this project, interposes a *safety shield* between the agent and the simulator. The shield (in its most developed form, the `CarlaAdaptiveHorizonShield` described in Chapter 6) monitors the agent's proposed action, predicts the vehicle's short-horizon trajectory using a kinematic bicycle

model, and deterministically overrides the action whenever the prediction indicates an unacceptable risk (collision, lane departure, or deadlock). The agent still learns from experience, including from the cases in which its action was corrected, but the action that actually reaches the simulator is bounded by an explicit, inspectable, rule-based safety envelope.

This option does not maximise raw learning speed or asymptotic reward in the same unconstrained sense as Option A: the shield occasionally prevents the agent from taking the action it would have chosen, and the credit-assignment machinery (Chapter 6) must be designed carefully so that the agent is not penalised for the shield's intervention. What Option B does provide, by construction, is a *guaranteed upper bound on the severity of unsafe behaviour* at every stage of training and deployment, together with a fully interpretable component (the shield) whose decisions can be logged, audited, and explained independently of the neural network.

The ethical trade-off.

Stated in its simplest form, the choice between Option A and Option B is a choice between *optimising for performance under an implicit assumption that safety will emerge*, and *optimising for performance under an explicit constraint that safety is guaranteed by design*. The first option treats safety as a metric to be learned, while the second treats safety as a precondition that is never violated, regardless of what the learned component has or has not yet learned. From an engineering ethics standpoint, these are not equivalent options separated by a small performance gap: they represent two fundamentally different attitudes toward the question of who, or what, is responsible for preventing harm.

12.3. JUSTIFICATION OF THE SELECTED DECISION

The Shielded-RL architecture (Option B) was selected, and this decision is justified along the three axes specified for this assessment.

Engineering professional ethics.

Professional codes of conduct for engineers (including those referenced by the *Colegio de Ingenieros* bodies and by international bodies such as the IEEE and ACM) consistently place the protection of public health, safety, and welfare above all other professional obligations, including those toward employers, clients, or the advancement of the technology itself. In the context of this project, that principle translates directly into a design requirement: *no action that the simulated vehicle executes should be allowed to cause a collision or an off-road excursion if a safe alternative exists and can be computed in real time*. The

`CarlaAdaptiveHorizonShield` implements this requirement. Its risk classification (safe / warning / critical, Chapter 6) and its trajectory-prediction logic exist precisely so that the "first do no harm" principle is enforced at the level of the control loop, not merely hoped for as a statistical outcome of training. Choosing to build and integrate this component, rather than relying on the learned policy alone, is a direct application of the duty to prioritise public safety over project convenience or raw performance metrics.

Social impact and the role of deterministic guardrails in building trust.

Public acceptance of autonomous systems (whether autonomous vehicles or, more broadly, AI systems embedded in everyday infrastructure) depends heavily on the existence of *verifiable, deterministic guardrails* that bound the system's worst-case behaviour. This is the same principle that motivates safety filters and content guardrails around large language models: a system that is occasionally impressive but unboundedly unpredictable does not earn public trust, whereas a system whose worst-case behaviour is provably constrained, even if its average behaviour is more modest, can be reasoned about, certified, and ultimately trusted.

The shield architecture in this project plays exactly this role for the driving agent. Because the shield's corrective logic is rule-based and inspectable (kinematic predictions, explicit distance and time-to-collision thresholds), its behaviour can be explained to a non-specialist in terms of physical quantities ("the vehicle was predicted to leave its lane within N steps, so its steering was corrected"), rather than in terms of opaque network activations. This interpretability is itself a contribution to the social acceptability of learned autonomous controllers: it demonstrates that performance and verifiable safety are not mutually exclusive design goals, and that a learning system can be embedded inside, rather than substituted for, a deterministic safety framework.

Alignment with Deusto's values and the Sustainable Development Goals.

This project is also consistent with the broader institutional commitments of the University of Deusto, in particular the principles articulated in the *Declaración Deusto de Derechos Humanos en Entornos Digitales*, which calls for the design of digital and automated systems that actively safeguard human rights, including the right to life and physical integrity, rather than treating their protection as an afterthought. A driving system whose safety properties are designed in from the outset, and whose decisions remain auditable, is a direct application of that principle to the domain of autonomous mobility.

The project's goals also align with several of the United Nations Sustainable Development Goals:

- **SDG 3 (Good Health and Well-being):** road traffic collisions are a leading cause of injury and death

worldwide [1]. By developing and evaluating mechanisms that reduce the rate of crashes and off-road events during the learning process of an autonomous agent (Chapter 8), this project contributes, even at a research scale, to the broader goal of reducing traffic-related harm.

- **SDG 9 (Industry, Innovation, and Infrastructure):** the project advances the state of the art in safe reinforcement learning for autonomous driving, proposing and evaluating a concrete architecture (the adaptive horizon shield) that other researchers and practitioners can build upon, fostering responsible innovation in intelligent transport infrastructure.

- **SDG 11 (Sustainable Cities and Communities):** autonomous vehicles are widely expected to play a role in future urban mobility systems. Safety mechanisms such as the one developed in this project are a prerequisite for the responsible integration of such vehicles into shared urban spaces, where pedestrians, cyclists, and other vehicles must coexist safely with autonomous agents.

Summary.

Taken together, these considerations support the conclusion that the Shielded-RL architecture is not merely a technical refinement of the baseline PPO agent, but the ethically appropriate design choice for this project. It reflects the professional duty to prioritise public safety over unconstrained optimisation, it embodies the kind of deterministic, interpretable guardrail that is increasingly recognised as a precondition for public trust in autonomous systems, and it situates the project within the human rights and sustainability commitments that frame the work of engineers trained at the University of Deusto.

BIBLIOGRAFÍA

[1] World Health Organization, “Global status report on road safety 2023,” World Health Organization, Geneva, Switzerland, Tech. Rep., 2023.

[2] SAE International, “Taxonomy and definitions for terms related to driving automation systems for on-road motor vehicles,” SAE International, Warrendale, PA, USA, Standard J3016, 2021.

[3] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.

[4] B. R. Kiran, I. Sobh, V. Talpaert, P. Mannion, A. A. Al Sallab, S. Yogamani, and P. Pérez, “Deep reinforcement learning for autonomous driving: A survey,” *IEEE Transactions on Intelligent Transportation Systems*, vol. 23, no. 6, pp. 4909–4926, 2022.

[5] S. Aradi, “Survey of deep reinforcement learning for motion planning of autonomous vehicles,” *IEEE Transactions on Intelligent Transportation Systems*, vol. 23, no. 2, pp. 740–759, 2022.

[6] J. García and F. Fernández, “A comprehensive survey on safe reinforcement learning,” *Journal of Machine Learning Research*, vol. 16, no. 42, pp. 1437–1480, 2015.

[7] M. Alshiekh, R. Bloem, R. Ehlers, B. Könighofer, S. Niekum, and U. Topcu, “Safe reinforcement learning via shielding,” in *Proc. 32nd AAAI Conf. Artificial Intelligence*, 2018, pp. 2669–2678.

[8] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.

[9] A. Dosovitskiy, G. Ros, F. Codevilla, A. López, and V. Koltun, “CARLA: An open urban driving simulator,” in *Proc. 1st Conf. Robot Learning (CoRL)*, 2017, pp. 1–16.

[10] R. Rajamani, *Vehicle Dynamics and Control*, 2nd ed. Boston, MA, USA: Springer, 2012.

[11] M. L. Puterman, *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. New York, NY, USA: John Wiley & Sons, 1994.

[12] R. S. Sutton, D. McAllester, S. Singh, and Y. Mansour, “Policy gradient methods for reinforcement learning with function approximation,” in *Advances in Neural Information Processing Systems*, vol. 12, 1999, pp.

1057–1063.

[13] V. Mnih, A. P. Badia, M. Mirza, A. Graves, T. P. Lillicrap, T. Harley, D. Silver, and K. Kavukcuoglu, “Asynchronous methods for deep reinforcement learning,” in *Proc. 33rd Int. Conf. Machine Learning (ICML)*, 2016, pp. 1928–1937.

[14] J. Schulman, S. Levine, P. Abbeel, M. I. Jordan, and P. Moritz, “Trust region policy optimization,” in *Proc. 32nd Int. Conf. Machine Learning (ICML)*, 2015, pp. 1889–1897.

[15] M. Andrychowicz, A. Raichuk, P. Stańczyk, M. Orsini, S. Girgin, R. Marinier, L. Hussenot, M. Geist, O. Pietquin, M. Michalski, S. Gelly, and O. Bachem, “What matters for on-policy deep actor-critic methods? a large-scale study,” in *Int. Conf. Learning Representations (ICLR)*, 2021.

[16] L. Engstrom, A. Ilyas, S. Santurkar, D. Tsipras, F. Janoos, L. Rudolph, and A. Mądry, “Implementation matters in deep policy gradients: A case study on PPO and TRPO,” in *Int. Conf. Learning Representations (ICLR)*, 2020.

[17] J. Schulman, “Approximating KL divergence,” http://joschu.net/blog/kl-approx.html, 2020, online technical note, accessed 2025.

[18] J. Schulman, P. Moritz, S. Levine, M. I. Jordan, and P. Abbeel, “High-dimensional continuous control using generalized advantage estimation,” in *Int. Conf. Learning Representations (ICLR)*, 2016.

[19] J. Achiam, D. Held, A. Tamar, and P. Abbeel, “Constrained policy optimization,” in *Proc. 34th Int. Conf. Machine Learning (ICML)*, 2017, pp. 22–31.

[20] A. Stooke, J. Achiam, and P. Abbeel, “Responsive safety in reinforcement learning by PID Lagrangian methods,” in *Proc. 37th Int. Conf. Machine Learning (ICML)*, 2020, pp. 9133–9143.

[21] N. Jansen, B. Könighofer, S. Junges, A. C. Serban, and R. Bloem, “Safe reinforcement learning using probabilistic shields,” in *Proc. 31st Int. Conf. Concurrency Theory (CONCUR)*, 2020, pp. 3:1–3:16.

[22] J. Kong, M. Pfeiffer, G. Schildbach, and F. Borrelli, “Kinematic and dynamic vehicle models for autonomous driving control design,” in *Proc. IEEE Intelligent Vehicles Symposium (IV)*, 2015, pp. 1094–1099.

[23] J. Chen, B. Yuan, and M. Tomizuka, “Model-free deep reinforcement learning for urban autonomous driving,” in *Proc. 22nd IEEE Int. Conf. Intelligent Transportation Systems (ITSC)*, 2019, pp. 2765–2771.

[24] K. Chitta, A. Prakash, B. Jaeger, Z. Yu, K. Renz, and A. Geiger, "TransFuser: Imitation with transformer-based sensor fusion for autonomous driving," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 45, no. 11, pp. 12 878–12 895, 2023.

[25] E. Leurent, "An environment for autonomous driving decision-making," https://github.com/eleurent/highway-env, 2018, gitHub repository, accessed 2025.

[26] M. Althoff, M. Koschi, and S. Manzinger, "CommonRoad: Composable benchmarks for motion planning on roads," in *Proc. IEEE Intelligent Vehicles Symposium (IV)*, 2017, pp. 719–726.

[27] H. Krasowski, X. Wang, and M. Althoff, "Safe reinforcement learning for autonomous lane changing using set-based prediction," in *Proc. 23rd IEEE Int. Conf. Intelligent Transportation Systems (ITSC)*, 2020, pp. 1–7.

[28] K. P. Wabersich and M. N. Zeilinger, "A predictive safety filter for learning-based control of constrained nonlinear dynamical systems," *Automatica*, vol. 129, p. 109597, 2021.

[29] Q. Li, Z. Peng, L. Feng, Q. Zhang, Z. Xue, and B. Zhou, "MetaDrive: Composing diverse driving scenarios for generalizable reinforcement learning," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 45, no. 3, pp. 3461–3475, 2023.

[30] Y. Bengio, J. Louradour, R. Collobert, and J. Weston, "Curriculum learning," in *Proc. 26th Int. Conf. Machine Learning (ICML)*, 2009, pp. 41–48.

[31] I. Akkaya, M. Andrychowicz, M. Chociej, M. Litwin, B. McGrew, A. Petron, A. Paino, M. Plappert, G. Powell, R. Ribas, J. Schneider, N. Tezak, J. Tworek, P. Welinder, L. Weng, Q. Yuan, W. Zaremba, and L. Zhang, "Solving Rubik's cube with a robot hand," *arXiv preprint arXiv:1910.07113*, 2019.

[32] ISO/IEC, "Systems and software engineering — systems and software quality requirements and evaluation (SQuaRE) — system and software quality models," International Organization for Standardization, Geneva, Switzerland, Standard 25010:2011, 2011.

[33] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *Proc. 35th Int. Conf. Machine Learning (ICML)*, 2018, pp. 1861–1870.

[34] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, "Stable-baselines3: Reliable reinforcement learning implementations," *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8,

2021.

[35] B. P. Welford, “Note on a method for calculating corrected sums of squares and products,” *Technometrics*, vol. 4, no. 3, pp. 419–420, 1962.

[36] SQLite Development Team, “Write-ahead logging (WAL),” https://www.sqlite.org/wal.html, 2024, sQLite technical documentation, accessed 2025.

[37] M. Towers, A. Kwiatkowski, J. Terry, J. U. Balis, G. De Cola, T. Deleu, M. G. ao, A. Kallinteris, M. Krimmel, A. KG, R. Perez-Vicente, A. Pierré, S. Schulhoff, J. J. Tai, H. Tan, and O. G. Younis, “Gymnasium: A standard interface for reinforcement learning environments,” *arXiv preprint arXiv:2407.17032*, 2024.

[38] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Z. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, “PyTorch: An imperative style, high-performance deep learning library,” in *Advances in Neural Information Processing Systems*, vol. 32, 2019, pp. 8024–8035.

[39] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *Int. Conf. Learning Representations (ICLR)*, 2015.

[40] Y. J. Ma, W. Liang, G. Wang, D.-A. Huang, O. Bastani, D. Jayaraman, Y. Zhu, L. Fan, and A. Anandkumar, “Eureka: Human-level reward design via coding large language models,” in *Proc. 12th Int. Conf. Learning Representations (ICLR)*, 2024.

[41] Gemma Team, T. Mesnard, C. Hardin, R. Dadashi, S. Bhupatiraju, S. Pathak, L. Sifre, M. Rivière, M. S. Kale, J. Love, P. Tafti, L. Hussenot *et al.*, “Gemma: Open models based on Gemini research and technology,” *arXiv preprint arXiv:2403.08295*, 2024.

[42] S. Ross, G. J. Gordon, and J. A. Bagnell, “A reduction of imitation learning and structured prediction to no-regret online learning,” in *Proc. 14th Int. Conf. Artificial Intelligence and Statistics (AISTATS)*, 2011, pp. 627–635.

[43] M. Klissarov, P. D’Oro, S. Sodhani, R. Raileanu, P.-L. Bacon, P. Vincent, A. Zhang, and M. Henaff, “Motif: Intrinsic motivation from artificial intelligence feedback,” arXiv preprint arXiv:2310.00166, 2023, available: https://arxiv.org/abs/2310.00166.

[44] P. F. Christiano, J. Leike, T. B. Brown, M. Martic, S. Legg, and D. Amodei, “Deep reinforcement learning

from human preferences," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017, available: https://arxiv.org/abs/1706.03741.

[45] A. Y. Ng, D. Harada, and S. Russell, "Policy invariance under reward transformations: Theory and application to reward shaping," in *Proceedings of the Sixteenth International Conference on Machine Learning (ICML)*, 1999, pp. 278–287.

[46] Z. Huang, Z. Sheng, Y. Qu, J. You, and S. Chen, "VLM-RL: A unified vision language models and reinforcement learning framework for safe autonomous driving," arXiv preprint arXiv:2412.15544, 2024, available: https://arxiv.org/abs/2412.15544.

[47] J. Zhang, C. Xu, and B. Li, "ChatScene: Knowledge-enabled safety-critical scenario generation for autonomous vehicles," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024, available: https://arxiv.org/abs/2405.14062.

[48] S. Bansal, M. Chen, S. Herbert, and C. J. Tomlin, "Hamilton–Jacobi reachability: A brief overview and recent advances," in *Proc. 56th IEEE Conf. Decision and Control (CDC)*, 2017, pp. 2242–2253.

[49] J. F. Fisac, A. K. Akametalu, M. N. Zeilinger, S. Kaynama, J. Gillula, and C. J. Tomlin, "A general safety framework for learning-based control in uncertain robotic systems," *IEEE Transactions on Automatic Control*, vol. 64, no. 7, pp. 2737–2752, 2019.

[50] Anthropic, "Claude Code: Agentic coding tool," Computer software, 2025, available: https://www.anthropic.com/claude-code.

[51] Gemini Team and Google, "Gemini: A family of highly capable multimodal models," arXiv preprint arXiv:2312.11805, 2023, available: https://arxiv.org/abs/2312.11805.

A. HYPERPARAMETERS AND DEFAULT CONFIGURATION

This appendix collects the hyperparameter values used in the experiments presented in this thesis, together with the constructor defaults of the core components. Where the experimental CLI flags differ from the code constructor defaults, a footnote marks the affected rows.

A.1. PPO AGENT

Table A.1.: Default PPO hyperparameters.

Parameter	**CLI flag / constructor**	**Experiment value***
Learning rate (initial)	`-lr`	1×10^{-4}
Learning rate (final)	internal	1×10^{-5}
LR schedule	internal	Cosine annealing
Discount factor γ	constructor	0.99
GAE λ	constructor	0.95
PPO clip ϵ	constructor	0.20
Value-loss coefficient	`-value_loss_coef`	0.50
Entropy coefficient (initial)†	`-entropy_coef`	0.001
Entropy coefficient (final)†	`-entropy_coef_min`	0.0
Entropy decay updates†	`-entropy_coef_decay_updates`	50
Rollout length	`-update_timestep`	$2\,048$
Minibatch size	`-minibatch_size`	64
PPO epochs per update	`-k_epochs`	10
Approximate-KL target†	`-kl_target`	0.08 (0 disables)
Max gradient ℓ_2-norm	constructor	0.5
Value clipping ξ	constructor	disabled
Hidden size	constructor	256
LIDAR encoder dims	constructor	$720 \to 256 \to 64$
$\log \sigma$ initial value	constructor	-1.5 ($\sigma \approx 0.22$)
$\log \sigma$ bounds (tanh-squash)	constructor	$[-3.0,\ -1.2]$ ($\sigma \in [0.05, 0.30]$)
Obs. normaliser clip	`RunningMeanStd`	$[-5, 5]$
Obs. normalisation enabled	`-obs-norm`	enabled (vector block only)
Return normalisation	internal	enabled (std only)

* CLI flags passed in the experiments presented in this thesis. Constructor defaults may differ. † Constructor defaults: `entropy_coef`=0.01, `entropy_coef_min`=0.005, `entropy_coef_decay_updates`=500, `kl_target`=0.05.

A.2. ENVIRONMENT

Table A.2.: Default environment hyperparameters.

Parameter	CLI flag / constructor	Default
CARLA host	`-host`	`localhost`
CARLA port	`-port`	2 000
Traffic Manager port	`-tm_port`	8 000
Map	`-map`	`Town04`
Number of NPC vehicles	`-num_npc`	40
Weather preset	`-weather`	`ClearNoon`
Target speed	`-target_speed_kmh`	50 km/h
Success distance	`-success_distance`	250 m
Synchronous mode	constructor	`True`
Simulator time step	constructor	0.05 s (20 Hz)
Number of LIDAR rays	constructor	240 (per channel)
LIDAR range	constructor	50 m
Max steps per episode	`-max_steps`	1 000
Training seed	`-seed`	42
Evaluation seed	constructor (eval)	100
Base success reward	constructor	+30
Base crash penalty	constructor	−10
Base off-road penalty	constructor	−30

A.3. REWARD SHAPER

Table A.3.: Default weights of the reward shaper. Symbols refer to the equations of Section 6.6.

Symbol	Description	CLI flag	Default
w_{v}	Speed-bonus scale	`-speed_weight`	0.20
w_{sm}	Smoothness penalty scale	`-smoothness_weight`	0.10
w_{lc}	Lane-centring scale	`-lane_centering_weight`	0.30
w_{h}	Heading-alignment scale	constructor	0.10
w_{inv}	Lane-invasion penalty	`-lane_invasion_penalty`	0.25
w_{edge}	Edge-warning scale	constructor	0.30
w_{prog}	Progress-milestone bonus	constructor	0.30
w_{wh}	Wrong-heading penalty	constructor	0.50
w_{road}	Off-road penalty (shaper)	`-off_road_penalty`	1.00
w_{sh}	Shield-intervention penalty	`-shield_intervention_penalty`	0.00
w_{idle}	Idle penalty scale	`-idle_penalty_weight`	0.25
w_{dr}	Drift penalty scale	constructor	0.08
c_{curv}	Curvature speed-target scale	constructor	0.4
η	Speed-limit tolerance	constructor	0.05
G	Shield grace-period length	constructor	10 steps
Δ_{prog}	Progress-milestone length	constant (`PROGRESS_MILESTONE_M`)	$25\,\mathrm{m}$

A.4. SAFETY SHIELDS

Table A.4.: Default hyperparameters of both shields. "Basic only" and "Adaptive only" columns indicate parameters that exist solely in one of the shields.

Parameter	CLI flag	Scope	Default
Front LIDAR threshold	`-front_threshold`	both	0.15
Side LIDAR threshold	`-side_threshold`	both	0.04
Lateral offset threshold	`-lateral_threshold`	both	0.65 (train), 0.82 (eval)
Heading threshold	constructor	basic only	0.60
Lane-correction gain	constructor	both	1.5
Emergency brake	constructor	both	-0.6
Pedestrian emergency dist.	constant	adaptive only	$4.0\,\mathrm{m}$
Risk-level horizons (H)	internal	adaptive only	$1, 5, 10$
Threshold multipliers (κ)	internal	adaptive only	$1.0, 1.5, 2.0$
Frontal-risk boundaries	internal	adaptive only	$0.50,\ 0.20$
Lateral-risk boundaries	internal	adaptive only	$0.50,\ 0.70$
Edge guard floor (`EDGE_GUARD_MIN_NORM`)	constant	adaptive only	0.15
Side clearance threshold (`IN_LANE_SAFE_THRESHOLD`)	constant	adaptive only	0.30

A.5. CURRICULUM MANAGER

Table A.5.: Default hyperparameters of the curriculum manager.

Parameter	CLI flag / constructor	Default
Curriculum enabled	`-curriculum` / `-no-curriculum`	enabled
Maximum NPC count	`-num_npc`	40
Number of stages	internal	5 (uniform in $[0, N_{\max}]$)
Min. episodes per stage	constructor	100
Rollback patience	constructor	50
Rollback counter decay	constructor	-1 per non-collapsing episode

A.6. BC TEACHER (SHIELD-AS-TEACHER, R2)

When the BC teacher is enabled (Chapter 9), three additional hyperparameters control the behavioural-cloning loss added on top of the PPO surrogate.

Table A.6.: BC-teacher hyperparameters (`ppo_agent.py` constructor and CLI flags).

Parameter	CLI flag / constructor	Default
BC loss weight, initial (λ_{BC})	`-shield_imitation_coef`	0.0 (disabled)
BC loss anneal updates	`-shield_imitation_anneal_updates`	150
Clone steering only	`-shield_imitation_steer_only`	`True`

The BC loss is applied only when $\lambda_{\mathrm{BC}} > 0$. It targets the steering dimension of the shield-corrected action and is weighted by the shield blending factor α_t:

$$\mathcal{L}_{\mathrm{BC}}(\theta) = \frac{\sum_t \alpha_t \left(\tanh\left(\mu_\theta^{\mathrm{steer}}(s_t)\right) - a_t^{\star,\mathrm{steer}}\right)^2}{\sum_t \alpha_t}.$$

The coefficient λ_{BC} is annealed linearly to zero over `anneal_updates` PPO gradient updates so that the shield acts as an early teacher but releases the policy as training matures.

A.7. LLM META-CONTROLLER

The LLM meta-controller (Chapter 9) comprises three components: an LLM-based permissiveness proposer, a deterministic safety governor that mediates the proposal, and a tunable-shield layer that maps the scalar permissiveness $p \in [0, 1]$ to concrete threshold values.

LLM tuner (`src/meta/llm_tuner.py`)

Table A.7.: LLM tuner constants.

Parameter	Value	Notes
Model	`gemma4:e4b`	Small quantised model via Ollama
Temperature	0.0	Fully deterministic
Seed	42	Reproducibility
`keep_alive`	0	Unload VRAM after each call
Request timeout	$600\,\mathrm{s}$	
Run interval	100 episodes	`should_run(ep) = ep % 100 == 0`

Safety governor (`src/meta/safety_governor.py`)

Table A.8.: Safety-governor `GovernorConfig` defaults.

Parameter	Value	Role
`crash_ceiling`	0.10	Force-tighten if crash rate exceeds this
`offroad_ceiling`	0.15	Force-tighten if off-road rate exceeds this
`crash_target`	0.05	Safe-to-loosen threshold
`offroad_target`	0.08	Safe-to-loosen threshold
`max_step`	0.10	Max $\lvert\Delta p\rvert$ per voluntary change
`force_tighten_step`	0.20	Asymmetric rapid-tighten step size
`dwell_episodes`	100	Min. episodes to wait between loosening changes

Tunable-shield weanable endpoints (`src/meta/tunable_shield.py`)

Each weanable parameter is interpolated linearly between a *strict* value (enforced at $p = 1$) and a *permissive* value (reached at $p = 0$): $\tau_i(p) = \tau_i^{\text{strict}} + (1 - p)(\tau_i^{\text{perm}} - \tau_i^{\text{strict}})$.

Table A.9.: Weanable threshold endpoints.

Parameter	Strict ($p = 1$)	Permissive ($p = 0$)
front_threshold_base	0.15	0.08
side_threshold_base	0.02	0.008
LATERAL_WARNING_OFFSET	0.50	0.75
LATERAL_CRITICAL_OFFSET	0.70	0.90
HEADING_WARNING_DEG	10.0°	20.0°
HEADING_CRITICAL_DEG	20.0°	35.0°
MAX_HEADING_DEV_RAD	0.52 rad	0.80 rad
MAX_LATERAL_DRIFT_M	1.75 m	2.50 m
horizon.warning.threshold_multiplier	1.5	1.0
horizon.critical.threshold_multiplier	2.0	1.0
horizon.safe.lateral_thr	0.55	0.75
horizon.warning.lateral_thr	0.40	0.60
horizon.critical.lateral_thr	0.30	0.50

The following parameters are invariant (INVARIANT_PARAMS) and are never modified by the weaning mechanism: PED_EMERGENCY_M, EDGE_GUARD_MIN_NORM, STALL_SPEED_KMH, STALL_RECOVERY_EXIT_KMH, STALL_PATIENCE, STALL_RECOVERY_THROTTLE, LATERAL_RECOVERY_THROTTLE, emergency_brake.

B. OBSERVATION SPACE BREAKDOWN

This appendix describes every one of the 739 dimensions of the observation vector s_t. Indices are zero-based and match the slicing conventions used in `src/CARLA/Env/carla_env.py`.

B.1. LIDAR BLOCK (DIMENSIONS 0–719)

The first 720 dimensions contain three semantic LIDAR channels of 240 rays each. Each ray is normalised to $[0, 1]$, where 1 means a clear ray over the whole $50\,\mathrm{m}$ range and values close to 0 mean an obstacle within $\approx 1\,\mathrm{m}$. Ray $k \in \{0, \ldots, 239\}$ points towards the azimuth angle $\phi_k = k \cdot 1.5^\circ$ measured counter-clockwise from the vehicle's forward direction.

Table B.1.: LIDAR block of the observation vector.

Indices	Channel	Description
$[0, 240)$	Combined	All obstacles (vehicles, pedestrians, static, road edges).
$[240, 480)$	Dynamic	Only vehicles and pedestrians.
$[480, 720)$	Static	Only walls, barriers, poles and similar static objects.

B.2. LANE FEATURES (DIMENSIONS 720–727)

The next 8 dimensions are lane features computed from CARLA's Waypoint API at the vehicle's current position.

Table B.2.: Lane features.

Idx.	Name	Range	Meaning
720	`lateral_offset_norm`	$[-1, 1]$	Signed lateral offset of the vehicle with respect to the lane centre, normalised by the lane half-width.
721	`heading_error_norm`	$[-1, 1]$	Signed heading error between the vehicle and the lane tangent, normalised by 180°.
722	`on_edge_warning`	$[0, 1]$	Continuous warning that grows quadratically as the closest lane edge approaches.
723	`lane_width_norm`	$[0, 1]$	Lane width at the current position, normalised by a reference width of $4.5\,\mathrm{m}$.
724	`dist_left_edge_norm`	$[0, 1]$	Distance to the left lane edge, normalised by the lane width.
725	`dist_right_edge_norm`	$[0, 1]$	Distance to the right lane edge, normalised by the lane width.
726	`lane_change_left`	$\{0, 1\}$	1 if a left-side lane change is permitted at the current position (only the left direction is exposed).
727	`road_curvature_norm`	$[-1, 1]$	Estimated signed road curvature, normalised by a reference curvature.

B.3. LANE-MARKING TYPE (DIMENSIONS 728–731)

Four binary flags classify the lane-marking type on each side of the vehicle, derived from CARLA's `LaneMarkingType` at the current position.

Table B.3.: Lane-marking type features.

Idx.	Name	Range	Meaning
728	`solid_left`	$\{0, 1\}$	1 if the left lane marking is solid (crossing is forbidden).
729	`solid_right`	$\{0, 1\}$	1 if the right lane marking is solid.
730	`dashed_left`	$\{0, 1\}$	1 if the left marking is dashed (lane change permitted).
731	`dashed_right`	$\{0, 1\}$	1 if the right marking is dashed.

These flags allow the policy to distinguish legal lane changes (dashed marking) from illegal solid-line crossings without relying on CARLA's lane-invasion event alone.

B.4. VEHICLE STATE (DIMENSIONS 732–733)

Table B.4.: Vehicle state features.

Idx.	Name	Range	Meaning
732	`speed_norm`	$[0, 1]$	Vehicle speed normalised by a dynamic reference: $\max(\text{speed\_limit} \times 1.5,\ 10\,\text{km/h})$.
733	`steering`	$[-1, 1]$	Last steering command actually applied to the vehicle (after shield correction).

B.5. ROUTE INFORMATION (DIMENSIONS 734–738)

The final 5 dimensions provide look-ahead information along the route graph.

Table B.5.: Route features.

Idx.	Name	Range	Meaning
734	`next_wp_angle_norm`	$[-1, 1]$	Signed angle to the next waypoint $5\,\mathrm{m}$ ahead, normalised by 180°.
735	`wp_angle_20m_norm`	$[-1, 1]$	Signed angle to the waypoint $20\,\mathrm{m}$ ahead, normalised by 180° (look-ahead for smooth turning).
736	`progress_norm`	$[0, 1]$	Fraction of the success distance traversed so far during the current episode.
737	`speed_limit_norm`	$[0, 1]$	Current speed limit normalised by the reference speed.
738	`speed_ratio`	$[0, 2]$	Ratio of the vehicle speed to the current speed limit (clipped at 2).

B.6. SUMMARY

The full observation vector has the structure

$$s_t = \underbrace{\left[\,\mathsf{LIDAR}_t^{\mathsf{comb}}\,\right]}_{240} \,\|\, \underbrace{\left[\,\mathsf{LIDAR}_t^{\mathsf{dyn}}\,\right]}_{240} \,\|\, \underbrace{\left[\,\mathsf{LIDAR}_t^{\mathsf{stat}}\,\right]}_{240} \,\|\, \underbrace{\left[\,\mathsf{Lane}_t\,\right]}_{8} \,\|\, \underbrace{\left[\,\mathsf{Marking}_t\,\right]}_{4} \,\|\, \underbrace{\left[\,\mathsf{Veh}_t\,\right]}_{2} \,\|\, \underbrace{\left[\,\mathsf{Route}_t\,\right]}_{5} \in \mathbb{R}^{739}.$$

Only the rightmost 19 dimensions (lane, marking, vehicle and route) are normalised online by the running-mean estimator of Section 6.4.3; the 720 LIDAR dimensions are already in $[0, 1]$ by construction and are left untouched.

C. REPRODUCIBILITY GUIDE

This appendix lists the exact commands needed to reproduce every experiment reported in Chapter 8, together with the SQL queries used to extract aggregate statistics from the per-run SQLite databases.

C.1. ENVIRONMENT SETUP

1. Install the CARLA 0.9.16 binary distribution. On Windows, launch the server with the command shown in Figure C.1.

2. Activate the project virtual environment (for example, the shared venv at `C:\Users\Isaac\Documents\TFG\Shielded-RL\TFG-RL`).

3. Verify that the Python dependencies are installed (PyTorch with CUDA support, Gymnasium, NumPy, Matplotlib, and Streamlit). SQLite is part of the Python standard library and requires no separate installation.

repro_launch

```
.\CARLA_0.9.16\CarlaUE4.exe -RenderOffScreen -carla-port=2000
```

Figure C.1.: CARLA server launch command. The `-RenderOffScreen` flag disables the GPU renderer for headless training; `-carla-port=2000` sets the TCP port.

C.2. TRAINING COMMANDS

Figure C.2 shows the commands used to reproduce the three training configurations of Chapter 8.

repro_training

```
# Baseline: PPO without any shield
python main_train.py --model_name baseline --shield_type none

# Basic shield
python main_train.py --model_name run_basic --shield_type basic

# Adaptive shield (recommended)
python main_train.py --model_name run_adaptive --shield_type adaptive

# Curriculum ablation (adaptive shield, curriculum disabled)
python main_train.py --model_name run_adaptive_nocur \
--shield_type adaptive --no-curriculum
```

Figure C.2.: Training commands for the baseline (no shield), basic shield, adaptive shield and curriculum-ablation configurations. Each run writes metrics to `runs/<model>_<shield>_<timestamp>/metrics.sqlite` and checkpoints to `data/models/<model>_<shield>_{best,final}.pth`.

C.3. EVALUATION COMMANDS

Figure C.3 shows the evaluation commands, both in headless mode (metrics only) and with the Matplotlib dashboard.

repro_evaluation

```
# Headless, deterministic evaluation (the deployed policy), 50 episodes
python main_eval.py --model_name run_adaptive_adaptive_final.pth \
    --shield_type adaptive --episodes 50

# Shield-dependency ablation in one command (same scenarios per episode):
python main_eval.py --model_name run_adaptive_adaptive_final.pth \
    --shield_type none adaptive --episodes 50 --out ablation.json

# Interactive run with CARLA camera + Matplotlib dashboard (opt-in):
python main_eval.py --model_name run_adaptive_adaptive_final.pth \
    --shield_type adaptive --render --dashboard --episodes 10
```

Figure C.3.: Evaluation commands. The shield configuration must match the one used during training, as a policy trained with a shield has learned to rely on its interventions.

C.4. LIVE DASHBOARD

Figure C.4 shows how to launch the Streamlit dashboard. The dashboard reads every `runs/*/metrics.sqlite` found under the project root and allows the user to select the run(s) to visualise. Because the database is opened in WAL mode, the dashboard can be launched while a training run is still in progress.

repro_dashboard

```
streamlit run dataVisualizer.py
```

Figure C.4.: Streamlit dashboard launch command.

C.5. QUERY RECIPES

The following queries document how the empirical numbers in the tables of Chapter 8 are computed from the SQLite databases.

repro_sql_rates

```sql
SELECT AVG(value) FROM metric_events
 WHERE axis = 'episode' AND metric_name = 'Reward/Average_100_Episodes'
   AND step >= (SELECT MAX(step) - 99 FROM metric_events
                WHERE axis = 'episode'
                  AND metric_name = 'Reward/Average_100_Episodes');

SELECT AVG(value) FROM metric_events
 WHERE axis = 'episode' AND metric_name = 'Training/Crash_Rate'
   AND step >= (SELECT MAX(step) - 99 FROM metric_events
                WHERE axis = 'episode' AND metric_name = 'Training/Crash_Rate');

SELECT AVG(value) FROM metric_events
 WHERE axis = 'episode' AND metric_name = 'Training/Offroad_Rate'
   AND step >= (SELECT MAX(step) - 99 FROM metric_events
                WHERE axis = 'episode' AND metric_name = 'Training/Offroad_Rate');
```

Figure C.5.: SQL queries for the final 100-episode rolling reward and safety rates (crash and off-road). Replace the `key` strings with the metric of interest.

repro_sql_risklevels

```sql
SELECT metric_name, AVG(value) AS mean_fraction
  FROM metric_events
 WHERE axis = 'episode'
   AND metric_name IN ('Safety/Semantic/Safe_Step_Rate',
                       'Safety/Semantic/Warning_Step_Rate',
                       'Safety/Semantic/Critical_Step_Rate')
 GROUP BY metric_name;
```

Figure C.6.: SQL query for the fraction of steps spent in each risk level (adaptive shield only).

repro_sql_interventions

```sql
SELECT metric_name, AVG(value) AS mean_count
  FROM metric_events
 WHERE axis = 'episode'
   AND metric_name IN ('Safety/Semantic/Dynamic_Interventions',
                       'Safety/Semantic/Static_Interventions',
                       'Safety/Semantic/Pedestrian_Interventions')
 GROUP BY metric_name;
```

Figure C.7.: SQL query for the average number of shield interventions per episode by threat category (adaptive shield only).

repro_sql_curriculum

```sql
WITH ordered AS (
  SELECT step, value,
         LAG(value) OVER (ORDER BY step) AS prev_value
    FROM metric_events
   WHERE axis = 'episode' AND metric_name = 'Training/Curriculum_NPC'
)
SELECT step AS episode,
       CASE WHEN value > prev_value THEN 'advance'
            WHEN value < prev_value THEN 'rollback'
       END AS event,
       prev_value AS from_npcs,
       value      AS to_npcs
  FROM ordered
 WHERE value <> prev_value;
```

Figure C.8.: SQL query for curriculum stage transitions. The `LAG` window function identifies advance and rollback events by comparing consecutive NPC counts.

C.6. CHECKPOINT INSPECTION

The utility class `ModelManager` in `utils/` exposes helpers to list, compare and inspect checkpoints. Figure C.9 shows a quick sanity-check invocation.

repro_modelmanager

```python
from utils import ModelManager
ModelManager.list_models()
ModelManager.print_model_info('./data/models/run_adaptive_adaptive_final.pth')
```

Figure C.9.: Checkpoint inspection with `ModelManager`. The `print_model_info` call reports the network architecture, checkpoint format version, training seed, and the state of the running-mean normaliser.

C.7. UNIT TESTS

The reward shaper tests (Figure C.10) run without a CARLA server. They exercise each component of the shaped reward of Equation (D.3) against a crafted `info` dictionary and ensure that disabling a weight zeroes out the corresponding component. They are run locally before every commit that touches `src/reward_shaper.py`.

repro_pytest

```
python -m pytest tests/test_reward_balance.py -v
```

Figure C.10.: Running the reward-shaper unit tests. The suite completes in under one second and requires no running CARLA server.

D. MATHEMATICAL FORMULATION

For readability, several quantities in the main text are presented through figures and tables rather than equations. This appendix reproduces their exact closed forms for reference, with every symbol defined. In each case the figure cited in the main text is the primary, intuition-carrying presentation, and the equations below are its algebraic counterpart.

D.1. RISK-LEVEL CLASSIFICATION (ADAPTIVE SHIELD)

The frontal-distance criterion of the adaptive shield (Section 6.5.2, Figure 6.5 and Table 6.1) maps the minimum of the dynamic-only LIDAR front sector $d_t^{\mathrm{dyn}} \in [0, 1]$ to a discrete risk level:

$$\mathrm{level}_t^{\mathrm{frontal}} = \begin{cases} \mathrm{safe} & \text{if } d_t^{\mathrm{dyn}} > 0.50, \\ \mathrm{warning} & \text{if } 0.20 < d_t^{\mathrm{dyn}} \leq 0.50, \\ \mathrm{critical} & \text{if } d_t^{\mathrm{dyn}} \leq 0.20, \end{cases} \tag{D.1}$$

where a higher d_t^{dyn} denotes a more distant in-path obstacle (1 is a clear lane). The lateral-position criterion applies the analogous thresholds of Table 6.1 to the absolute lateral offset $|\ell_t|$ and the absolute heading error $|\psi_t|$; the overall risk level is the worst (most severe) of the two criteria.

D.2. LANE-KEEP GATE (REWARD SHAPER)

The lane-keep gate of Section 6.6, visualised in Figure 6.8, multiplies the forward-progress bonuses by

$$\mu_t = \underbrace{\max(1 - |\ell_t|,\, 0)}_{\text{centering factor}} \times \underbrace{\max\left(1 - \frac{|\psi_t|}{20^\circ},\, 0\right)}_{\text{heading factor}} \in [0, 1], \tag{D.2}$$

where ℓ_t is the normalised lateral offset (0 at the lane centre, ± 1 at the edges) and ψ_t is the heading error in degrees. The gate equals 1 only when the vehicle is both centred and aligned, and 0 as soon as $|\ell_t| \geq 1$ or $|\psi_t| \geq 20^\circ$. During a permitted lane change ($\ell_t > 0.5$ with CARLA reporting the change allowed) μ_t is floored at 0.3.

D.3. SHAPED REWARD (REWARD SHAPER)

The total shaped reward of Section 6.6, summarised in Figure 6.7 and Table 6.3, is the sum of the sparse base reward and the dense bonus and penalty terms:

$$\begin{aligned}\tilde{r}_t \;=\; & r_t^{\text{base}} + r_t^{\text{prog}} + r_t^{\text{accel}} + r_t^{\text{speed}} + r_t^{\text{lane}} + r_t^{\text{head}} + r_t^{\text{mile}} \\ & - p_t^{\text{smooth}} - p_t^{\text{inv}} - p_t^{\text{solid}} - p_t^{\text{edge}} - p_t^{\text{wrong-h}} - p_t^{\text{idle}} - p_t^{\text{drift}} - p_t^{\text{lc}} - p_t^{\text{shield}}.\end{aligned} \tag{D.3}$$

Each symbol is defined in Table 6.3; the default weights are listed in Appendix A, Table A.3.

D.4. SHIELD PERMISSIVENESS (LLM META-CONTROLLER)

The single permissiveness scalar $p \in [0, 1]$ of Section 9.3, illustrated in Figure 9.1, sets every tunable threshold τ_i of the adaptive shield by linear interpolation between a strict and a permissive endpoint:

$$\tau_i(p) \;=\; \tau_i^{\text{strict}} \;+\; (1-p)\,\big(\tau_i^{\text{perm}} - \tau_i^{\text{strict}}\big), \tag{D.4}$$

where τ_i^{strict} and τ_i^{perm} are the strict and permissive endpoints of threshold i. Thus $p = 1$ recovers the production shield (every $\tau_i = \tau_i^{\text{strict}}$) and $p = 0$ the maximally weaned shield (every $\tau_i = \tau_i^{\text{perm}}$).

E. TRANSPARENT USE OF AI

Figure E.1.: Disclosure banner indicating that parts of this project's development and documentation were produced with the assistance of AI tools, under continuous human supervision.

In line with the growing institutional expectation that academic work disclose the use of generative AI tools, this appendix documents how, where, and why such tools were employed during the development of *CALA_SafeRL*, the analysis of experimental results, and the writing of this memoir.

Human oversight statement.

It must be stated explicitly that every output produced with the assistance of an AI tool, whether source code, data analysis, or written text, was reviewed, tested, and revised by the author before being incorporated into the project or this document. AI tools were used strictly in an assistive capacity, as accelerators for routine or exploratory work, debugging aids, and proofreading support. At no point did an AI tool act as an autonomous decision-maker with respect to the project's scientific conclusions, the design of the safety shield, or the interpretation of results. Code suggested by an assistant was run and inspected by the author, and quantitative claims were cross-checked against the underlying SQLite metric databases before being reported. This principle of continuous human oversight is reiterated at the end of this appendix.

E.1. CLAUDE CODE AS A PROGRAMMING ASSISTANT

Claude Code [50] is an agentic command-line coding assistant developed by Anthropic, used as the primary programming assistant during implementation and experimentation. It operated directly inside the repository, with access to the source tree, the test suite, and the per-run metrics database.

Debugging the wrapper chain.

The layered `gymnasium` wrapper chain connecting the PPO agent to the CARLA simulator (Chapter 6) is a frequent source of subtle bugs: the agent interacts with `CarlaRewardShaper`, which wraps the active safety shield, which in turn wraps `CarlaEnv`. An error in how one layer modifies the action, observation, or reward can surface as a confusing symptom several layers away, for example a mismatch between the policy's log-probabilities and the action actually executed in the simulator (`info["executed_action"]`).

The typical workflow was to describe a symptom in plain language, ask Claude Code to trace the relevant code path across the wrapper boundaries, and then verify the proposed explanation manually against the source code and the relevant unit test in `tests/` before applying and re-testing a fix. This four-step pattern (describe, trace, verify, fix) was repeated for most of the non-trivial bugs in the shield credit-assignment logic and in the continuous action-space handling.

Continuous action space and curriculum learning.

Debugging the saturation of `log_std` (the effective policy standard deviation becoming pinned at its upper bound during early training) required correlating per-update telemetry (`log_std_steering_raw`, `log_std_throttle_raw`, `log_std_saturated_fraction`) with the entropy-coefficient schedule and the `tanh`-squashed parameterisation in `ActorCritic.log_std()`. Claude Code was used to write throwaway scripts that queried `metrics.sqlite` and plotted these quantities across runs, which sped up forming and discarding hypotheses. The resulting decision, to replace the hard clamp with the smooth squashed parameterisation described in Chapter 6, was the author's. The assistant's role was limited to producing the exploratory tooling.

A similar use applied to the multi-phase curriculum in `train_curriculum.py`, where each phase warm-starts from the previous phase's checkpoint. Claude Code helped compare checkpoint metadata (`obs_normalizer`, `ret_rms`, `returns_acc`) across phase boundaries, ruling out checkpoint-compatibility issues and focusing the investigation on the hyperparameter changes themselves.

Large-scale analysis of PPO training metrics.

Each training run produces per-episode and per-update SQLite databases (Chapter 6), and many model variants were trained across shield configurations, curriculum settings, and reward-shaping parameters. Claude Code drafted SQL queries and analysis scripts to aggregate crash rate, off-road rate, mean reward, and shielded

fraction across runs, producing the comparison tables used in Chapter 8. As before, every aggregate figure reported in this memoir was independently spot-checked by the author against the raw databases.

E.2. OLLAMA WITH GEMMA AS A LOCAL, PRIVACY-PRESERVING ASSISTANT

A second class of tasks benefited from running entirely on local hardware, using *Ollama* to host a *Gemma* [41] model.

Motivation.

Two reasons motivated a local deployment. First, the per-run `metrics.sqlite` databases contain project-specific experimental data, and keeping their analysis on the local machine avoids any ambiguity about data retention by a third party. Second, the `dataVisualizer.py` dashboard (Chapter 6) provides live analysis while training is in progress, and a local model avoids the latency and connectivity requirements of a remote API during long unattended runs.

Local analysis of training metrics.

The dashboard optionally invokes the local Gemma model to narrate the Plotly charts rendered from the live database, for example summarising whether a run's crash-rate curve has plateaued or shows the rollback behaviour implemented by `CurriculumManager`. The prompts include only numerical summaries already computed locally (rolling crash/off-road rates, mean rewards, curriculum stage transitions), not raw trajectory data. These summaries were treated as a first pass to flag runs for closer inspection, never as a substitute for the quantitative analysis in Chapter 8.

Collision-type categorisation.

A second, ongoing use of the local Gemma deployment is the categorisation of collision events for the planned LLM-based reward-exposure work (`tex/08_llm_reward_exposure.tex`, in preparation). The goal is to classify recorded collision episodes into semantically meaningful categories, for example distinguishing a collision with an oncoming vehicle during a forced lane change from one with a stationary obstacle near the road edge, in order to inform the preference-based reward signal described in that chapter. Running this locally allows the pipeline to be iterated on quickly without per-call costs and keeps the per-collision context on the local machine. The categories proposed by the model are a starting point for the preference-annotation pipeline and remain subject to manual spot-checking, a validation step that is part of the work still to be completed.

E.3. GEMINI AS A WRITING AND PROOFREADING ASSISTANT

Gemini [51], developed by Google, was used during the writing of this memoir as a structural and proofreading assistant, confined to the presentation layer of the document.

Draft sections, written first by the author based directly on the implementation and on the results in Chapter 8, were passed to Gemini with requests such as checking clarity and terminology consistency, or suggesting a clearer paragraph structure without changing the technical content. Suggestions concerned sentence-level clarity, consistent use of terms such as “safety shield”, the ordering of paragraphs, and minor grammatical corrections.

Every suggestion was reviewed by the author, who verified that no rewording altered the technical meaning of a statement, introduced unsupported claims, or removed hedging language around still-open questions, such as the work-in-progress status of the collision-categorisation pipeline described in Section E.2. Gemini therefore improved readability and consistency, but did not influence the technical content, argumentation, or conclusions of this memoir.

E.4. SUMMARY OF HUMAN OVERSIGHT

Three AI tools were used during this project, each in a distinct and bounded role: Claude Code [50] as a programming and data-analysis assistant on the source code and metric databases; a locally-hosted Gemma model [41], served through Ollama, for live metric narration and, in ongoing work, collision categorisation; and Gemini [51] as a proofreading and structural-editing assistant for this memoir. In every case the role was assistive, accelerating routine or exploratory tasks without making design decisions, determining experimental conclusions, or producing final text or code without human review. All code, all quantitative results, and all written content in this project and in this memoir were ultimately validated, tested, and approved by the author, who takes full responsibility for their correctness and content.